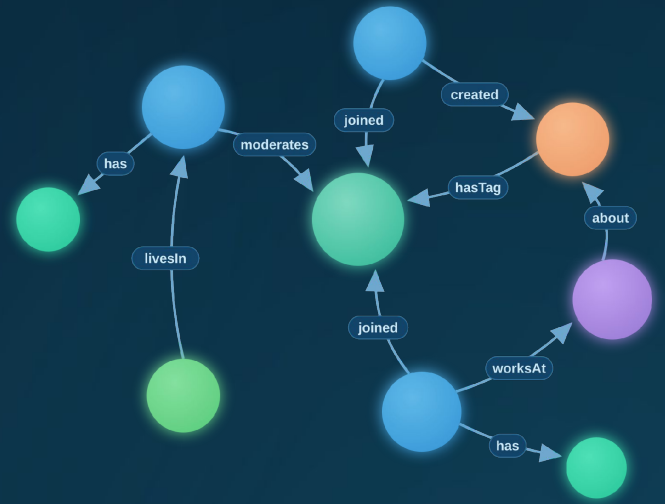
ISWC 2026 · HALF-DAY TUTORIAL

Semantic-Aware Partitioning of Property Graphs



From structure-only cuts to meaning-driven distribution models, algorithms, systems, evaluation, and open problems.

Elisjana Ymeralli[1] · Georgia. Troullinou[1] · Stefania Dumbrava[2] · Haridimos Kondylakis[1]

[1] Institute of Computer Science, ICS-FORTH, UOC, Greece

[2] ENSIIE & SAMOVAR – Télécom SudParis, France

INTRODUCTION

Introduction & Motivation

Presenters, why semantics matter when partitioning, learning outcomes and programme

Why This Tutorial?

1 Graphs outgrow a single machine

- Social, knowledge and financial graphs reach billions of nodes and edges
- They must be split across machines for storage, query processing and reasoning
- Classical partitioners optimise structure only: edge cut and load balance

2 Property graphs carry meaning

- Keys, graph functional dependencies and graph generating dependencies
- These constraints govern identity, consistency and inference
- A single constraint often spans several vertices and edges

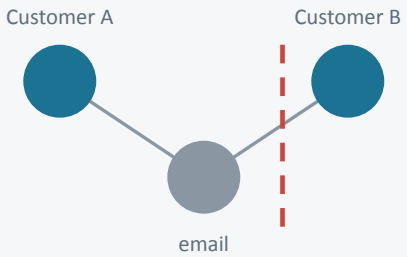
The gap: a purely structural cut can split a constraint across machines, so checking it or reasoning with it needs data from more than one partition.

The Problem in One Picture

A structural cut sees edges; meaning lives in groups of nodes and edges

Identity

Is it the same entity?

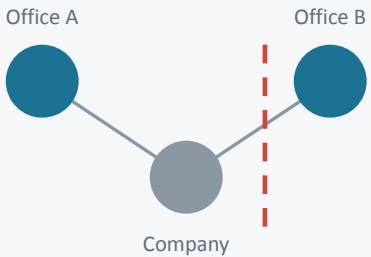


Two customers share one email: are they the same person?

Key check needs both machines

Consistency

Do the values agree?

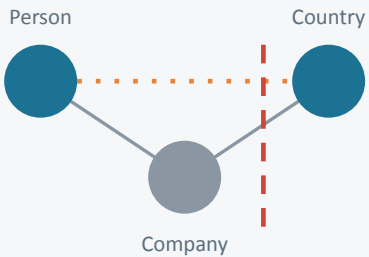


Two offices of one company must list the same headquarters country.

Dependency check needs both machines

Inference

What new fact follows?



Works at a company located in a country ⇒ lives in that country.

Rule needs both machines

This tutorial: partition property graphs so that the data each key, dependency and rule needs stays on one machine, while the load stays balanced.

Why Now, and Why at ISWC?

Standards are here

Graph Query Language (GQL)
ISO/IEC 39075:2024

SQL/Property Graph Queries (PGQ)
ISO/IEC 9075-16:2023

Already in systems
Neo4j: most mandatory GQL features (Cypher 25, 2026)
TigerGraph: GQL patterns inside GSQL (2025)

Constraints and schemas have matured

PG-Keys
keys for property graphs (2021)

PG-Schema
schemas for property graphs (2023)

Common foundations
PG-Schema, Shapes Constraint Language and Shape Expressions (2025)

The same questions as the Semantic Web

Data placement
already a concern for RDF graphs and SPARQL query processing

Constraint preservation
the same issue for shapes and validation

A gap
no work uses graph constraints to guide property graph partitioning

Learning Outcomes

After the tutorial, attendees will be able to:

1. **Describe** the basics of property graphs and how they relate to RDF
2. **Explain** graph partitioning models, algorithms and quality metrics
3. **Understand** how constraint-aware partitioning preserves semantic context
4. **Measure** partition quality with both structural and semantic metrics
5. **Identify** open challenges in partitioning, reasoning and validation

Tutorial Structure

P1

Property Graphs

The model, querying with patterns, schemas, and how they relate to RDF

P2

Foundations of PG Partitioning

Why we partition, how we measure it, and the classical structural methods

P3

Semantic-Aware Partitioning

Labels, schemas and constraints guide placement; evaluation metrics and a survey of algorithms

P4

Hands-on Demonstration & Open Challenges

capPG in action, then open problems in constraint-aware partitioning

Tutorial Programme

Session 1 · 9:00 – 10:40

9:00 – 9:10	Introduction & Motivation	10 min
9:10 – 9:50	Part 1 · Property Graphs	40 min
9:50 – 10:30	Part 2 · Foundations of Graph Partitioning	40 min
10:30 – 10:40	Questions & Discussion	10 min

Coffee break 10:40 – 11:10 · 30 min

Session 2 · 11:10 – 12:50

11:10 – 11:15	Recap of the morning	5 min
11:15 – 11:55	Part 3 · Semantic-Aware Partitioning	40 min
11:55 – 12:30	Part 4 · Demonstration & Open Challenges	35 min
12:30 – 12:50	Questions & Discussion	20 min

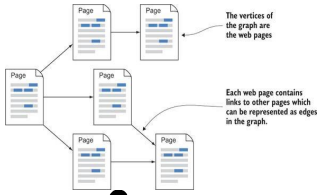
PART 1

Property Graphs

The property graph model, query languages, schemas and mappings to RDF

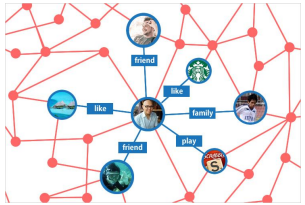


Graph Data Processing Evolution



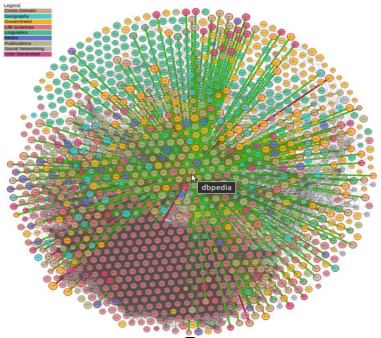
1990s

The World Wide Web



2000s

Social Networks



2010s

Knowledge Graphs (RDF triple stores)



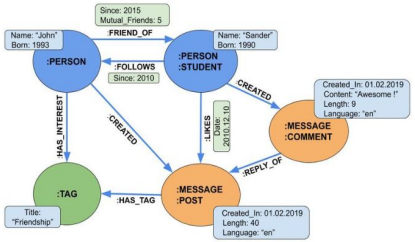
2020s

Graph Databases (Labeled Property Graphs)

ISO/IEC JTC1 SC32 WG3 → SQL/PGQ, GQL

Liaison

LDBC Property Graph Schema Working Group → PG-Keys, PG-Schema



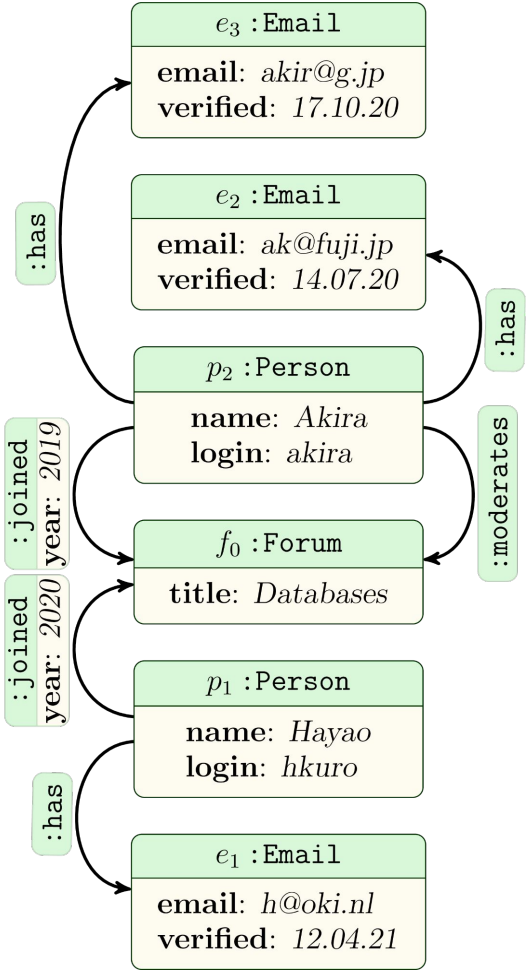
Why Property Graphs?

Graphs & graph analytics are ubiquitous

- Social networks
- Bio- & chemical networks
- Logistics, transportation, smart cities
- Communication & financial networks
- Crime networks
- Knowledge graphs

The property-graph (PG) model is becoming a standard

- Node- & edge-labeled directed graph
- Nodes and edges carry properties
- i.e. attribute–value pairs attached to any element



Knowledge Graphs (KGs)

Organizing and processing large volumes of varied, multi-source data

Heterogeneous data management

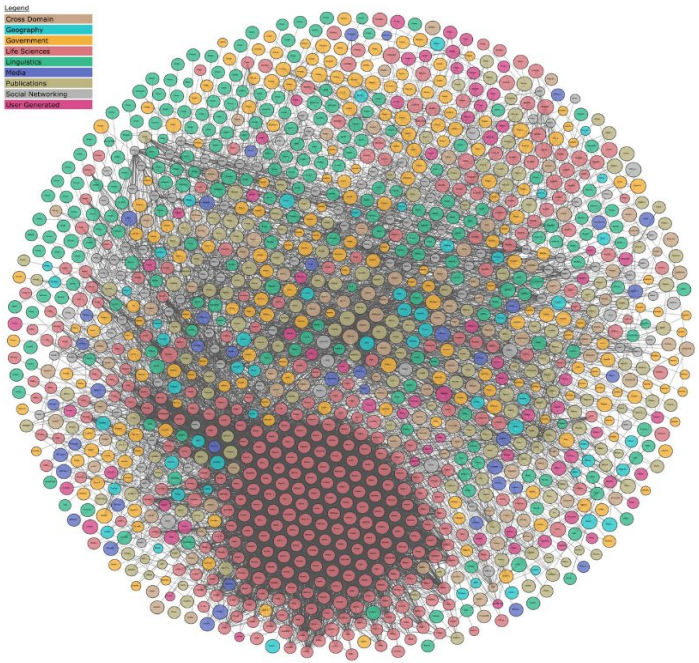
Unify multi-source data ⟶ efficient management & pattern detection

Semantic search

Enhance search with context ⟶ more relevant, precise results

AI & ML

Structured, rich source for Retrieval-Augmented Generation (RAG) ⟶ better accuracy & reliability



Property graphs vs. RDF



RDF / Semantic Web

- Everything is a triple ⟨s, p, o⟩
- Schema/ontology via RDFS & OWL (classes, hierarchy)
- Queried with SPARQL, join-centric
- Example engines: Virtuoso, gStore
- **Identified by:** global IRIs, so the same resource connects across datasets
- **Best for:** data integration, shared vocabularies, reasoning with RDFS and OWL, and Linked Data
- **Example:**
```
:alice :knows :bob .
:alice :since "2010" .
```

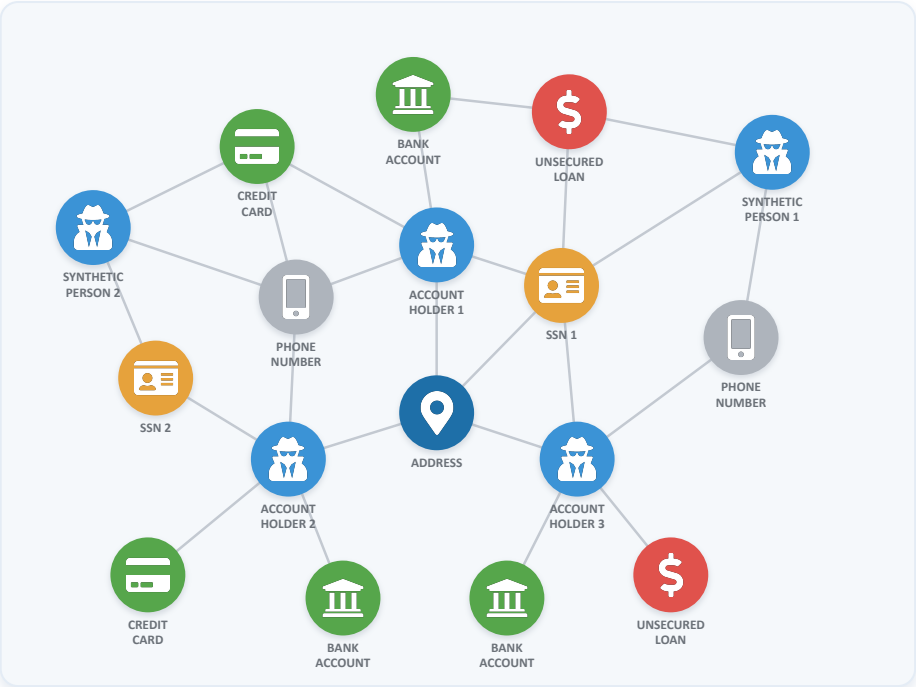


Labelled Property Graph

- Nodes & edges are first-class objects with identity
- Both carry labels + inline key–value properties
- Queries are pattern-centric, mostly in Cypher or GQL
- Example engines: Neo4j, TigerGraph, JanusGraph, Amazon Neptune, NebulaGraph
- **Stored as:** direct pointers from each node to its neighbours, so every hop is fast
- **Best for:** traversal and analytics (fraud rings, recommendations, community detection), with built-in algorithm libraries
- **Example:** `(alice)-[:KNOWS {since:2010}]->(bob)`
  One edge that carries its own property

Fraud Detection Graphs

A typical property graph application: detecting fraud rings through entity link analysis

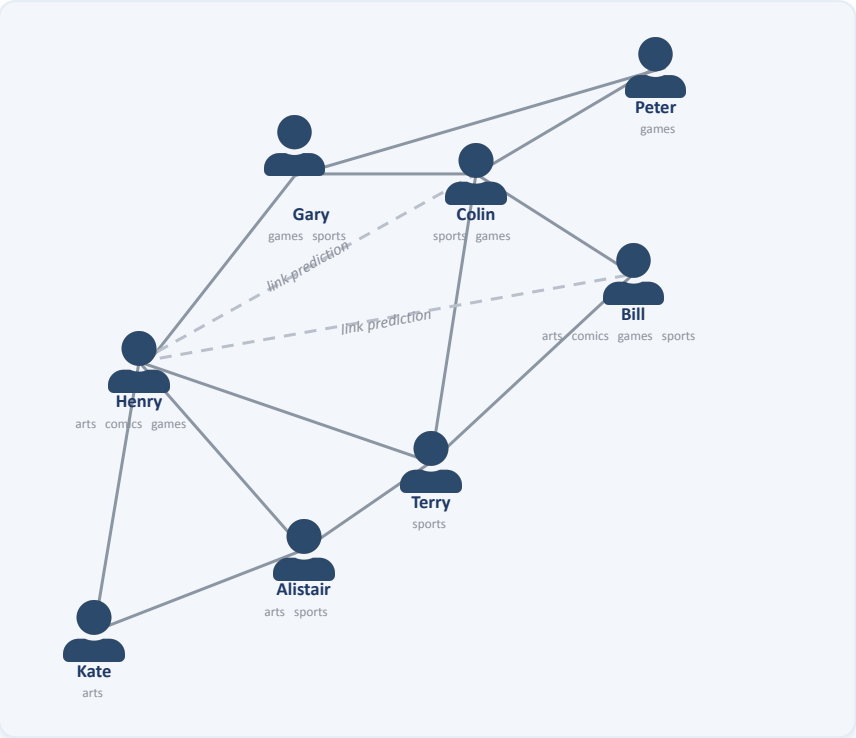


Source: https://neo4j.com/blog/enterprise-fraud-detection/

- **Credit card fraud**
  Map transaction relationships & patterns to identify unusual spending behaviors.
- **Insurance fraud**
  Reveal suspicious patterns in claims & policyholder data to detect fraudulent activity and protect insurance integrity.
- **Healthcare fraud**
  Examine healthcare claims & interactions to find billing inconsistencies and reduce system exploitation.
- **E-commerce fraud**
  Analyze shopping patterns & user behaviors to flag potential fraud and secure online transactions.
- **Identity fraud**
  Scrutinize account connections & activities to detect fake profiles, bot networks, and maintain online trust.

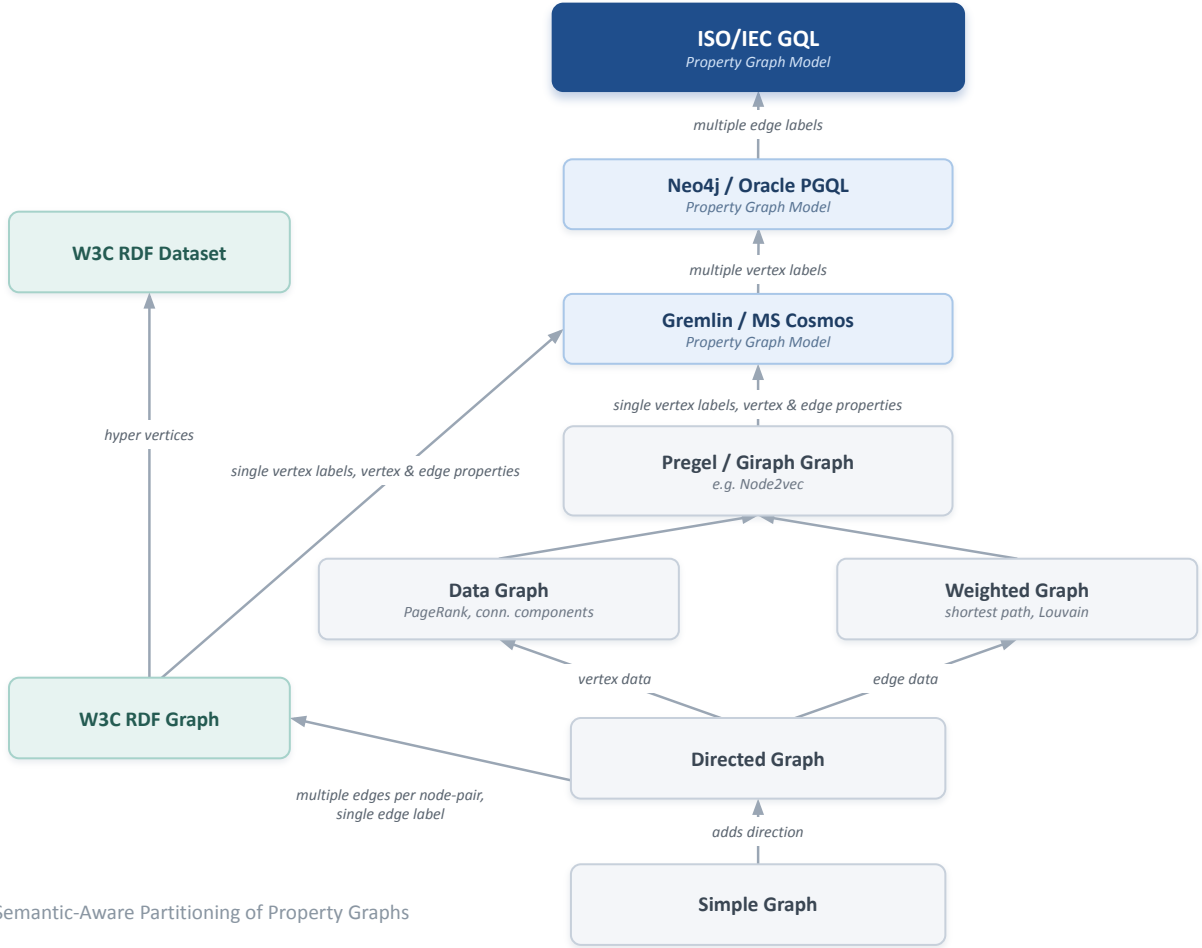
Graph-Based Recommender Systems

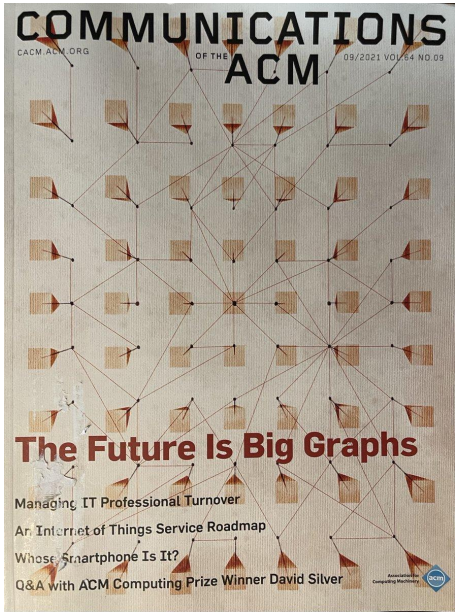
A typical property graph application: recommendations through graph exploration and link prediction



- **E-Commerce recommendations**
  Tailor product suggestions by analyzing customer data → boosting purchases & satisfaction.
- **Content recommendation**
  Custom content suggestions in streaming applications → enhanced viewer engagement.
- **Personalized news feeds**
  News feeds based on user interests → improved reading experience.
- **Travel recommendations**
  Personalized booking suggestions → increased loyalty & bookings.
- **Health suggestions**
  Custom wellness plans → user health goals support.
- **Job matching**
  Job & connection recommendations in professional networks → career development support.

Family of Graph Data Models





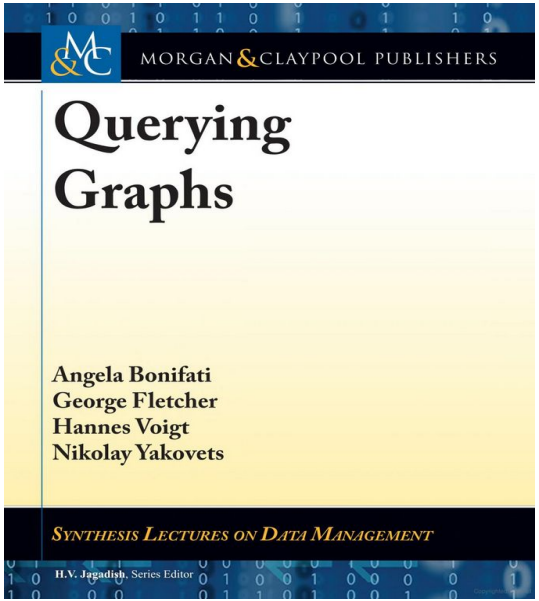
Source: *S.Sakr, A.Bonifati, H.Voigt, A.Iosup, et al : The future is big graphs: a community view on graph processing systems. Commun. ACM 64(9): 62-71 (2021)*

The Property Graph Model

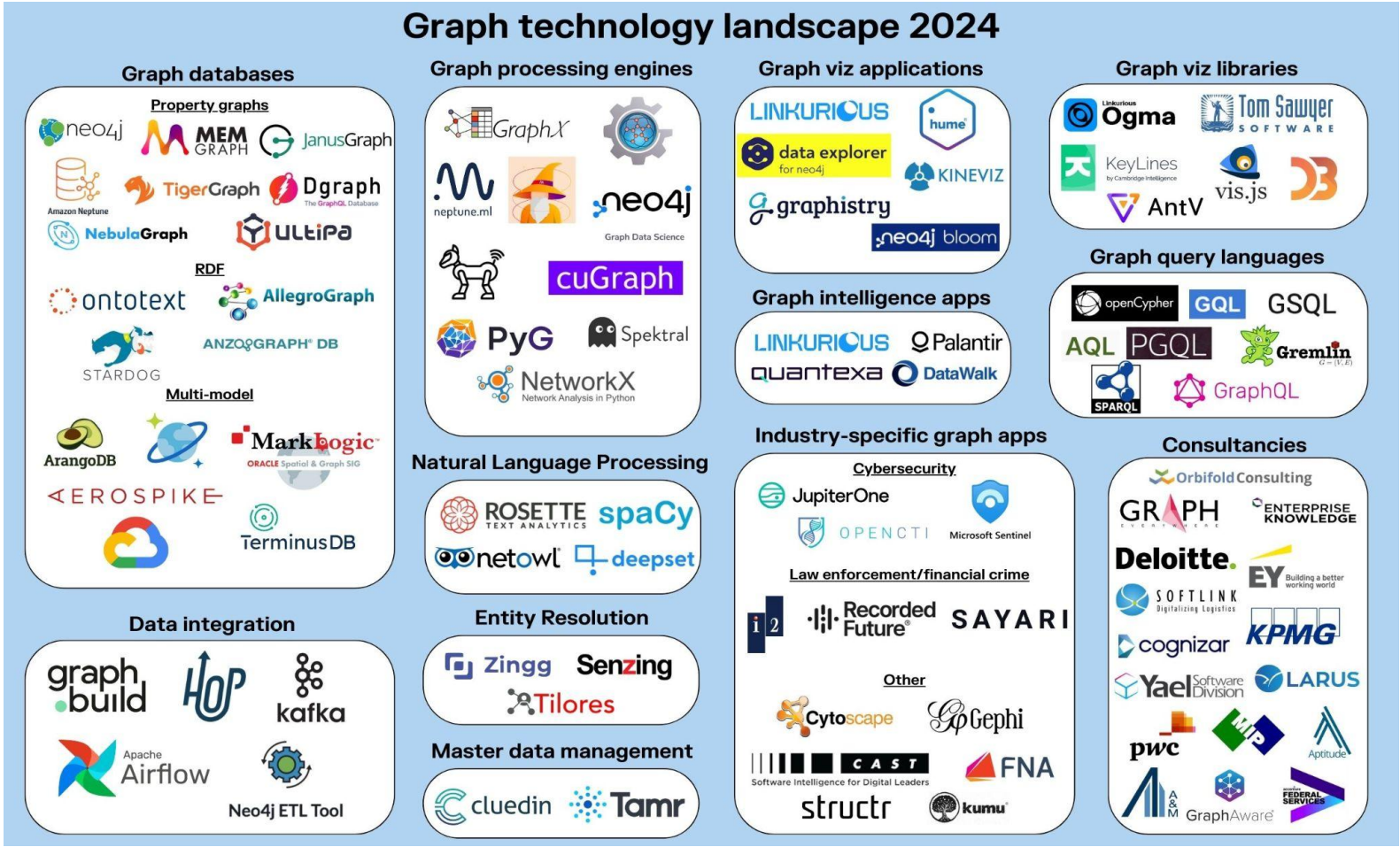
$\mathcal{O}$: objects $\mathcal{L}$: labels $\mathcal{K}$: property keys $\mathcal{N}$: values

A structure $(\mathcal{V}, \mathcal{E}, \eta, \lambda, \nu)$ such that:

$\mathcal{V} \subseteq \mathcal{O}$	a finite set of objects (the nodes)
$\mathcal{E} \subseteq \mathcal{O}$	a finite set of objects (the edges)
$\eta : \mathcal{E} \rightarrow \mathcal{V} \times \mathcal{V}$	edge assignment (source, target)
$\lambda : \mathcal{V} \cup \mathcal{E} \rightarrow \mathcal{P}(\mathcal{L})$	label assignment
$\nu : (\mathcal{V} \cup \mathcal{E}) \times \mathcal{K} \rightarrow \mathcal{N}$	key–value assignment



***Source**: A. Bonifati, G. Fletcher, H. Voigt, N. Yakovets — Querying Graphs*



Source: https://linkurious.com/blog/introduction-graph-technology-landscape/"

Main Graph DBMS Features

Efficient management and analysis of highly connected data through flexible schemas, expressive graph queries, and built-in analytics



Schema-flexible

- No a-priori schema constraints on instances
- Manage rapidly-evolving, heterogeneous data
- Greater scalability & performance



Expressive querying

- Extract & manipulate complex patterns
- Label-constrained reachability queries
- Navigational paths with native recursion



Analytics & AI

- Large-scale analytics & ML tooling
- Custom graph & ML algorithms
- Community detection, embeddings, link prediction

Example Implementation Differences

Differences in vertex and edge identities, labels, and properties across property graph implementations

Vertex identity

User-specified (TinkerPop, JanusGraph) vs. system-generated (Neo4j)

Edge identity

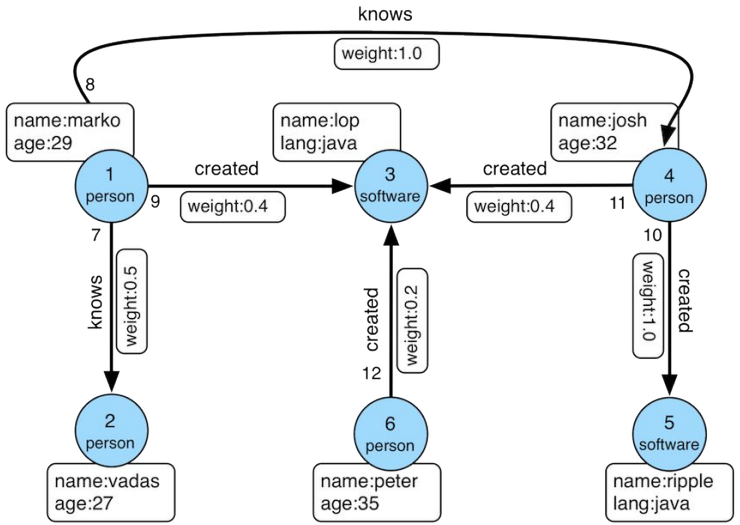
Implicit (source & target) vs. explicit / visible vs. hidden / user- vs. system-specified

Labels

Single label (TinkerPop, Neo4j edges) vs. multiple labels (Neo4j vertices)

Properties

Multi-valued vs. single-valued properties



Example of property graph in TinkerPop

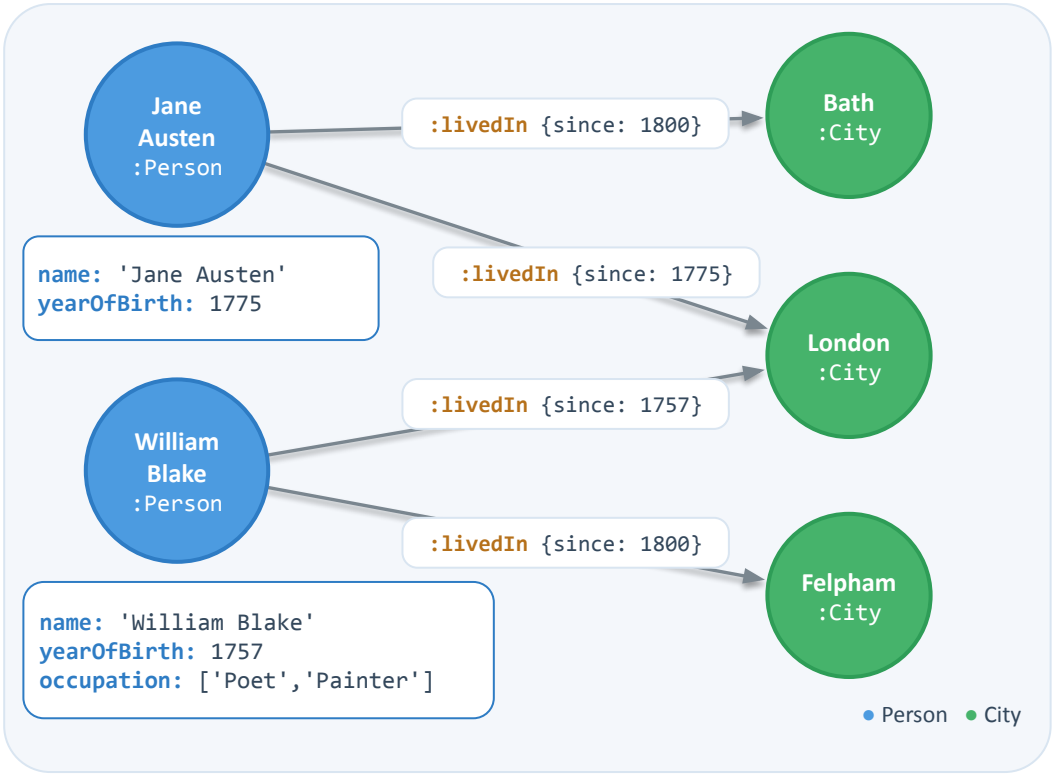
BLOCK 1.2

Property Graph Modeling and Querying

Modeling entities and relationships, query languages and graph patterns

The Property Graph Model: Nodes and Edges

A graph of labelled nodes joined by labelled, directed edges; both carry key–value properties



Node

Labels: one or more, e.g. :Person

Properties: key–value pairs, e.g. name, yearOfBirth

Values: numbers, strings, dates, or lists

Edge

Label: its type, e.g. :livedIn

Direction: from Jane Austen to Bath

Properties: also allowed, e.g. since: 1800

Written in Cypher:

```
(ja:Person {name: 'Jane Austen'})
(ja)-[:livedIn {since: 1800}]->(ba)
```

Property Graph Modeling — Entities

Entity with properties

- Maps naturally from the entity/relationship model
- Schema is typically implicit, given with the instances

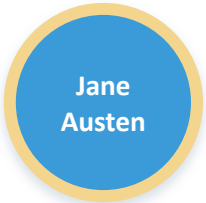
Cypher ASCII-art instance

```
(ja:Person { name: 'Jane Austen', yearOfBirth: 1775 })
```

Person

name = "Jane Austen"

yearOfBirth = 1775



Person : **name**: Jane Austen ,
yearOfBirth: 1775

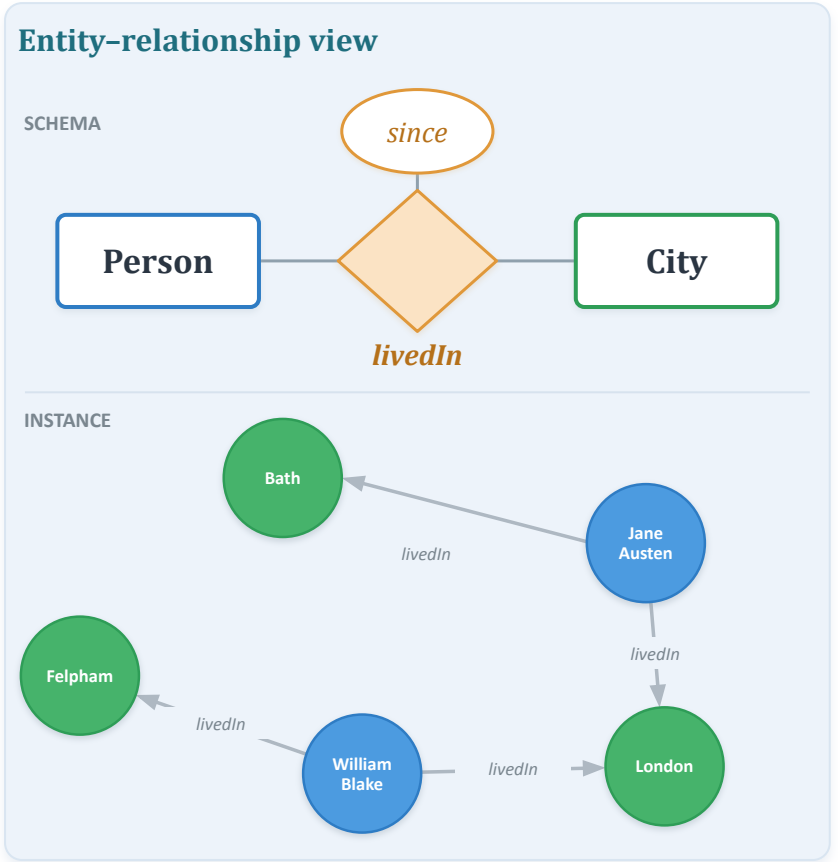
Property Graph Modeling — Relationships (N:M)

Vertices

```
(ja:Person { name: 'Jane Austen', yearOfBirth: 1775 })
(wb:Person { name: 'William Blake', yearOfBirth: 1757 })
(lo:City { name: 'London' })
(ba:City { name: 'Bath' })
(fe:City { name: 'Felpham' })
```

Edges

```
(ja)-[:livedIn { since: 1775 }]->(lo)
(ja)-[:livedIn { since: 1800 }]->(ba)
(wb)-[:livedIn { since: 1757 }]->(lo)
(wb)-[:livedIn { since: 1800 }]->(fe)
```



N-ary Relationships & Multi-valued Properties

N-ary relationships (N > 2)

- Not directly expressible; an edge connects exactly two vertices
- Solution: reification of the relationship as a node

```
(ja:Person { name:'Jane Austen', … })
(wh:Publisher { name:'Whitehall' })
(sas:Book { title:'Sense and Sensibility' })
(pub:Publication { year: 1811 })
(pub)-[:author]->(ja)
(pub)-[:book]->(sas)
(pub)-[:publisher]->(wh)
```

Multi-valued properties

- Directly possible in Neo4j via list-valued properties
- If not expressible: reify the values of the property

```
(wb:Person { name:'William Blake',
hadOccupation:['Poet','Painter','Printmaker']})
```



Person: hadOccupation -> {Poet, Painter, Printmaker}

PG Query Languages

Main usage

- Extract graph patterns, their associated (meta-)data & derived metrics
- Create & modify graph data: insert / delete / update nodes, edges & properties
- Advanced usage supports the full data lifecycle

P Procedural

Gremlin -> TinkerPop, JanusGraph

GSQL -> TigerGraph

AQL -> ArangoDB

D Declarative

Cypher -> Neo4j, Neptune, SAP HANA, Memgraph

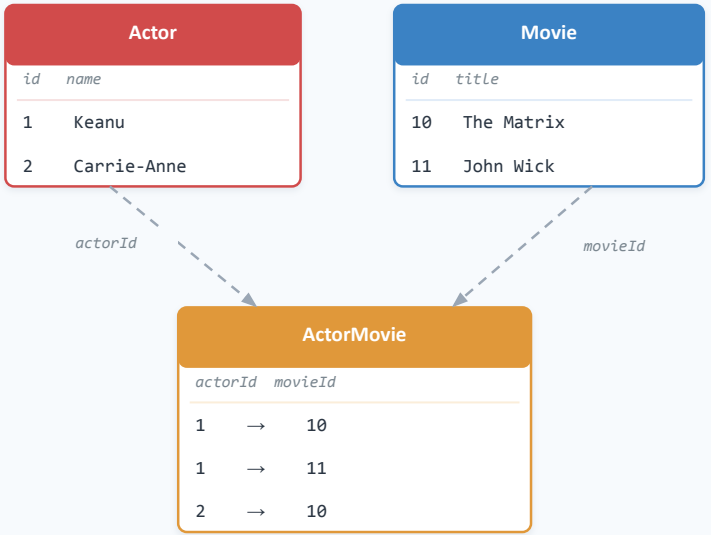
PGQL -> Oracle

GQL / SQL-PGQ -> new ISO standards

Relational Tables vs. Property Graph

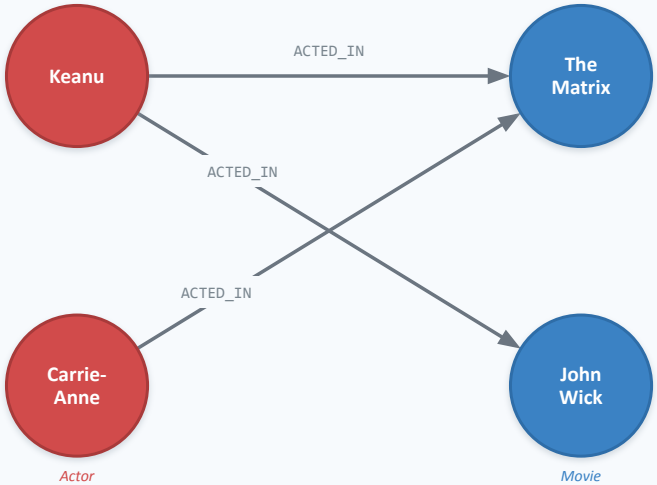
Modeling a many-to-many relationship TO Actor ↔ Movie

RELATIONAL tables + join table



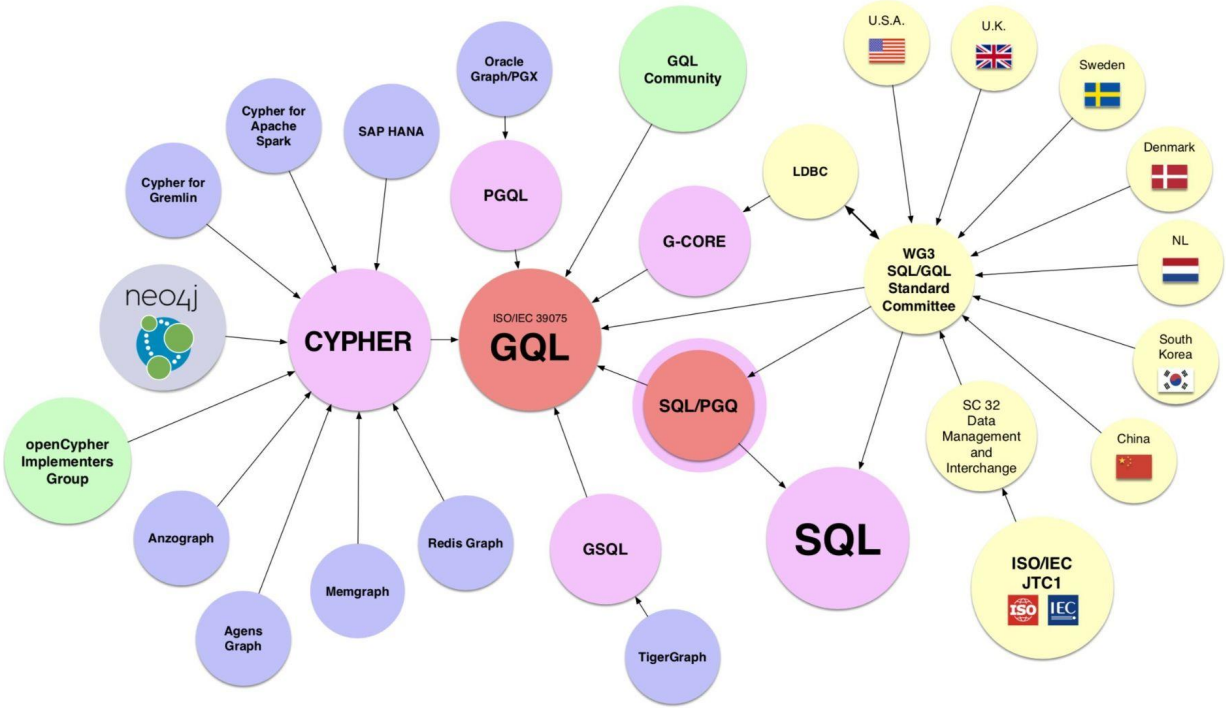
Many-to-many needs a junction table + joins

GRAPH nodes + edges



Direct relationships -> traverse edges, no join table

PG-Query Languages Landscape



https://neo4j.com/blog/gql-standard-query-language-property-graphs/

New Standards: GQL & SQL/PGQ

GQL

ISO/IEC standard since 2019 · published April 2024

- Works directly over a property graph
- Syntax similar to Cypher
- **Databases:** Neo4j, Google Spanner Graph, Microsoft Fabric
- **Also:** NebulaGraph, TigerGraph (support is partial)

SQL/PGQ

Part 16 of SQL:2023 · published June 2023

- Works over a relational view of a graph
- SQL with MATCH-clause extensions for patterns
- **Databases:** Oracle Database 23ai, Google Spanner
- **Also:** DuckDB, through the DuckPGQ research extension

Common pattern-matching layer: **Graph Pattern Calculus (GPC)** — *Francis et al., PODS 2023*

https://www.gqlstandards.org/
https://neo4j.com/docs/cypher-manual/current/appendix/gql-conformance/supported-mandatory/

Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, Domagoj Vrgoč: **GPC: A Pattern Calculus for Property Graphs**. PODS 2023: 241-250
Daniel ten Wolde, Tavneet Singh, Gábor Szárnyas, Peter A. Boncz: **DuckPGQ: Efficient Property Graph Queries in an analytical RDBMS**. CIDR 2023

Why Patterns?

Queries, constraints and partitioning cost are all about patterns

1 Queries are patterns

Every graph query looks for a small graph of nodes, edges and conditions. A workload is a set of patterns.

```
MATCH (p:Person)-[:WORKS_AT]->(c)
```

Returns in Part 2: how a distributed query runs

2 Constraints are patterns

Keys, functional dependencies and generating dependencies all say: if this pattern matches, then something must hold.

```
FOR (p:Person) IDENTIFIER p.email
```

Returns in Part 3: graph dependencies

3 Matches decide the cost

A match whose nodes and edges sit on different machines needs network communication. Good partitioning keeps matches together.

```
match on 1 machine → no hop
```

Returns in Parts 2 and 3: partitioning

In this part: what a pattern is, how it is matched against a graph, and how to read the queries used in the rest of the tutorial.

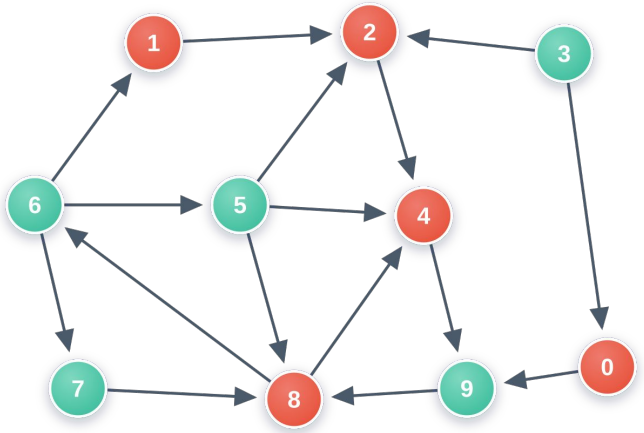
Notation: *patterns are written in the ASCII-art syntax of Cypher, which GQL and SQL/PGQ adopted (slide 30), so they read the same in all three. Keys use the PG-Keys syntax.*

Graph Pattern Matching

Subgraph pattern {p}



How many matches does p have on graph G ?



Matching p on the graph G

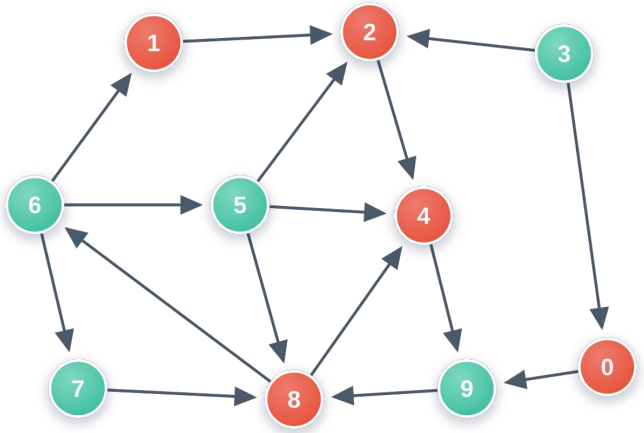
Graph Pattern Matching

Subgraph pattern {p}



How many matches does p have on graph G ?

A	5
B	8
C	10
D	14



Matching p on the graph G

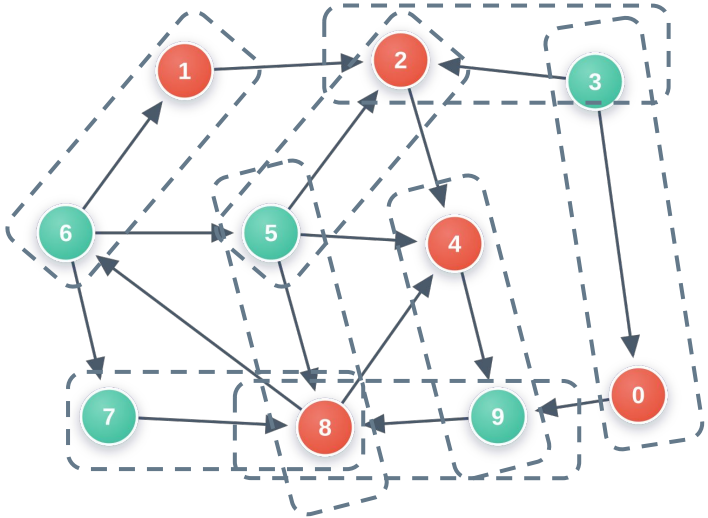
Graph Pattern Matching

Subgraph pattern {p}



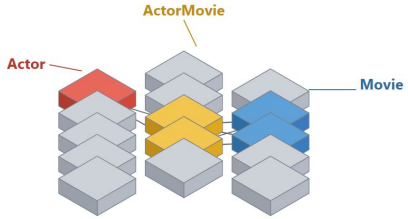
How many matches does p have on graph G ?

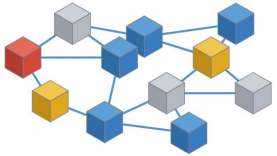
A	5
B	8 ✓ matches
C	10
D	14



Matching p on the graph G → 8 directed pairs

Relational vs. Graph Querying





Q_1: *What actors played in the movie “V for Vendetta”?*

SQL

```
SELECT name FROM Actor
LEFT JOIN ActorMovie
  ON Actor.Id = ActorMovie.ActorId
LEFT JOIN Movie
  ON Movie.Id = ActorMovie.MovieId
WHERE Movie.name = "V for Vendetta"
```

Cypher

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE m.name = "V for Vendetta"
RETURN a.name
```

SQL

Offers limited recursion & cannot return values on paths

Pattern processing

A bottleneck vs. in-memory path traversal of native graph DBMSs

Graph queries

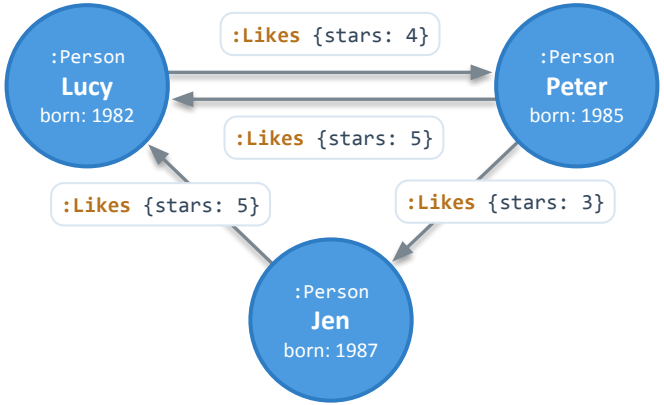
Can call dedicated graph-data-science algorithms

Matching

http://neo4j.com/docs/developer-manual/current/cypher/clauses/match/]

[http://neo4j.com/docs/developer-manual/current/cypher/clauses/return/]

Cypher: *Neo4j's declarative query language, opened as openCypher in 2015 and a main input to GQL.*



MATCH clause

- Primary way of getting data in Cypher, and in GQL
- Lets you specify subgraph patterns to search for
- Each named pattern element is bound to the matched instance
- A query can contain multiple `MATCH` clauses

```
MATCH (p:Person)-[:Likes]->(f:Person)
RETURN p.name, f.born
```

→ *How many rows will this query return?*

Number of Results	
A	2
B	3
C	4
D	5

?

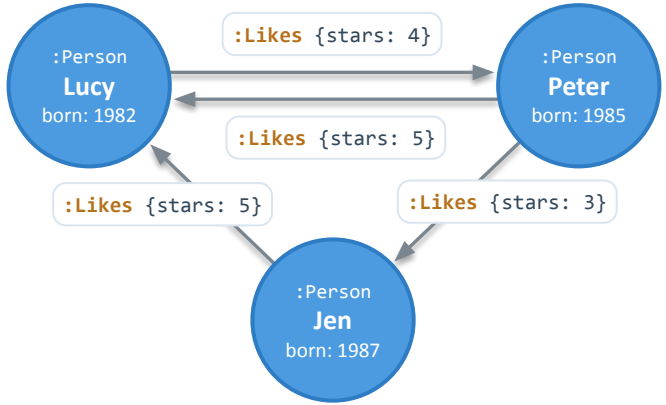
Matching

http://neo4j.com/docs/developer-manual/current/cypher/clauses/match/]

[http://neo4j.com/docs/developer-manual/current/cypher/clauses/return/]

MATCH clause

- Primary way of getting data from a Neo4j database
- Lets you specify subgraph patterns to search for
- Each named pattern element is bound to the matched instance
- A query can contain multiple `MATCH` clauses




```
MATCH (p:Person)-[:Likes]->(f:Person)
RETURN p.name, f.born
```

→ ***4 rows returned*** ⇒ *answer C*

p.name	f.born
Lucy	1985
Peter	1987
Jen	1982
Peter	1982

Matching

http://neo4j.com/docs/developer-manual/current/cypher/clauses/match/]

[http://neo4j.com/docs/developer-manual/current/cypher/clauses/return/]

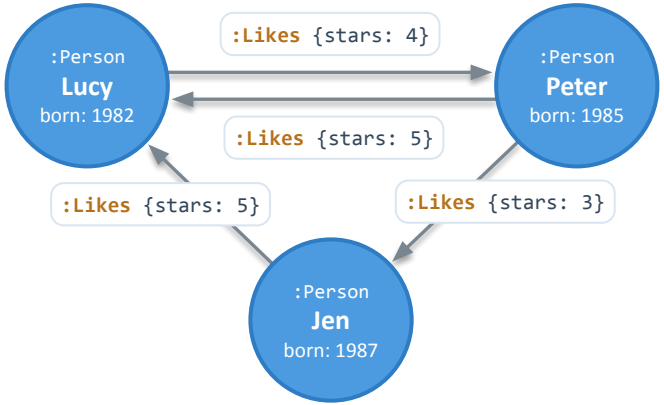
MATCH clause

- Primary way of getting data from a Neo4j database
- Lets you specify subgraph patterns to search for
- Each named pattern element is bound to the matched instance
- A query can contain multiple `MATCH` clauses

```
MATCH (p:Person)-[:Likes]->(f:Person)
RETURN p.name, f.born
```

```
MATCH (p:Person)-[:Likes]->(:Person)-[:Likes]->(fof:Person)
RETURN p.name, fof.name
```

→ *friends-of-friends: how many rows now?*



p.name	f.born
Lucy	1985
Peter	1987
Jen	1982
Peter	1982

Number of Results	
A	2
B	3
C	4
D	5

?

Matching

http://neo4j.com/docs/developer-manual/current/cypher/clauses/match/]

[http://neo4j.com/docs/developer-manual/current/cypher/clauses/return/]

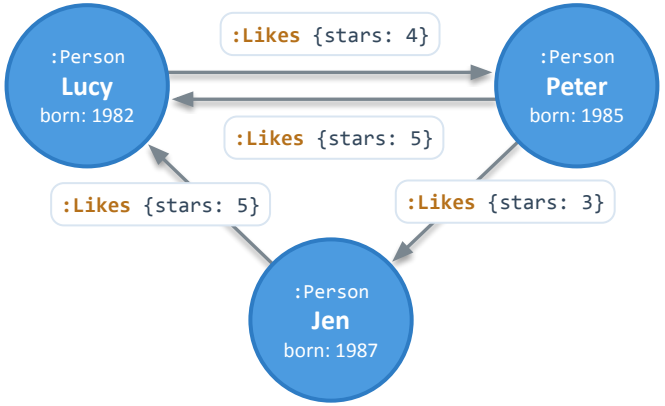
MATCH clause

- Primary way of getting data from a Neo4j database
- Lets you specify subgraph patterns to search for
- Each named pattern element is bound to the matched instance
- A query can contain multiple `MATCH` clauses

```
MATCH (p:Person)-[:Likes]->(f:Person)
RETURN p.name, f.born
```

```
MATCH (p:Person)-[:Likes]->(:Person)-[:Likes]->(fof:Person)
RETURN p.name, fof.name
```

→ ***5 rows returned*** ⇒ *answer D*



p.name	f.born
Lucy	1985
Peter	1987
Jen	1982
Peter	1982

p.name	fof.name
Lucy	Jen
Peter	Lucy
Peter	Peter
Jen	Peter
Lucy	Lucy

OPTIONAL MATCH & Example

neo4j.com/docs · cypher/clauses/optional-match

OPTIONAL MATCH

- If no matches are found, uses NULLs as bindings
- Behaves like a relational left outer join

```
MATCH (a:Movie)
OPTIONAL MATCH (a)<-[:WROTE]-(x)
RETURN a.title, x.name
```

→ *Movies with no writer still appear, with x.name = null*

$ MATCH (a:Movie) OPTIONAL MATCH (a)<-[:WROTE]-(x)…

Rows

Code

a.title	x.name
The Matrix	null
The Matrix Reloaded	null
The Matrix Revolutions	null
The Devil's Advocate	null
A Few Good Men	Aaron Sorkin
Top Gun	Jim Cash
Jerry Maguire	Cameron Crowe
Stand By Me	null
As Good as It Gets	null
What Dreams May Come	null
Snow Falling on Cedars	null
You've Got Mail	null
Sleepless in Seattle	null
Joe Versus the Volcano	null
When Harry Met Sally	Nora Ephron
That Thing You Do	null

Returned 41 rows in 40 ms.

Pattern Syntax

neo4j.com/docs · cypher/syntax/patterns

Vertex patterns

```
()
```

unidentified vertex

```
(matrix)
```

vertex identified by variable *matrix*

```
(:Movie)
```

unidentified vertex with label *Movie*

```
(matrix:Movie:Action)
```

labels *Movie* and *Action*, variable *matrix*

```
(matrix:Movie {title: "The Matrix"})
```

+ property *title* equals the string "The Matrix"

```
(matrix:Movie {title: "The Matrix", released: 1997})
```

+ property *released* equals the integer *1997*

Edge patterns

```
--
```

unidentified undirected edge (either direction)

```
-->
```

unidentified directed edge

```
-[role]->
```

directed edge identified by variable *role*

```
-[:ACTED_IN]->
```

unidentified directed edge with label *ACTED_IN*

```
-[role:ACTED_IN]->
```

directed edge *ACTED_IN*, variable *role*

```
-[role:ACTED_IN {roles: ["Neo"]}]->
```

+ property *roles* contains the string "Neo"

Pattern Syntax

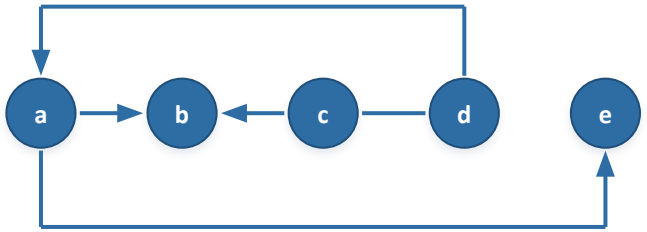
neo4j.com/docs · cypher/syntax/patterns

Path patterns

- String of alternating vertex pattern and edge pattern
- Starts and ends with a vertex pattern

```
(a)-->(b)<--(c)--(d)-->(a)-->(e)
```

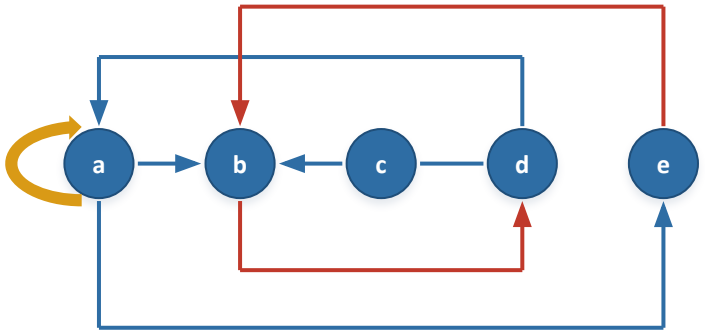
```
(keanu:Person:Actor {name: "Keanu Reeves"})-[role:ACTED_IN
{roles: ["Neo"]}]->(matrix:Movie {title: "The Matrix"})
```



Graph patterns

- One or multiple path patterns
- Path patterns should share at least one variable
- Without a shared variable the pattern is disconnected
  - cross-product of the connected subpatterns
  - quadratic blow-up in result size & complexity

```
(a)-->(b)<--(c)--(d)-->(a)-->(e), (e)-->(b)-->(d), (a)-->(a)
```



Pattern Syntax

neo4j.com/docs · cypher/syntax/patterns

Which patterns are valid?

A `(a,b:Movie)-[:SHOWN_IN]->(e),(f)`

B `(a:Movie)-[:SHOWN_IN]->()`

C `(:Movie)-[:SHOWN_IN]->`

D `()<--(a:Movie)`

Which patterns specify a loop?

A `(a:Movie {name: "Matrix"})-->(a)`

B `(a:Movie {name: "Matrix"})-->(b:Movie {name: "Matrix"})`

C `(a:Movie {name: "Matrix"})-->(a:Movie {name: "Matrix"})`

D `(a:Movie {name: "Matrix"})-->({name: "Matrix"})`

Pattern Syntax

neo4j.com/docs · cypher/syntax/patterns

Which patterns are valid?

A `(a,b:Movie)-[:SHOWN_IN]->(e),f)` — ✗ invalid

B `(a:Movie)-[:SHOWN_IN]->()` — ✓ valid

C `(:Movie)-[:SHOWN_IN]->` — ✗ invalid

D `()<--(a:Movie)` — ✓ valid

A two variables in one node · C path must end on a node.

Which patterns specify a loop?

A `(a:Movie {name: "Matrix"})-->(a)` — ⟳

B `(a:Movie {name: "Matrix"})-->(b:Movie {name: "Matrix"})` — –

C `(a:Movie {name: "Matrix"})-->(a:Movie {name: "Matrix"})` — ⟳

D `(a:Movie {name: "Matrix"})-->({name: "Matrix"})` — –

A, C reuse variable a ⟶ same node. B, D end on a distinct / anonymous node.

BLOCK 1.3

Property Graphs and RDF

Formal models, schemas (SHACL, PG-Schema) and mappings between the two

Knowledge Graph Formalisms

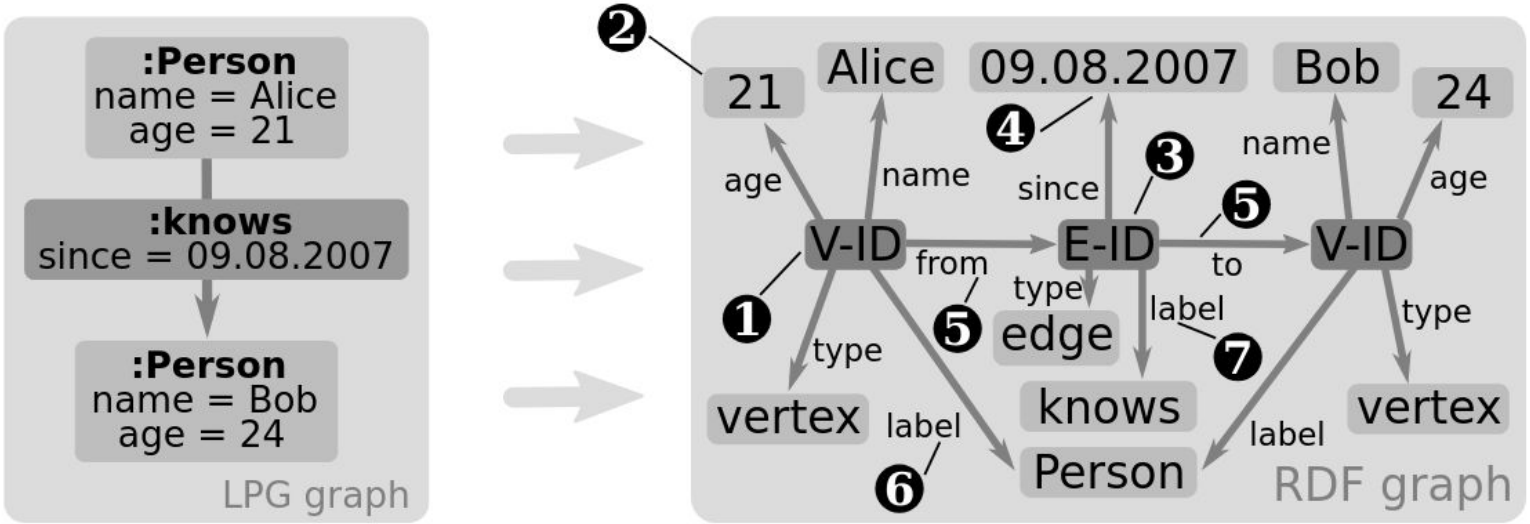
Labeled Property Graphs (LPG)	RDF Graphs
• Multi-labeled multigraphs with key-value attributes attached to nodes & edges. • Entities & relationships have internal structure beyond labels. • Properties attached to nodes & edges. • IDs are internal to the graph database. The user has no direct control over these. • **GQL** & **SQL/PGQ** standard query languages. • Limited & varying graph schema support. • PG-Keys & PG-Schema. • Application-specific exchange formats: CSV, GraphML, and research proposals (PGEF, YARS-PG, GDL)	• Subject-Predicate-Object representation of nodes and relations. • Entities & relationships do not have any internal structures (only URIs). • Properties are also triples, with literal values. • IDs are global - URIs. Under user control. • Standard **SPARQL** standard query language. • Ontology support: RDFS, OWL2 vocabularies. • SHACL & SHEX integrity constraint languages. • N-Triples, N-Quads, RDF/XML, JSONLD

GQL Query Language Standard: https://www.gqlstandards.org/ (Officially Published: April 2024)
SQL/PGQ Query Language Standard: https://www.iso.org/standard/79473.html (Officially Published: June 2023)
W3C SPARQL Query Language: https://www.w3.org/TR/rdf-sparql-query/ (Officially Published: January 2008)

Knowledge Graph Formalisms

Labeled Property Graphs (LPG)	RDF Graphs
O: set of objects, L: finite set of labels, K: set of property keys, N: set of values. Structure (V, E, η, λ, v) such that • $V \subseteq O$ is a finite set of objects (nodes) • $E \subseteq O$ is a finite set of objects (edges) • $\eta : E \rightarrow V \times V$ (edge assignment) • $\lambda : V \cup E \rightarrow P(L)$ (label assignment) • $v : (V \cup E) \times K \rightarrow N$ (key value assignment)	U: set of Uniform Resource Identifiers (URIs), L: set of literals (constants), B: set of blank nodes. Triple set $(s, p, o) \in (U \cup B) \times U \times (U \cup B \cup L)$ • s represents a subject • p models a predicate • o represents an object

Knowledge Graph Formalisms



Maciej Besta, Robert Gerstenberger, Emanuel Peter, Marc Fischer, Michal Podstawski, Claude Barthels, Gustavo Alonso, Torsten Hoefler: **Demystifying Graph Databases: Analysis and Taxonomy of Data Organization, System Designs, and Graph Queries**. ACM Computing Surveys. 56(2): 31:1-31:40 (2024)

SHACL: Shapes Constraint Language

SHACL — *W3C Recommendation, 2017 · the same customer data as the PG-Schema slide*

Constraints for RDF graphs

- A shape describes what the data around a node must look like
- Target: which nodes to check, for example every ex:Person
- Property constraints: path, data type, minimum and maximum count, class of the value
- Closed-world: missing data is reported as a violation
- Output: a validation report listing every violation

Alternative: *ShEx (Shape Expressions), a grammar-style shape language*

```
ex:PersonShape a sh:NodeShape ;
  sh:targetClass ex:Person ;
  sh:property [ sh:path ex:name ; sh:datatype xsd:string ;
                sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [ sh:path ex:birthdate ; sh:datatype xsd:date ;
                sh:maxCount 1 ] .

ex:CustomerShape a sh:NodeShape ;
  sh:targetClass ex:Customer ;
  sh:property [ sh:path ex:id ; sh:datatype xsd:int ; sh:minCount 1 ] ;
  sh:property [ sh:path ex:owns ; sh:class ex:Account ;
                sh:minCount 1 ] .

ex:AccountShape a sh:NodeShape ;
  sh:targetClass ex:Account ;
  sh:property [ sh:path ex:iban ; sh:datatype xsd:string ;
                sh:minCount 1 ; sh:maxCount 1 ] .
```

PG-Keys: Keys for Property Graphs

PG-Keys — R. Angles et al., SIGMOD 2021 · keys for the same account data as the SHACL slide

Keys for property graphs

- A key has a scope (which nodes or edges) and a descriptor (how to get each key value)
- Modes, combined as needed: EXCLUSIVE (no two share it), MANDATORY (must exist), SINGLETON (only one)
- Checked by a query on the same graph; Neo4j enforces only simple keys
- Not in GQL 2024 (graph types check types, not keys); PG-Schema proposes both

Compared with SHACL: scope ≈ target, descriptor ≈ path, modes ≈ constraints. EXCLUSIVE has no core SHACL equivalent.

Basic rule: `FOR <scope> <mode> <descriptor>`

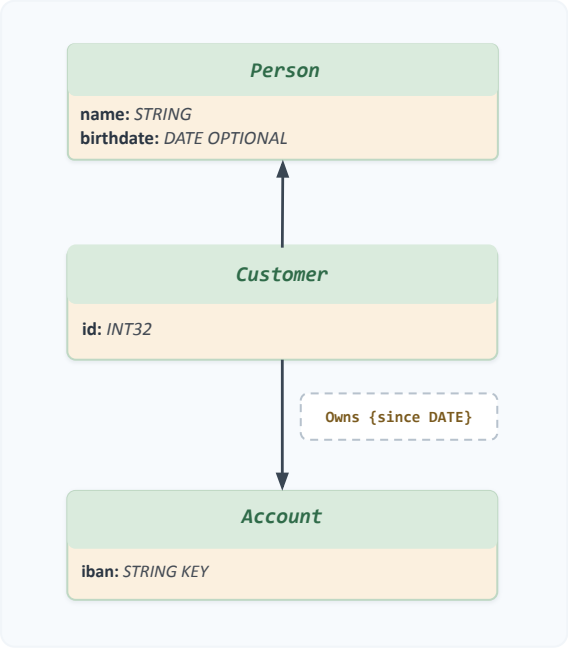
```
FOR (a:Account)
    EXCLUSIVE MANDATORY SINGLETON a.iban
// scope: every account   descriptor: its IBAN

// check EXCLUSIVE in Cypher: each row is a violation
MATCH (a:Account), (b:Account)
WHERE a.iban = b.iban AND elementId(a) < elementId(b)
RETURN a, b

// simple keys can also be enforced by Neo4j itself
CREATE CONSTRAINT FOR (a:Account)
    REQUIRE a.iban IS NODE KEY
```

PG-Schemas: Graph Database Schemas

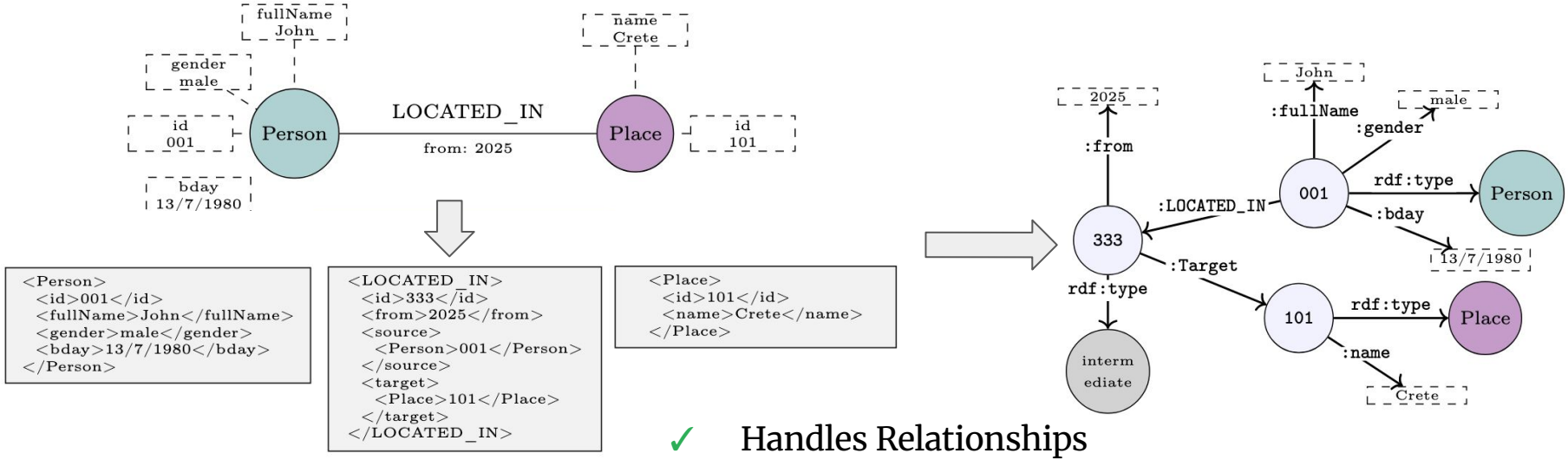
PG-Schema — *R. Angles et al., SIGMOD 2023*




```
// node type:  (nameType: Label {property TYPE, ...})

CREATE GRAPH TYPE customerGraphType STRICT {
  (personType: Person
      {name STRING, OPTIONAL birthdate DATE}),
  (customerType: personType & Customer {id INT32}),
  (accountType: Account {iban STRING}),
  (:customerType)
      -[ownsType: Owns {since DATE}]->(:accountType),
  FOR (x:accountType) EXCLUSIVE MANDATORY SINGLETON x.iban
}
```

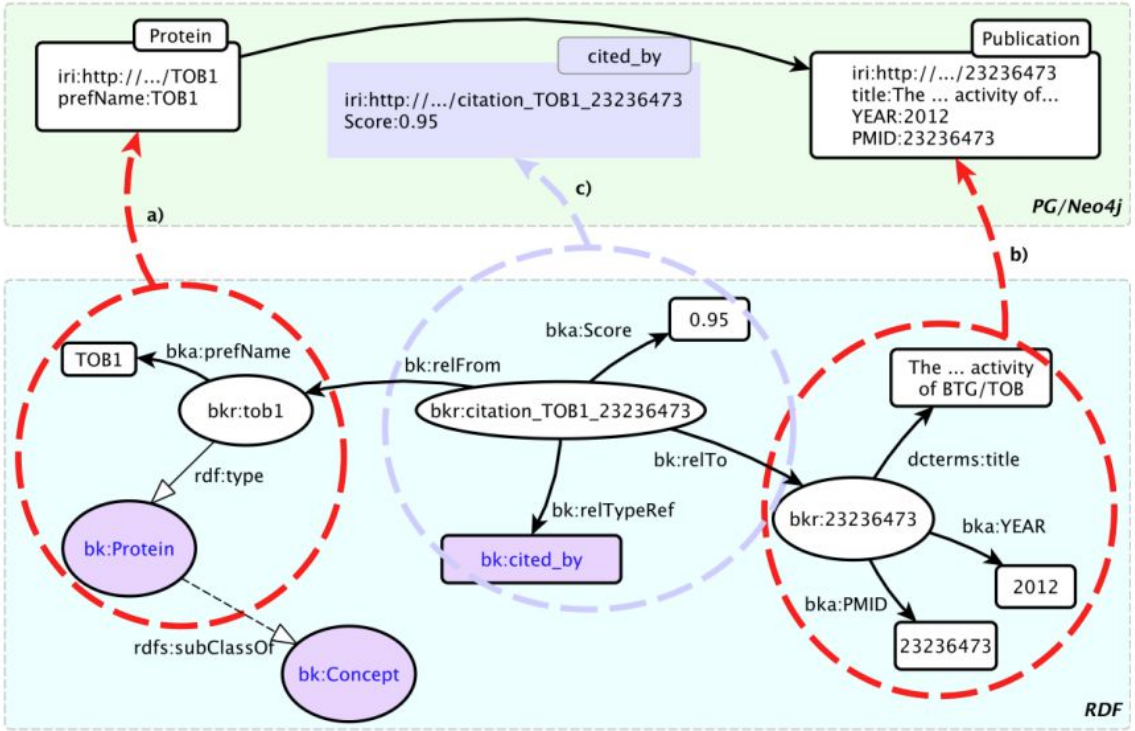
Transforming PGs → RDF Graphs : PG2RDF approach



- ✓ Handles Relationships
- ✓ Automation
- ✓ Lossless
- ✓ Schema-aware
- ✓ Supports X3ML and RML mappings

Sophia Sideri, Vasilis Efthymiou, Dimitris Plexousakis and Haridimos Kondylakis, **PG2RDF: A preliminary approach for automating the transformation of property graph data into RDF**, IJCKG 2025

Transforming RDF Graphs → PGs : rdf2neo mapping

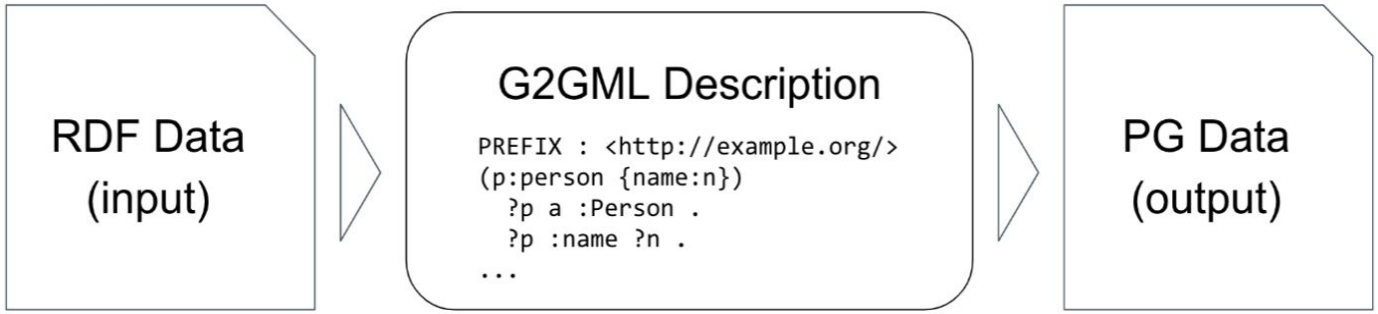


- ✓ Handles Relationships
- ✓ Retain RDF structure

A flexible, manual tool: you choose what to copy, and it keeps the RDF names.

Brandizi, M., Singh, A., Hassani-Pak, K.: **Getting the best of linked data and property graphs: rdf2neo and the knetminer use case**. In: SWAT4LS. vol. 2275 (2018)

Transforming RDF Graphs → PGs : G2GML mapping

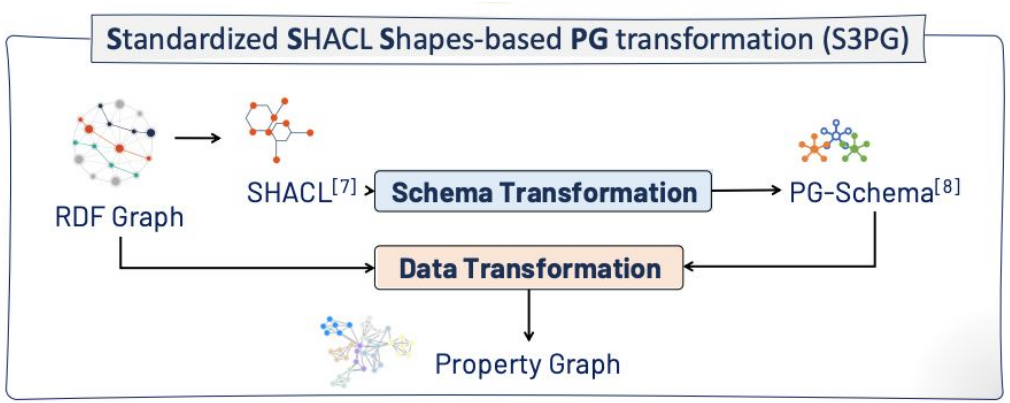


- ✓ Handles Relationships
- ✓ Customizable structure

- *x* One-way transformation
- *x* Automation

Chiba, H., Yamanaka, R., Matsumoto, S.: G2gml: **Graph to graph mapping language for bridging rdf and property graphs**. In: ISWC. pp. 160–175 (2020)

Transforming RDF Graphs → PGs via Schemas (S3PG)



- ✓ Handles Relationships
- ✓ Automation
- ✓ Lossless

- x One-way transformation

Kashif Rabbani, Matteo Lissandrini, Angela Bonifati, Katja Hose: **Transforming RDF Graphs to Property Graphs using Standardized Schemas**. Proc. ACM Manag. Data 2(6): 242:1-242:25 (2024)

PART 2

Foundations of Graph Partitioning

Motivation, Strategies, and the Classical Approaches

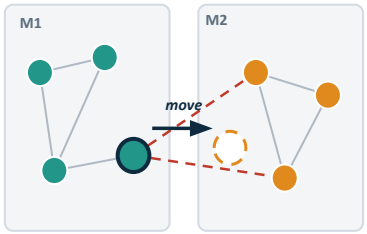
BLOCK 2.1

Why Partition, and the Problem

Why graphs are split, what a cut costs, and how the problem is defined

Graph Partitioning Revolution

Five decades: from moving single vertices to partitioning by meaning and workload



1970 – 1982

Local heuristics

Kernighan–Lin 1970

Fiduccia–Mattheyses 1982

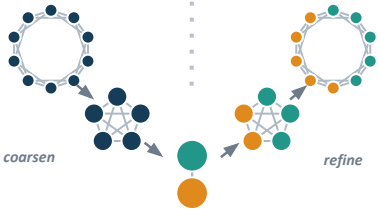
1990 – 1998

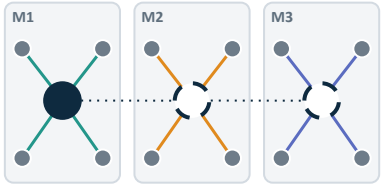
Spectral and multilevel

Spectral bisection 1990

Multilevel 1995

METIS 1998





one hub, copied on every machine

2010 – 2015

Distributed graph systems

Pregel 2010

PowerGraph 2012

PowerLyra 2015

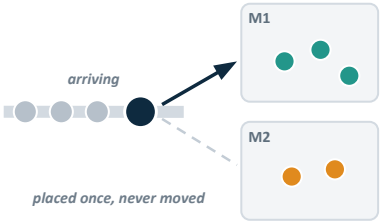
2012 – 2015

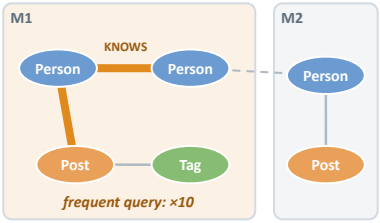
Streaming

LDG 2012

Fennel 2014

HDRF 2015





2017 – 2024

Workload and semantics

TAPER 2017

Loom 2018

MPC 2022

Shi et al. 2024

2019 – 2026

Learned and massive scale

GAP 2019

KaMinPar 2021

BuffCut 2026



Why Is Partitioning Required?

As graphs outgrow a single machine, they must be split for storage, query processing and reasoning

1 Storage

The graph does not fit (scale)

The need

Billions of nodes and edges, plus properties and indexes, exceed the memory and disk of one server.

What partitioning gives

Each machine stores a balanced share of the graph.

2 Query processing

Many queries at once (parallelism)

The need

Traversals and analytics such as PageRank or breadth-first search run concurrently and touch large parts of the graph.

What partitioning gives

Machines answer their parts in parallel; good placement keeps traversals local.

3 Reasoning

Rules need connected data

The need

Checking keys and dependencies, and inferring new facts, needs every node and edge of a pattern at once.

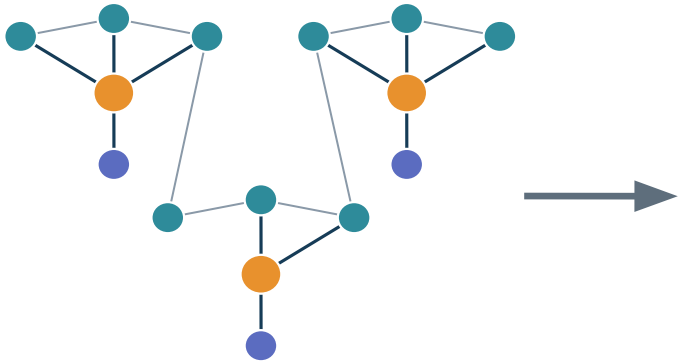
What partitioning gives

Keeping the data of a constraint together lets checks and inferences run on one machine.

The price: every edge between two machines turns a local step into network communication, so how we cut decides the cost.

What Partitioning Does to a Property Graph

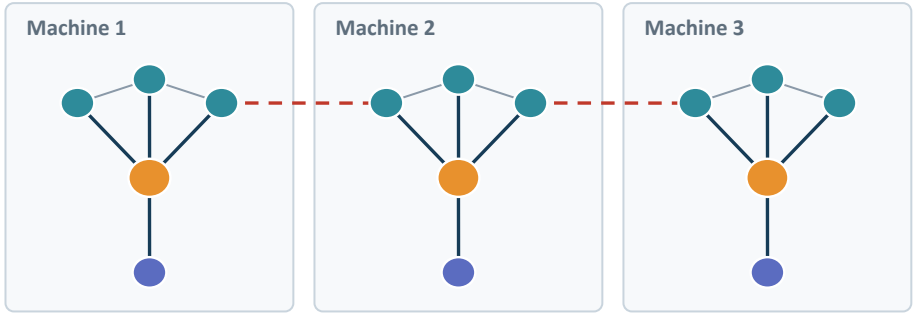
(a) Property graph G on a single machine



:Person :Company :City

Edges: :WORKS_AT, :LOCATED_IN (**dark**), :KNOWS (**light**)

(b) k = 3 partitions; dashed edges cross machines

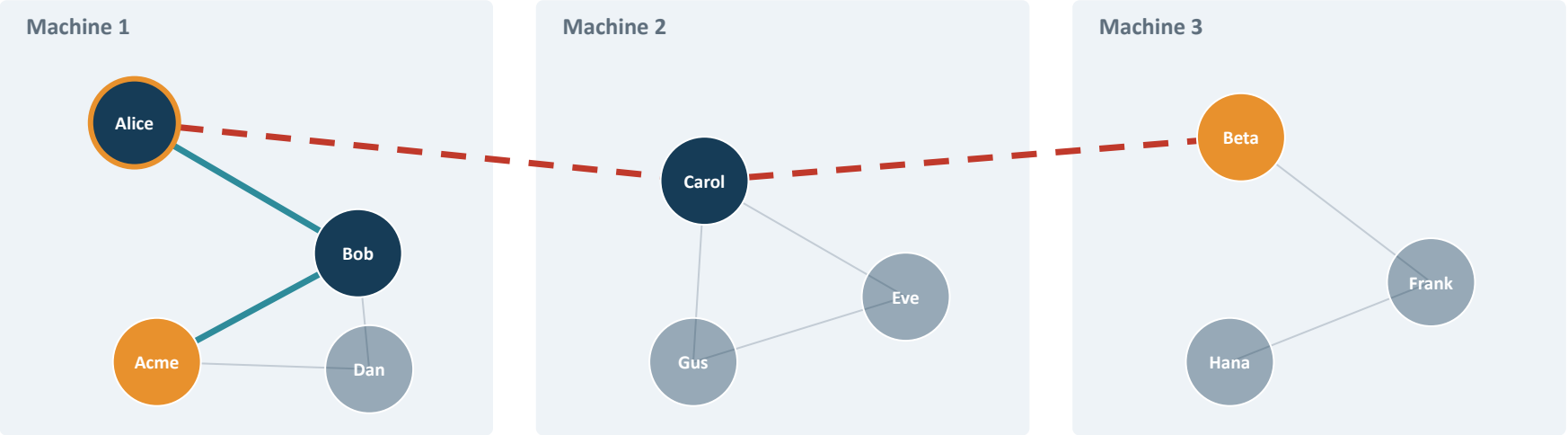


Locality. Queries are traversals. A traversal along an edge whose endpoints live on different machines becomes a network round-trip.

See e.g. Karypis & Kumar, SIAM J. Sci. Comput. 1998; Gonzalez et al., OSDI 2012; Firth & Missier, DAPD 2017.

How a Distributed Query Runs?

```
MATCH (a:Person {name:'Alice'})-[:KNOWS]->(f:Person)-[:WORKS_AT]->(c:Company) RETURN f.name, c.name
```



local Alice → Bob → Acme: same machine, 0 network hops

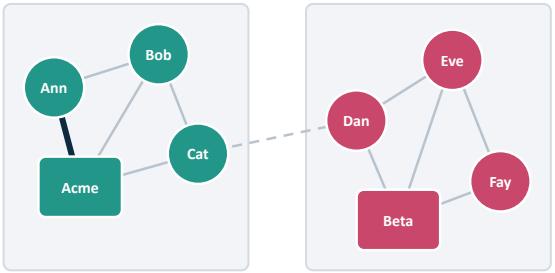
remote Alice → Carol → Beta: 2 network hops

Why partitioning matters: the same query is cheap or expensive depending only on where its data was placed.

When Does a Query Cost?

Q1. Local

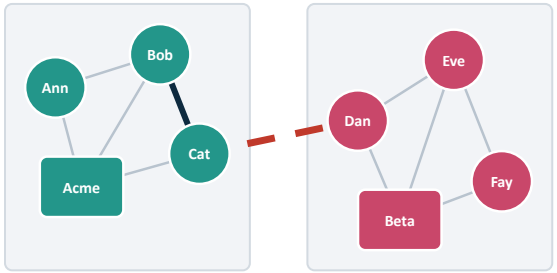
```
MATCH (:Person {name:'Ann'})
      -[:WORKS_AT]->(c)
RETURN c.name
```



1 machine, 0 network hops

Q2. Remote hop

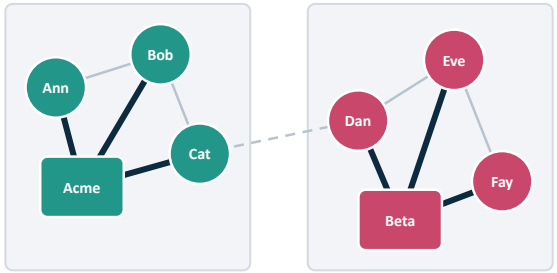
```
MATCH (:Person {name:'Bob'})
      -[:KNOWS]-(f)-[:KNOWS]-(g)
RETURN g.name
```



Cat → Dan crosses machines: 1 remote hop

Q3. All machines

```
MATCH (p:Person)-[:WORKS_AT]
      ->(c:Company)
RETURN c.name, count(p)
```



runs everywhere; waits for the slowest machine

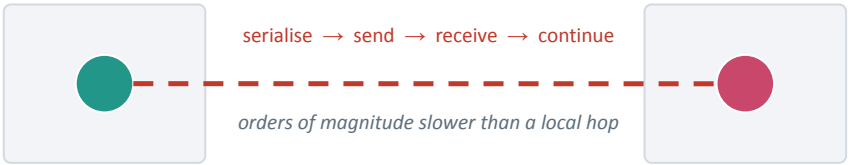
A query costs when its traversal crosses a cut edge, or it must touch many machines.

Why Does it Cost?

Local hop



Remote hop



Graph analytics

on any graph, for example PageRank or connected components: every edge is used

Communication messages per iteration grow with the number of cut edges
Synchronisation a barrier ends each iteration: all wait for the slowest

Imbalance an overloaded machine slows down every iteration

Pattern queries

typical of property graph databases: MATCH follows only some labels

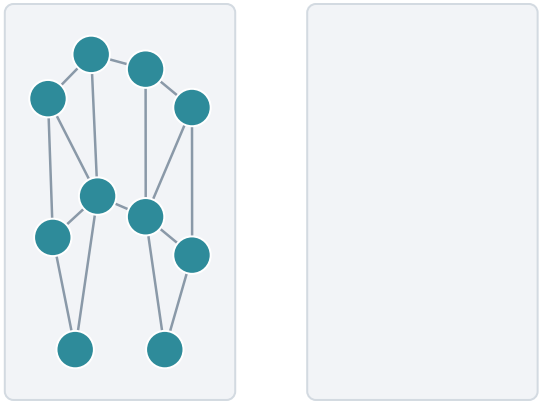
Traversed cut edges only cut edges the query follows cost anything

Shipped results partial matches grow with each hop and cross the network
Merge and lookups remote properties and the final merge add latency

Consequence Graph analytics: minimise all cut edges. Pattern queries: minimise the cut edges that queries traverse.

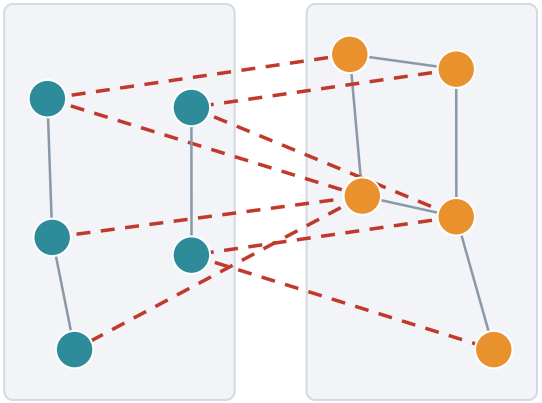
The Core Tension: Locality vs Balance

Locality



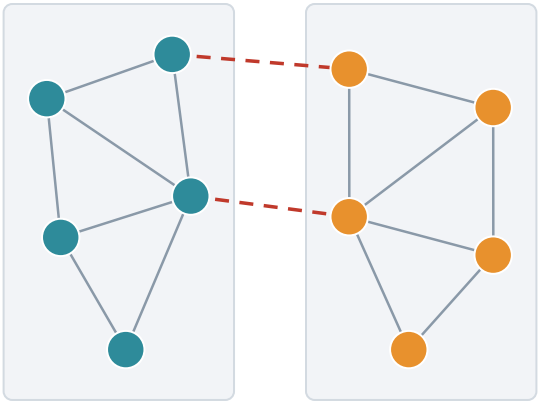
Cut edges **0** Load **10 : 0**

Balance Only



Cut edges **8** Load **5 : 5**

Locality and Balance



Cut edges **2** Load **5 : 5**

The trade-off Cutting fewest edges alone puts everything on one machine; balance alone scatters the graph. We need both.

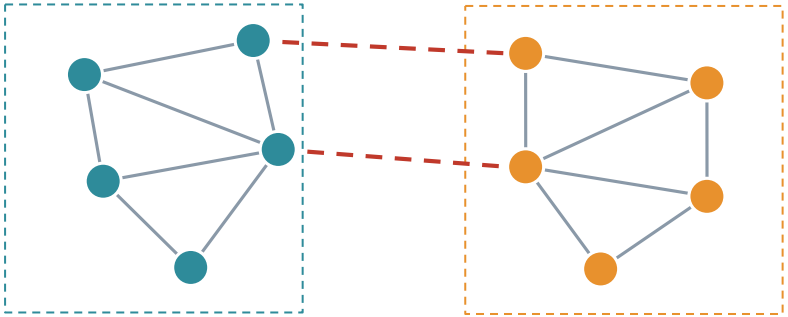
The Graph Partitioning Problem

1. Balanced k-way partitioning

Split the graph into **k equal-sized parts** and cut as **few edges** as possible.

```
minimise edge-cut   s.t. equal parts (±ε)
```

Example for balanced k=2 partitions



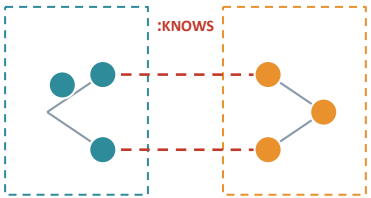
e.g., Vertices per partition = 5 and Edge-cut = 2

2. Workload-aware partitioning

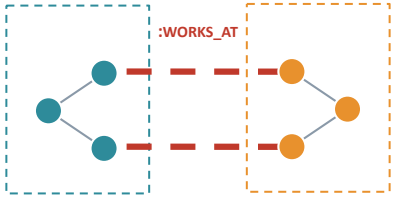
Same balance, but each edge has a **weight**: how often queries use it. Cut the **least-used** edges.

```
minimise Σ weight of cut edges   s.t. same balance
```

Example for weighted edge-cut and different costs



e.g., edge-cut = 2 and cost **2**

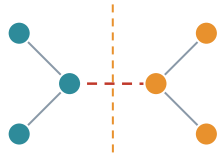


e.g., edge-cut = 2 and cost **20**

Three Design Choices Behind Every Partitioner

1 **What is placed on each machine?**

The partitioning model



vertices (edge-cut) | edges (vertex-cut) | hybrid

2 **How is the placement computed?**

The techniques

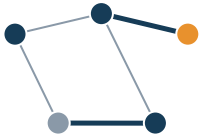


hashing | spectral, KL / FM | multilevel

streaming | label propagation

3 **Which information does it use?**

The family category



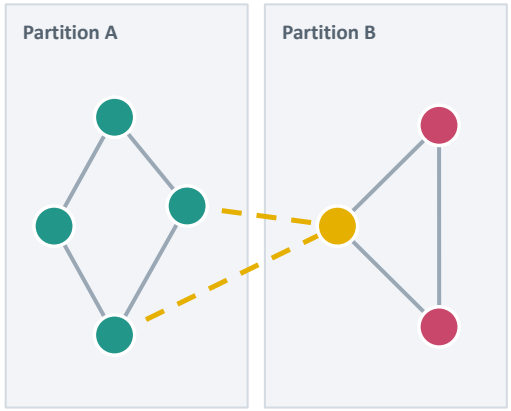
vertex IDs | graph structure | **query workload**

labels & schema

The Partitioning Model: What is Cut?

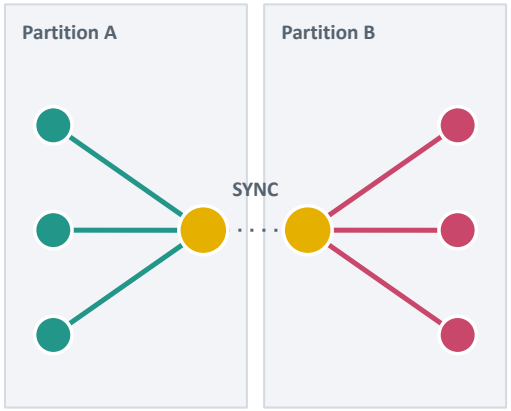
Comparing fundamental graph partitioning models and introducing the adaptive hybrid approach.

Edge-cut (vertex partitioning)



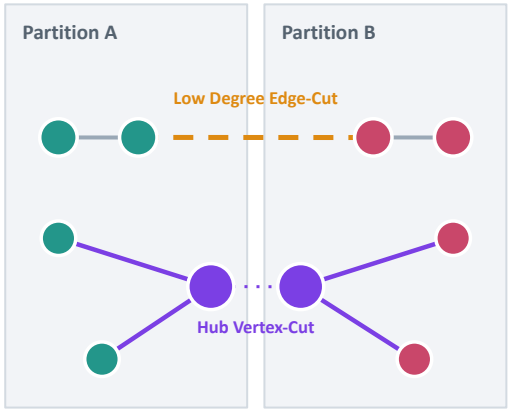
Assigns vertices to partitions while minimizing edge cuts. Best suited for balanced graphs with few hub nodes.

Vertex-cut (edge partitioning)



Assigns edges to partitions; vertices with edges in several partitions are copied (mostly hubs). Designed for skewed, power-law graph structures.

Hybrid-cut (adaptive cutting)



Selects edge-cut or vertex-cut based on graph characteristics to achieve better balance, scalability, and lower communication.

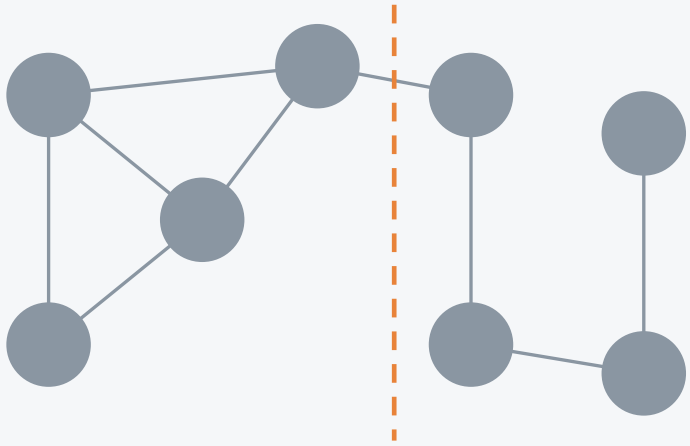
BLOCK 2.2

Structural vs Semantic Partitioning

Two ways to decide what goes together: by structure, or by meaning

What Structural Partitioning Sees

Only vertices and edges: the graph as a network of anonymous points



Minimum cut: 1 edge · balance 4 : 4

Input
The topology G = (V, E), optionally with vertex and edge weights

Goal
Split V into k parts of similar size with as few edges between parts as possible

Objective
minimise $|E_{cut}|$ subject to $|V_i| \leq (1 + \varepsilon) \cdot |V| / k$

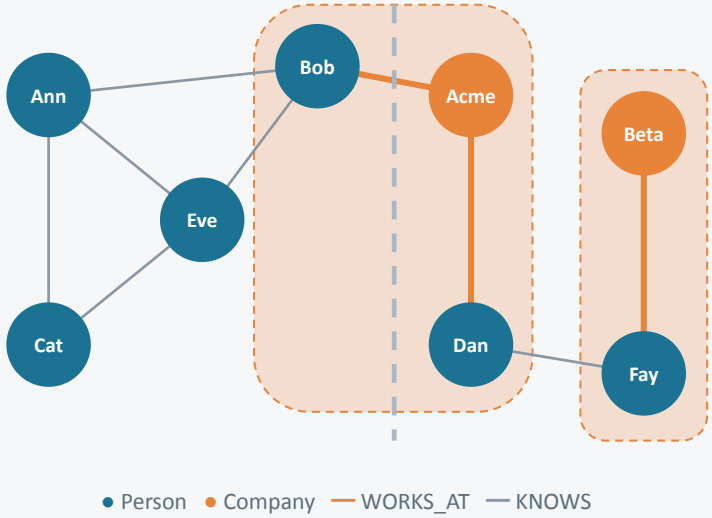
Ignores
Labels, properties, schema, constraints and the query workload

Methods in this part
Hashing, spectral and KL / FM, multilevel (METIS), streaming, label propagation

Strength and limit: works for any graph and scales well, but cannot distinguish a meaningful edge from an accidental one.

What Semantic Partitioning Sees

The same graph, plus what its nodes and edges mean



Input
The graph plus its meaning: labels and types, properties, schema (PG-Schema, SHACL), constraints (keys, dependencies)

Goal
Keep each semantic unit on one machine, for example a company with the people who work there

Objective
minimise semantic fragmentation (and edge cut) — subject to balance

Trade-off
May cut more edges than the structural optimum to keep units whole

Covered in
Part 3: constraints, algorithms and evaluation

In the example: the 1-edge minimum cut separates Bob from Acme and splits the unit {Bob, Dan, Acme}; a semantic partitioner keeps it whole.

Structural vs Semantic Partitioning

Two ways to decide which data belongs on the same machine

Aspect	Structural partitioning	Semantic partitioning
Decides with	Graph topology: vertices and edges only	Meaning: labels, types, schema, constraints
Optimises	Fewest cut edges, balanced load	Meaningful units kept on one machine
Unit of placement	A single vertex or edge	A class, a pattern or a constraint scope
Typical methods	Hashing, METIS, streaming, label propagation	Schema-based, characteristic sets, constraint-aware
Strength	General, scalable and well studied	Local queries, constraint checks and reasoning
Weakness	Blind to meaning: may split a key or a rule	Needs a schema or constraints; may cut more edges

Roadmap: *Part 2 covers the structural column; Part 3 adds meaning and constraints.*

BLOCK 2.3

Classical Techniques

Hashing, spectral, multilevel, streaming, label propagation, and related work

Classical Partitioning Techniques at a Glance

Approach	Quality	Scalability	Memory	Dynamic graphs	Uses semantics or workload
Hash / random	low	very high	O(1) state	yes	no
Spectral, KL & FM	high	low	whole graph	no	no
Multilevel (METIS, KaHIP)	very high	medium–high	whole graph	re-run	no
Streaming (LDG, Fennel, HDRF)	medium	very high	partition state	partially	no
Label propagation (Spinner)	medium	high	distributed	yes	no

Classical Partitioning: Hashing

Vertex hashing

Malewicz et al. 2010 (Pregel)

- part(v) = h(id(v)) mod k
- Edges are never read
- Any machine locates v by recomputing the hash

Constant time and balanced, but each edge is cut with probability 1 − 1/k

Random vertex-cut

Gonzalez et al. 2012 (PowerGraph)

- Hash edges instead of vertices
- A vertex is copied to every machine holding one of its edges
- Cost: the replication factor

Balances edges well on power-law graphs; hubs get many copies

Grid and degree-based

Jain et al. 2013 · Xie et al. 2014

- Grid: a vertex may only use one row and one column of a grid
- DBH: hash each edge by its lower-degree endpoint

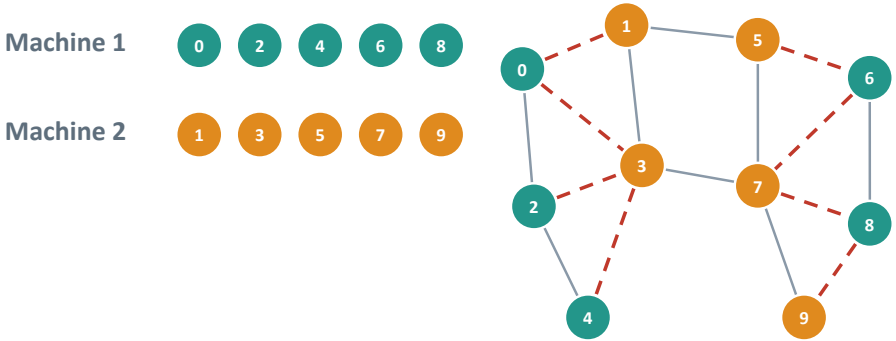
Still hash-based, but fewer copies than random vertex-cut

Hashing: Worked Example

Place each vertex by a function of its identifier alone; the edges are never read.

`part(v) = h(id(v)) mod k`

example: k = 2, h(id) = id · same graph in the next examples, to compare cuts



Result: **8 of 16 edges cut**, load 5 : 5

How it works

1. Compute the hash of the vertex identifier. No edge or property is read.
2. Assign the vertex to partition h(id) mod k. Random partitioning draws the partition uniformly instead.
3. With hashing, any machine finds a vertex by re-computing the hash; random placement needs a lookup table.

STRENGTH

Balanced in expectation; constant time per vertex; trivially parallel.

LIMITATION

Ignores edges: each edge is cut with probability 1 − 1/k.

REPRESENTATIVE ALGORITHMS

Hash partitioning (Malewicz et al., Pregel, 2010); random vertex-cut (Gonzalez et al., PowerGraph, 2012)

Classical Partitioning: Spectral, KL and FM

Spectral bisection

Fiedler 1973 · Pothen, Simon & Liou 1990

- Laplacian $L = D - A$
- Take the Fiedler vector x_2 (2nd smallest eigenvalue)
- Split vertices at the median of x_2

Global and elegant, but eigenvectors are costly on huge graphs

Kernighan–Lin

Kernighan & Lin 1970

- Start from any bisection
- Swap the vertex pair with the largest gain in cut
- Lock swapped vertices; keep the best prefix of swaps

Strong refiner; roughly quadratic work per pass

Fiduccia–Mattheyses

Fiduccia & Mattheyses 1982

- Move single vertices, not pairs
- Bucket structure gives the best move in O(1)
- Balance enforced as a constraint

Linear time per pass; the refiner inside METIS-style tools

Spectral, KL and FM: Worked Example

Spectral methods compute a global split from the graph Laplacian; KL and FM improve a split by local moves.

Spectral: vertices ordered by Fiedler value (connected vertices get similar values)



KL: swap the pair with the highest gain



How it works

1. Spectral: compute the Fiedler vector, the eigenvector of the second-smallest eigenvalue of the Laplacian L = D − A.
2. Sort vertices by their value and split at the median: a balanced bisection. Recurse for k parts.
3. KL / FM: from any bisection, swap (KL) or move (FM) the vertices with the highest gain, lock them, keep the best state.

STRENGTH

High-quality cuts; FM is the refinement step inside multilevel partitioners.

LIMITATION

Eigenvectors and gain passes are costly; the whole graph must be in memory.

REPRESENTATIVE ALGORITHMS

Spectral bisection (Pothen et al., 1990); KL (Kernighan & Lin, 1970); FM (Fiduccia & Mattheyses, 1982)

Classical Partitioning: Multilevel

1 Coarsening

Karypis & Kumar 1998

- Contract a matching, typically heavy-edge matching
- Vertex and edge weights add up
- Repeat until the graph is small

Keeps the structure of the graph at every scale

2 Initial partitioning

Hendrickson & Leland 1995

- Partition the smallest graph directly
- Spectral bisection or greedy growing
- Cheap: the graph has few vertices

Quality here matters less than refinement later

3 Uncoarsening

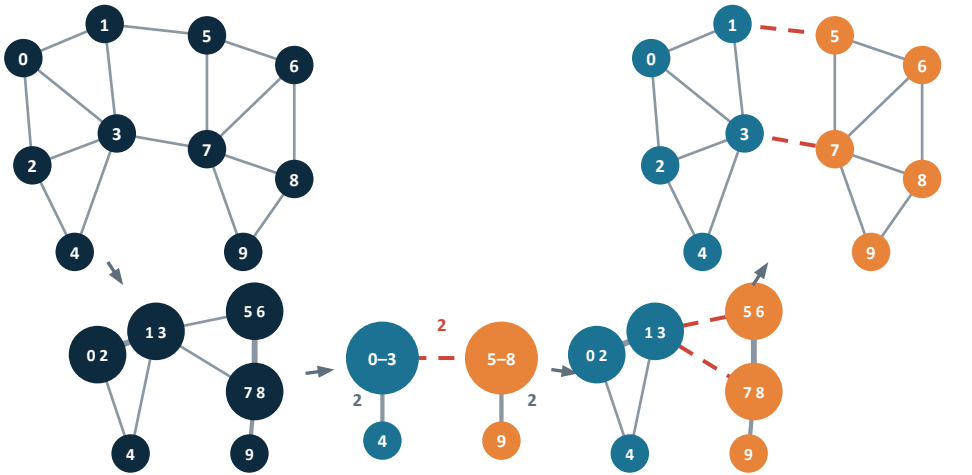
METIS · Scotch · KaHIP · KaMinPar

- Project the partition back level by level
- Refine with FM at every level
- Moves only boundary vertices

Best known quality for the time; needs the whole graph

Multilevel: Worked Example

Shrink the graph, partition the small graph, then project back and refine at every level.



1 Coarsen: 10 → 6 → 4 vertices **2 Initial partition:** 5 : 5 **3 Uncoarsen + FM:** no better move, cut 2

Result: 2 of 16 edges cut, load 5 : 5 · same as Kernighan–Lin; hashing cut 8

How it works

1. Coarsen: merge pairs of neighbouring vertices, preferring the heaviest edges, and repeat; merged vertices and edges add up their weights.
2. Initial partition: partition the small graph directly, for example with spectral bisection or greedy growing.
3. Uncoarsen: project the partition back level by level and refine it with FM at every level.

STRENGTH	LIMITATION	REPRESENTATIVE ALGORITHMS
Best known quality–speed trade-off; the de facto standard offline.	Needs the whole graph; must be recomputed when the graph changes.	METIS, ParMETIS (Karypis & Kumar, 1998); Scotch (Pellegrini & Roman, 1996); KaHIP (Sanders & Schulz, 2013); KaMinPar (Gottesbüren et al., 2021)

Classical Partitioning: Streaming

LDG

Stanton & Kliot 2012

- Vertices arrive one by one
- Pick the partition with most placed neighbours
- Score: $|N(v) \cap P| \cdot (1 - |P| / C)$

Simple and fast; quality depends on the stream order

Fennel

Tsourakakis et al. 2014

- Same idea as LDG
- Score: $|N(v) \cap P| - \alpha \gamma |P|^{(\gamma-1)}$
- Load penalty tunes locality versus balance

Usually cuts fewer edges than LDG

HDRF

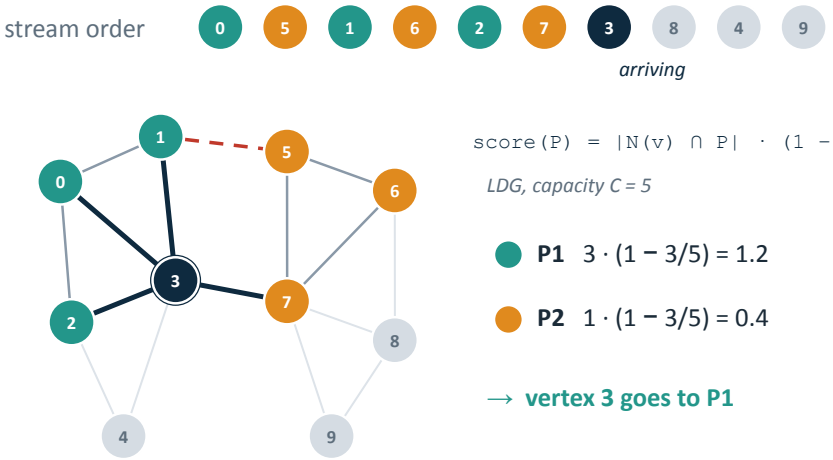
Petroni et al. 2015

- Edges arrive one by one
- Builds a vertex-cut
- Replicates the higher-degree endpoint first

Suited to power-law graphs with hubs

Streaming: Worked Example

Assign each vertex once, as it arrives, using only what has already been placed.



Order matters: on this graph LDG cuts 2 edges with order 0, 1, …, 9 and 9 edges with order 3, 7, 0, 8, 1, 6, 2, 9, 4, 5.

How it works

1. Vertices (or edges) arrive one at a time, e.g. while the graph is loaded.
2. Each is placed immediately using only its already-placed neighbours and the current partition loads.
3. The choice is final. LDG weights neighbours by remaining capacity; Fennel uses a load penalty instead.

STRENGTH	LIMITATION	REPRESENTATIVE ALGORITHMS
One pass, little memory; fits naturally into data loading.	Decisions are final and depend on the stream order.	LDG (Stanton & Kliot, 2012); Fennel (Tsourakakis et al., 2014); HDRF, edge streaming (Petroni et al., 2015)

Classical Partitioning: Label Propagation

Label propagation

Raghavan, Albert & Kumara 2007

- Each vertex has a label
- Adopt the label most frequent among neighbours
- Repeat until labels stabilise

Designed for communities; no balance guarantee

Spinner

Martella et al. 2017

- Labels = partitions
- Penalise labels of loaded partitions
- Vertex-centric on Giraph; restarts after changes

Balanced, scalable, adapts to graph changes

Ja-be-Ja

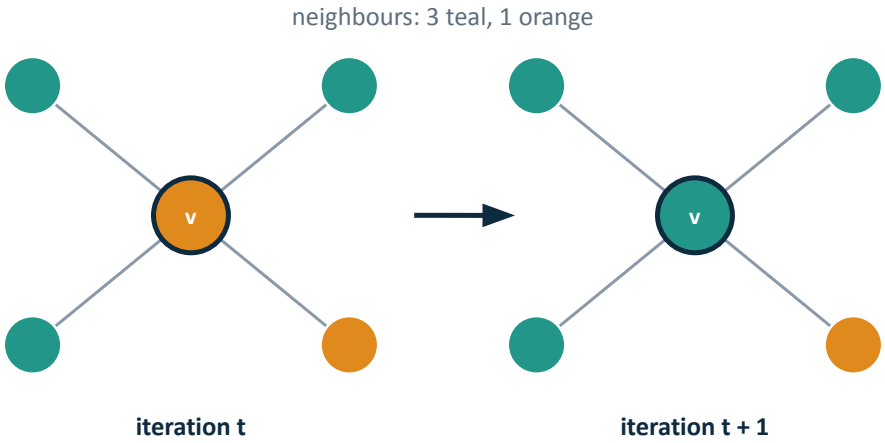
Rahimian et al. 2013

- Vertices swap labels with neighbours or random peers
- Swaps keep partition sizes fixed
- Simulated annealing escapes local optima

Fully decentralised; local search, not propagation

Label Propagation: Worked Example

Each vertex repeatedly adopts the partition most common among its neighbours, as long as that partition has room.



v moves to green if the green partition has room (Spinner: a load penalty).

How it works

1. Start from any assignment, e.g. random; each partition is a label.
2. Each vertex adopts the label most frequent among its neighbours, penalised by the load of that partition.
3. Repeat until labels stabilise. Runs vertex-centric in parallel (Pregel, Giraph); restarts cheaply after changes.

STRENGTH

Scales out across the cluster; adapts incrementally to graph changes.

LIMITATION

May stop in a local optimum; many iterations; lower quality than multilevel.

REPRESENTATIVE ALGORITHMS

LPA (Raghavan et al., 2007); Spinner (Martella et al., 2017); related: Ja-be-Ja (Rahimian et al., 2013)

Partitioning Model and Technique: Two Separate Choices

Kind	Work	Partitioning model	Technique	Kept together	Cut or copied
Partitioning tools	**METIS (Karypis & Kumar, 1998)**	vertices (edge-cut)	multilevel	dense regions	few edges cut
	KaHIP (Sanders & Schulz, 2013)	vertices (edge-cut)	multilevel, stronger refinement	dense regions	fewer cut edges
Graph processing systems	**Pregel (2010)**	vertices (edge-cut)	hashing	nothing in particular	most edges cut
	PowerGraph (2012)	edges (vertex-cut)	hashing (random vertex-cut)	nothing in particular	many vertex copies
Partitioning algorithms	**Fennel (2014)**	vertices (edge-cut)	streaming	placed neighbours	some edges cut
	HDRF (2015)	edges (vertex-cut)	streaming	low-degree edges	hubs copied
	PowerLyra (2015)	hybrid	hashing by vertex degree	low-degree vertices	only hubs copied
	Sheep: Margo & Seltzer (2015)	edges (vertex-cut)	elimination-tree splitting	edges close in the tree	vertex copies
	Spinner (Martella et al., 2017)	vertices (edge-cut)	label propagation	neighbours kept together	some edges cut
	HEP (2021)	edges (vertex-cut)	in-memory + streaming	dense neighbourhoods	hubs copied
	CUTTANA (2024)	vertices (edge-cut)	streaming + refinement	neighbours in a buffer	few edges cut
RDF partitioners, structural	**Huang, Abadi & Ren (2011)**	vertices (edge-cut)	multilevel (METIS) + copies	a subject's triples	border triples copied
	WARP (Hose & Schenkel, 2013)	vertices (edge-cut)	METIS + workload copies	often-queried triples	workload copies
	TriAD (Gurajada et al., 2014)	vertices (edge-cut)	METIS + summary graph	dense regions	some edges cut
	Wu and colleagues (2015)	paths	merging paths that share vertices	triples of a path	vertex copies

Same data stored either way: the partitioning model decides what stays together; the technique decides where each piece goes.

How Partitions are Evaluated

Level	Metric	What it measures	Why it matters	Goal
Partition quality *on the partition, before queries run*	**Edge-cut**	Edges between machines	A cut edge is a remote hop	*lower*
	Balance	Largest part vs. average part	Slowest machine sets the pace	*equal*
	Replication factor	Copies per vertex	Extra storage and sync cost	*1*
	Partitioning time	Time to compute the partition	Paid again after updates	*lower*
Run-time cost *while the workload runs*	**Communication cost**	Data sent between machines	Main source of latency	*lower*
	Execution time	Query latency, throughput	The final test	*lower*
	Memory usage	Storage per machine	Limits what fits	*lower*

Partition-quality metrics predict run-time cost; run-time cost is the final test.

Classical Techniques in Property Graph Systems

System or work	Technique (as in Part 2)	Partitioning model	Meaning used?
JanusGraph	hashing: random vertex placement	edge-cut; vertex-cut for chosen labels	no
TigerGraph	hashing, automatic	edge-cut; out-edges stay with their vertex	no
Neo4j composite databases	manual: designer chooses sharding keys	subgraphs chosen by hand	by hand only
Neo4j Infinigraph (2025)	property data sharded apart from the structure	properties; structure kept whole	no
Hermes (Nicoara et al., EDBT 2015), on Neo4j	moves vertices to where most neighbours are, like FM	edge-cut	no
TAPER (Firth & Missier, 2017)	refines a hash or METIS split by swaps, like Kernighan–Lin	edge-cut	labels in queries
Loom (Firth et al., EDBT 2018)	streaming, like LDG and Fennel	edge-cut	labels in queries

Property graph systems reuse the classical techniques. A few use the labels in queries; none uses the schema, keys or dependencies. That is the gap Part 3 addresses.

Semantic Partitioning Started in RDF

Semantic partitioning: the meaning of the data (classes, properties, schema) decides what is placed together; structure and balance only refine it

1 Semantic-Aware Partitioning on RDF Graphs

Xu, Wang, Wang, Yang and Feng · APWeb-WAIM 2017

- Builds a schema graph of the classes, from rdf:type
- Ranks the classes with classRank, a PageRank-style score
- Hashes instances of the top-ranked classes to machines; linked instances follow

Goal: fewer crossing edges by using types, not only topology

2 DIAERESIS

Troullinou, Agathangelos, Kondylakis, Stefanidis and Plexousakis · Semantic Web 2024

- Picks the most important classes (centrality and number of instances) as centroids
- Assigns every other class to the centroid it depends on most; instances follow their class
- One table per property; Spark SQL reads only the partitions it needs

Goal: less data accessed per query, on Spark

3 SemPart

Saidi, Matallah, Mesmoudi and Bellatreche · World Wide Web 2025

- Logical fragments from characteristic sets: entities sharing the same set of properties
- An administrator splits and groups them with RDPAL, a partitioning language
- METIS or MPC then allocates them to machines

Goal: semantic units, with a human in the loop

Common idea: they reuse familiar techniques (ranking, centrality, hashing), but run them on the schema and place classes, not single vertices. Next: what property graph systems do today.

From RDF to Property Graphs: What Changes for Partitioning

Why RDF methods do not carry over directly

Aspect	RDF (semantic web)	Property graphs
Data unit	triple: subject, predicate, object	node or edge with labels and properties
Semantics from	IRIs, vocabularies, RDFS and OWL	labels, PG-Schema, keys and dependencies
Edge attributes	reification or RDF-star	native properties on edges
Constraints	SHACL, ShEx	PG-Keys, GFDs, GGDs

Foundations: What to Take Away

1 **Placement decides cost**

Every cut edge a query follows becomes a network hop.

2 **Locality vs balance**

Balanced k-way partitioning is NP-hard; practical methods are heuristics.

3 **Classical methods see only structure**

Hashing, spectral, multilevel, streaming, label propagation: vertices and edges, nothing else.

The open question : Classical partitioning sees only structure. What changes when it also sees meaning and queries?

PART 3

Semantic-Aware Partitioning

Distributing property graphs by meaning, structure, and workload.

PART 3

From Structure to Semantics

What changes when graph partitioning considers semantics, not just structure?

Graph partitioning is not only about structure, but also about understanding and preserving the meaning of the data.

IN THIS PART

01 Partitioning by Structure

02 Why Meaning Matters

03 Partitioning with Meaning

04 Evaluating Partitioning Algorithms: Metrics

01

Partitioning by Structure

What can a partitioner achieve with vertices and edges alone?

Classical algorithms balance the load and cut few edges.

IN THIS PART

01 RECAP: Property Graphs, Foundations, Techniques

02 Partitioning Algorithms by Family

03 Two Multilevel Strategies: METIS & KAHIP

Property Graphs

The model, how we query it, how we constrain it, and how it relates to RDF

1 The model

- **Nodes and edges** are objects with identity
- **Labels** and key–value **properties** on both
- Formally (V, E, η, λ, ν): edges, labels, properties

2 Querying

- **Patterns**: MATCH (p:Person)-[:WORKS_AT]->(c)
- Cypher, Gremlin, PGQL; standards **GQL** (2024), **SQL/PGQ** (2023)
- A workload is a set of patterns

3 Schemas and constraints

- **PG-Schema** (2023): node and edge types, KEY, MANDATORY
- **PG-Keys** (2021): identity of nodes and edges
- RDF counterpart: **SHACL** shapes

4 Property graphs and RDF

- PG ⟶ RDF: **PG2RDF**
- RDF ⟶ PG: **rdf2neo**, **G2GML**, **S3PG** (via schemas)
- Same questions: data placement, constraint preservation

Key idea: Queries, constraints and partitioning cost are all about patterns: a match split across machines costs network hops.

Foundations of Graph Partitioning

Why we split, what a cut costs, and the choices behind every partitioner

Why partition?

- **Storage**: the graph does not fit on one machine
- **Query processing**: machines answer in parallel
- **Reasoning**: rules need all the data of a pattern

What a cut costs

- A cut edge a query follows is a **network hop**
- Minimise edges crossed by **queries traverse**
- **Locality vs balance**: NP-hard, so heuristics

Three design choices

- **Model**: edge-cut, vertex-cut, hybrid
- **Technique**: hashing, spectral, multilevel, streaming, label propagation
- **Information**: IDs, structure, workload, labels

HOW PARTITIONS ARE EVALUATED

Partition quality *before queries run*

edge-cut | balance | replication factor | partitioning time

Run-time cost *while the workload runs*

communication | execution time | memory usage

Key idea: Placement decides cost: the same query is cheap or expensive depending only on where its data was placed.

What a Structural Cut Can Split

A structural cut sees edges; meaning lives in groups of nodes and edges

1 What a structural cut sees

- **Vertices and edges** only
- Minimises **cut edges** under a balance constraint
- Cannot tell a meaningful edge from an accidental one

2 What a property graph carries

- **Labels, types and properties** on nodes and edges
- **Keys**, **functional** and **generating dependencies**
- One constraint often spans **several vertices and edges**

WHAT A STRUCTURAL CUT CAN SPLIT

Identity *Is it the same entity?*

Two customers share one email: are they the same person?

split → the key check needs both machines

Consistency *Do the values agree?*

Two offices of one company must list the same headquarters country.

split → the dependency check needs both machines

Inference *What new fact follows?*

Works at a company located in a country ⇒ lives in that country.

split → the rule needs both machines

Key idea: Partition property graphs so that the data each key, dependency and rule needs stays on one machine, while the load stays balanced.

Classical Techniques

Classical methods see only structure; property graph systems reuse them

Technique	Quality	Scalability	Uses meaning
Hash / random	low	very high	no
Spectral, KL & FM	high	low	no
Multilevel (METIS, KaHIP)	very high	medium–high	no
Streaming (LDG, Fennel, HDRF)	medium	very high	no
Label propagation (Spinner)	medium	high	no

In property graph systems today

- **JanusGraph, TigerGraph**: hashing
- **Neo4j**: shards chosen by hand
- **TAPER, Loom**: labels only through queries

Semantic-Aware Partitioning in RDF

- **Xu et al.** 2017, **DIAERESIS** 2024, **SemPart** 2025
- They place **classes**, not single vertices

The gap: No prior work uses graph constraints, such as keys and dependencies, to guide property graph partitioning.

Partitioning Algorithms by Family (1/2)

Families that decide from structure, identifiers, a learned model or neighbourhoods

Family	Technique	Algorithms	Graph
1 Classical *spectral, local search, geometric*	Local search	**Kernighan–Lin** (Kernighan & Lin, 1970), **Fiduccia–Mattheyses** (Fiduccia & Mattheyses, 1982)	**General**
	Geometric (coordinates)	**RCB** (Berger & Bokhari, 1987), **RIB** (Simon, 1991)	
	Spectral	**Recursive Spectral Bisection** (Pothen et al., 1990), **Spectral Clustering** (Ng et al., 2001)	
2 Multilevel *coarsen, partition, refine*	Multilevel	**SCOTCH** (Pellegrini & Roman, 1996), **METIS** (Karypis & Kumar, 1998), **KaHIP / KaFFPa** (Sanders & Schulz, 2011), **KaFFPaE** (Sanders & Schulz, 2012), **KaMinPar** (Gottesbüren et al., 2021)	**General**
	Parallel multilevel	**ParMETIS** (Karypis & Kumar, 1999), **PT-Scotch** (Chevalier & Pellegrini, 2008), **mt-METIS** (LaSalle & Karypis, 2013), **ParHIP** (Meyerhenke et al., 2017), **mt-KaHIP** (Akhremtsev et al., 2018), **dKaMinPar** (Sanders & Seemaier, 2023)	
	METIS + n-hop replication	**Huang et al.** (Huang, Abadi & Ren, 2011)	**RDF**
3 Learning-based *neural model*	Learned assignment	**GAP** (Nazi et al., 2019)	**General**
4 Hash-based *place by a hash*	Vertex hashing	**Hash on vertex ID** (Malewicz et al., 2010)	**General**
	Edge hashing (vertex-cut)	**Random vertex-cut** (Gonzalez et al., 2012), **Grid (2D)** (Jain et al., 2013), **DBH** (Xie et al., 2014)	
	Hybrid-cut (by vertex degree)	**PowerLyra** (Chen et al., 2015)	
	Semantic hashing of triple groups	**SHAPE** (Lee & Liu, 2013)	**RDF**
5 Streaming *one pass, decide once*	Vertex and edge streaming	**LDG** (Stanton & Kliot, 2012), **Fennel** (Tsourakakis et al., 2014), **HDRF** (Petroni et al., 2015)	**General**
6 Neighborhood-based *swaps, label propagation*	Vertex swaps, label propagation	**Ja-be-Ja** (Rahimian et al., 2013), **Spinner** (Martella et al., 2017)	**General**

* adapts at run time to changes in the graph or workload

Partitioning Algorithms by Family (2/2)

Families that decide from change, workload, stored data or meaning

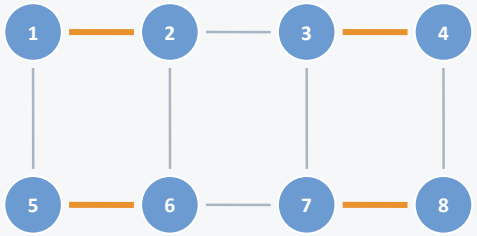
Family	Technique	Algorithms	Graph
7 Dynamic *repartition on change*	Incremental re-partitioning	**Adaptive Partitioning*** (Vaquero et al., 2014), **Leopard*** (Huang & Abadi, 2016)	**General**
8 Workload-aware *follow the queries*	Workload-driven refinement	**TAPER*** (Firth & Missier, 2017)	**General**
	Streaming, query motifs	**Loom*** (Firth et al., 2018)	
	Replication of query patterns	**WARP** (Hose & Schenkel, 2013)	**RDF**
	Fragmentation from a query log	**Partout** (Galárraga et al., 2014)	
	Adaptive redistribution	**AdPart*** (Harbi et al., 2016)	
	Read-weighted vertex migration	**Hermes*** (Nicoara et al., 2015)	**Property graph**
	Traffic-driven migration	**Adaptive Partitioning*** (Svitáková et al., 2020)	
9 Property-aware *balance stored data*	Volume-aware streaming	**Hammer** (Sakouhi et al., 2021)	**Property graph**
	Label propagation + swarm search	**LSPGP** (Cui et al., 2025)	
10 Semantic-aware *meaning decides placement*	Class ranking (classRank)	**Semantic-aware partitioning** (Xu et al., 2017)	**RDF**
	Minimum property-cut	**MPC** (Peng et al., 2022)	
	Class centroids and dependencies	**DIAERESIS** (Troullinou et al., 2024)	
	Characteristic sets + RDPAL	**SemPart** (Saidi et al., 2025)	
	Property-path decomposition	**Pang et al.** (Pang et al., 2019)	**Property graph**
	Relationship-type components	**RCP** (Shi et al., 2024)	

* adapts at run time to changes in the graph or workload · within each family: general, then RDF, then property graph, each in chronological order

METIS: The Multilevel Idea on a Small Graph

Karypis & Kumar, "A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs", SIAM J. Sci. Comput., 1998

1 Shrink (coarsening)

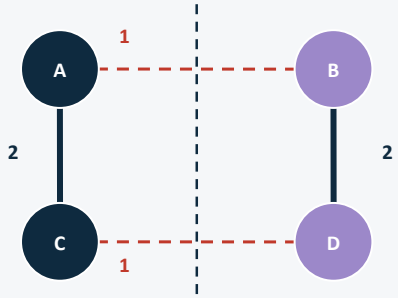


orange: matched pairs → A, B, C, D

Merge neighbours in pairs: 1–2, 3–4, 5–6, 7–8 become A, B, C, D. Repeat until few of vertices remain.

METIS uses **Heavy-Edge Matching (HEM)**

2 Cut (initial partitioning)

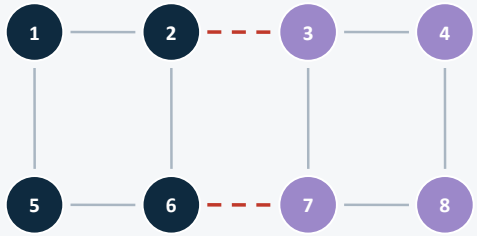


{A, C} | {B, D}: cut weight 2

Split the small graph into equal halves: {A, C} | {B, D} cuts weight 2, while {A, B} | {C, D} would cut 4.

METIS uses **Greedy Graph Growing (GGGP)**

3 Grow back and fix (refinement)



{1,2,5,6} | {3,4,7,8}: 2 cut edges

Undo the merges: {1, 2, 5, 6} | {3, 4, 7, 8}. Border vertices 2, 3, 6, 7 switch side only if the cut drops.

METIS uses **Kernighan–Lin-based refinement**

Why it works: Merged edges can never be cut, and a small graph is cheap to split well.

TAKES INTO ACCOUNT

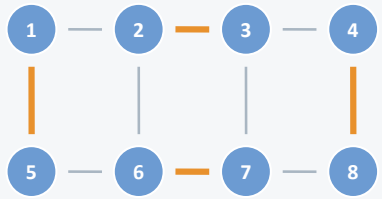
Edge-cut minimization

Vertex-weight balance per partition

KaHIP: Why Refinement Matters on a Small Graph

Sanders & Schulz, "Engineering Multilevel Graph Partitioning Algorithms", ESA 2011 (KaFFPa, the core of KaHIP)

1 Rate, match, contract

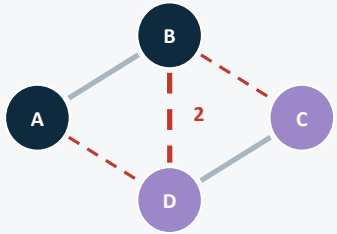


orange: matched pairs → A, B, C, D

Rate the edges, match on the ratings: pairs 1–5, 2–3, 4–8, 6–7 become A, B, C, D.

KaHIP uses **Edge-rating matching**

2 Initial partition

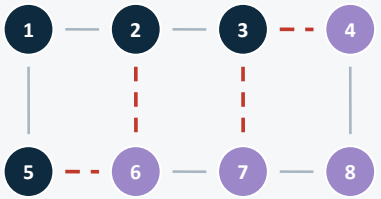


every balanced cut costs 4

Every balanced split cuts 4. Try several and keep the best: {A, B} | {C, D}.

KaHIP uses **Repeated initial partitioning**

3 Project back

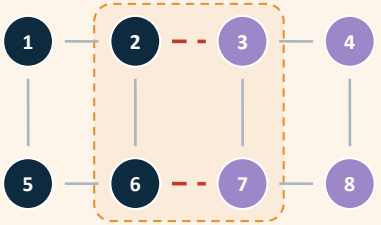


{1,2,3,5} | {4,6,7,8} · 4 cut edges

Uncoarsen: each multinode returns its vertices. The cut is unchanged: 4 edges.

KaHIP uses **Projection, level by level**

4 Refine: local search + flows



boundary region · 2 cut edges

Near the boundary, swap 3 and 6: {1, 2, 5, 6} | {3, 4, 7, 8}. Only 2 cut edges, still 4 | 4.

KaHIP uses **FM search + max-flow min-cut**

Why it works the coarse graph cannot see every good cut; refinement on the finer level finds it.

TAKES INTO ACCOUNT

Edge-cut minimization
Balance: at most (1 + ε) × average block weight

METIS vs KaHIP: Comparing Multilevel Partitioning Approaches

Same multilevel scheme, different priorities: speed or cut quality

Phase	METIS	KaHIP
Coarsening	Heavy-edge matching	matching on edge ratings
Initial partitioning	Greedy graph growing	Repeated initial partitioning, best kept
Refinement	Boundary Kernighan–Lin	FM local search + max-flow min-cut
Global search	—	F- and W-cycles: re-run the multilevel cycle
Priority	**Speed, Good quality**	**Cut quality**
Typical use	Fast default and common baseline	When the best cut matters

Takeaway: KaHIP spends more time in refinement to find cuts that METIS's coarse levels cannot see.

02

Why Meaning Matters

What Does a Property Graph Reveal Beyond Its Topology?

Labels, types and constraints say what belongs together.

IN THIS PART

01 Why Structure Alone isn't Enough

02 Beyond Graph Structure: The Role of Semantics

03 A Cut that Respects Semantics

04 Why Meaning Matters

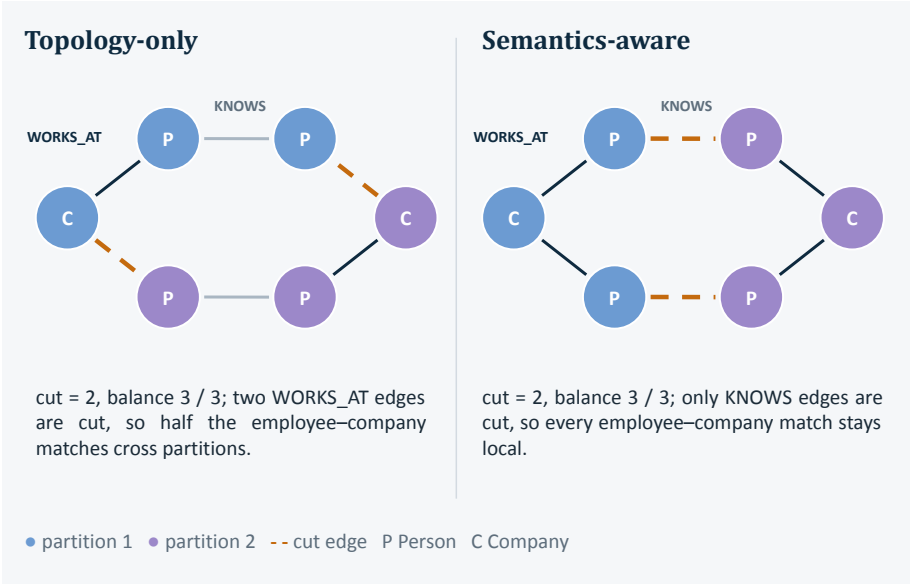
05 A Hierarchy of Semantic Signals

06 Exploiting Semantics: Labels, Schema and Constraints

07 What Partitioners Consider: The Gap

Why Structure Alone Isn't Enough

Two partitionings with the same edge-cut and the same balance, but not the same meaning



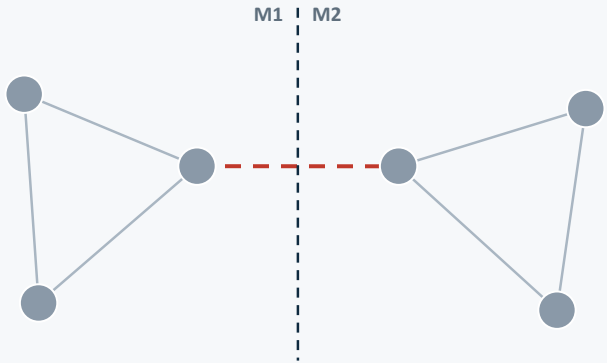
What topology-only partitioning misses

1. It sees vertices and edges, not labels, relationship types or properties.
2. Every edge costs the same: cutting KNOWS or WORKS_AT counts as one.
3. Queries follow labelled patterns; a split pattern match needs remote access.
4. Balance counts vertices, yet properties make vertices unequal in stored data.

Takeaway: A minimal cut is not necessarily a meaningful cut: structure cannot tell which edges a query needs.

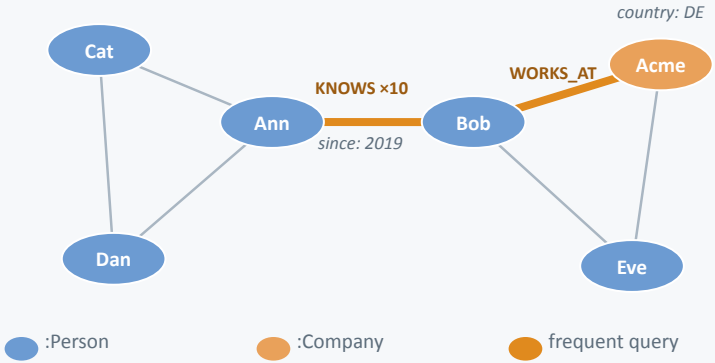
Beyond Graph Structure: The Role of Semantics

What a classical partitioner sees



Best balanced split: 3 | 3, only **1 cut edge**, the one the frequent query uses.

What the semantic graph means



Labels and types
:Person, :Company, :WORKS_AT look identical to the partitioner

Properties
since, country, timestamps are invisible

Schema / ontology
class hierarchies and cardinalities are ignored

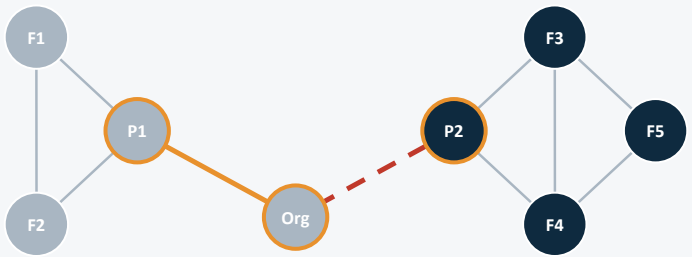
Query workload
which edges are hot and which are never traversed

Fewest cut edges is not the same as lowest query cost.

A Cut That Respects Semantics

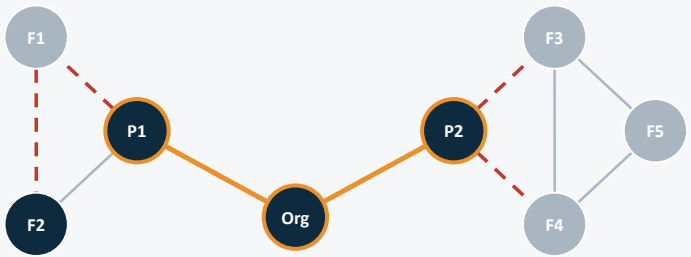
Same graph, two partitions of four nodes each

Classical: fewest cut edges



inferring KNOWS(P1, P2) needs both machines

Semantic-aware: keep the match



the dependency is checked and applied on one machine

Partition 1 dark **Partition 2** light **orange** dependency match (two persons, same Org) **red dashed** cut edge

---

The trade-off: Three more cut edges, but the constraint and its inference stay local.

Why Meaning Matters?

Graph structure reveals connectivity while semantics reveals relationships and dependencies relevant to data placement

1 Queries Follow Typed Patterns

Graph queries select specific labels, relationship types, and properties. Not every cut edge affects a query equally.

2 Not All Edges Are Equal

Different relationships contribute differently to query execution and constraint evaluation.

3 Constraints Span Several Elements

Keys and dependencies may involve several vertices, edges, and properties. Splitting them can require cross-partition checks.

4 Semantically Related Groups

Related entities and relationships can form meaningful groups that benefit from being processed together.

5 Elements Differ in Size

Properties increase storage requirements. Partition balance may need to consider data volume, not only vertex counts.

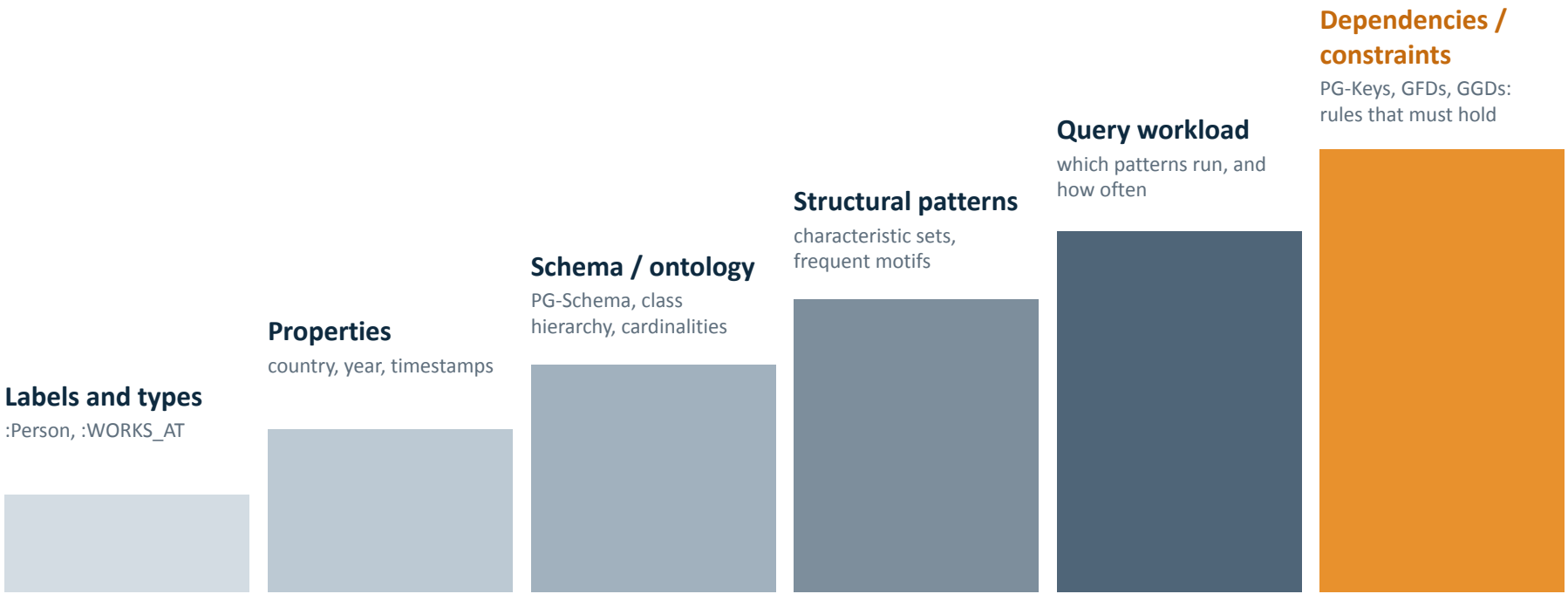
6 Semantic Information Is Available

Labels, types, properties, schemas, and declared constraints can provide guidance for partitioning

Takeaway: Structural partitioning preserves connectivity; semantic-aware partitioning additionally considers the relationships, properties, and constraints relevant to queries and reasoning .

A Hierarchy of Semantic Signals

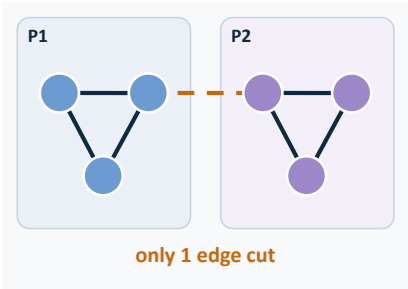
From graph information to workload and formal constraints



What Existing Partitioning Strategies Bring to Semantic-Aware Approaches?

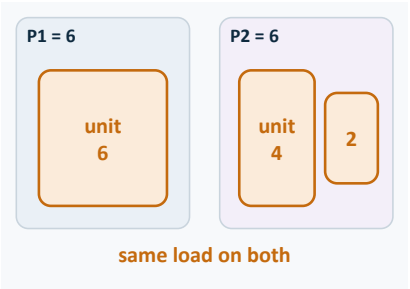
Established mechanisms provide the foundation; semantics introduces additional placement criteria.

1 Locality



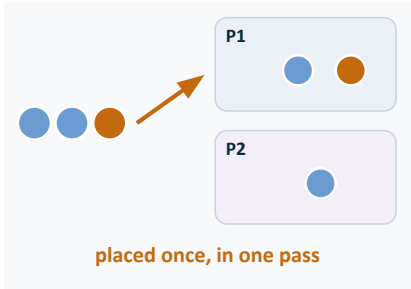
Minimize edge cuts while keeping semantically related elements together.

2 Balance



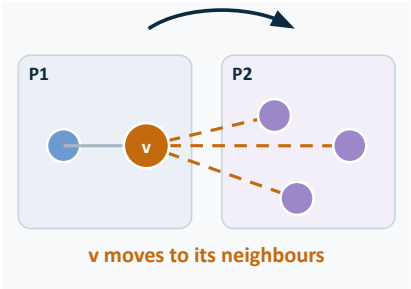
Distribute semantic groups while maintaining balanced partition weights.

3 Scalability



Extend efficient placement strategies to incorporate semantic information.

4 Adaptivity



Refine partition assignments as graph structure, workloads, or semantic requirements change.

Classical mechanisms → **Semantic guidance** → **Constraint locality**

"Beyond graph semantics, formal constraints guide which elements should be co-located for efficient validation and reasoning."

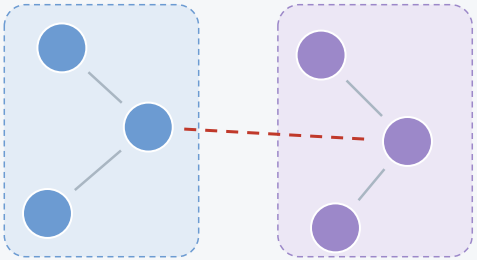
Takeaway Classical partitioning provides the mechanisms; semantics guides placement, while formal constraints introduce **dependency-aware locality**.

Exploiting Semantics: Labels, Schema and Constraints

Three ways meaning can decide what is placed together

Label-aware

Use vertex and edge types to decide placement.

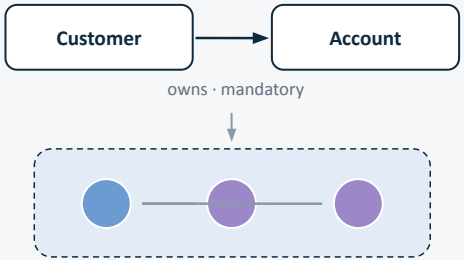


:Person | :Company

e.g. RCP (relationship labels); TAPER, Loom (vertex labels via the workload)

Schema-aware

Exploit structural patterns or schema information.

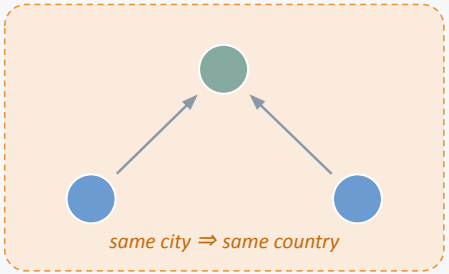


a customer with its accounts

e.g. RDF: Xu et al. (schema classes), OntoSP (ontology hierarchy); PG: PG-Schema types

Constraint-aware

Use dependencies and integrity constraints to decide what is colocated.



the GFD match stays together

e.g. keys, GFDs, GGDs: each constraint checked inside one partition

What Partitioners Consider: The Gap

Each family reads more of the graph; none yet reads its dependencies

Information	Hash	Structural	Workload-aware	Semantic-aware
Vertex identifiers	✓	—	—	—
Graph topology	—	✓	(✓)	(✓)
Labels and types	—	(✓)	(✓)	✓
Properties	—	(✓)	—	(✓)
Schema / ontology	—	—	—	(✓)
Query workload	—	—	✓	(✓)
Dependencies / constraints	—	—	—	?

✓ core input (✓) often, sometimes or optional ? potentially, not yet exploited — not used

The gap no family exploits dependencies and constraints (PG-Keys, GFDs, GGDs).

03

Partitioning with Meaning

How do partitioning methods turn workload and semantics into placement?

Workload-aware and semantic-aware algorithms, compared.

IN THIS PART

01 Integrating Semantics into Structural Partitioning

02 Extending Structural Partitioning: Existing Approaches

03 Property Graph Approaches

04 From Structural to Constraint-Aware Graph Partitioning

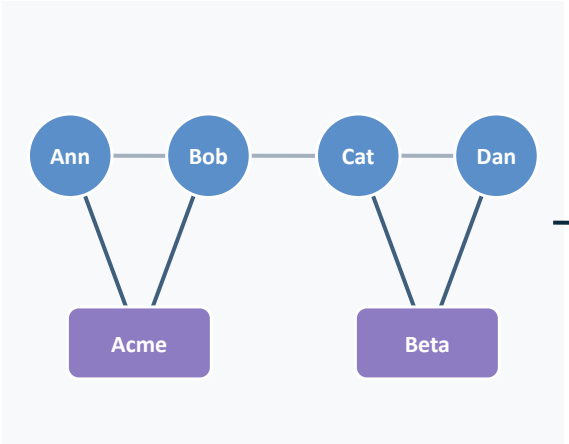
05 Beyond Graph Semantics: Understanding Formal Constraints

06 Semantic Constraints: Example

Integrating Semantics into Structural Partitioning

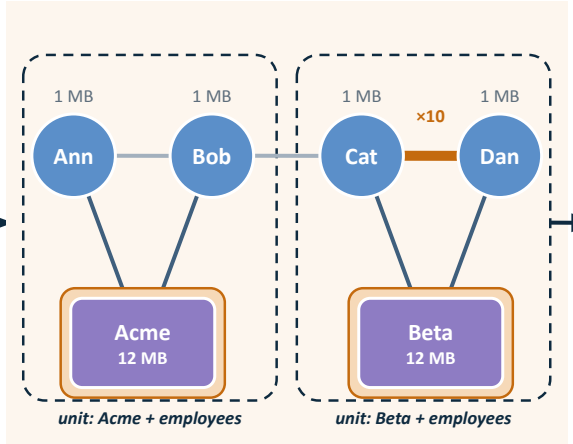
Turn property-graph information into weights and units, then let a structural partitioner do the split

1 The property graph



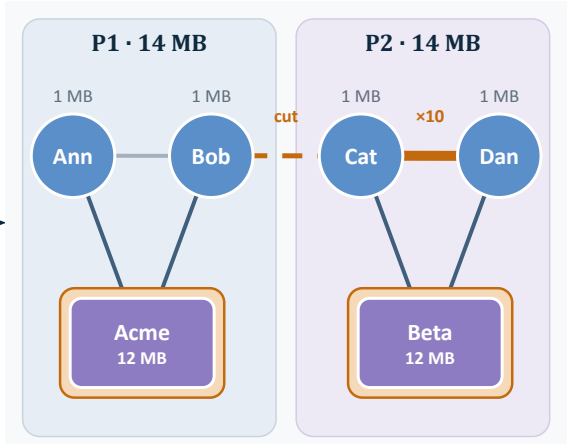
Persons and companies, linked by **KNOWS** and **WORKS_AT**. A structural partitioner sees only vertices and edges.

2 Add weights and units



Workload → edge weight (Cat–Dan ×10).
Semantics → units: each company with its employees.

3 Structural partition



A standard partitioner splits the weighted graph: units stay whole, the ×10 edge stays local, 14 / 14 MB, one KNOWS edge cut.

Extending Structural Partitioning: Existing Approaches

Six property-graph partitioners: what drives placement, when they run, and whether they use semantics or constraints

Algorithm	Strategy	Placement criteria	Execution	Main objective	Semantic	Constraints
Hammer 2021	Streaming	Data volume	One-pass	Storage balance, cut reduction	No*	No
LSPGP 2025	Label propagation + optimization	Property storage sizes	Offline	Cut reduction, storage balance	No*	No
Hermes 2015	Vertex migration	Read requests	Runtime	Communication reduction, load balance	No	No
Adaptive Partitioning 2020	Adaptive migration	Query traffic	Runtime	Communication reduction	No	No
Pang et al. 2019	Path decomposition	Semantic property paths	Offline	Path-query locality	**Yes**	No
RCP 2024	Type-based grouping	Relationship types	Offline	Pattern-query locality	**Yes**	No

What they read

storage size (Hammer, LSPGP), query load (Hermes, Adaptive), or meaning (Pang, RCP)

When they decide

while loading (Hammer), offline (LSPGP, Pang, RCP), or while the system runs (Hermes, Adaptive)

What they optimise

balance in stored data, communication under load, or locality of labelled query patterns

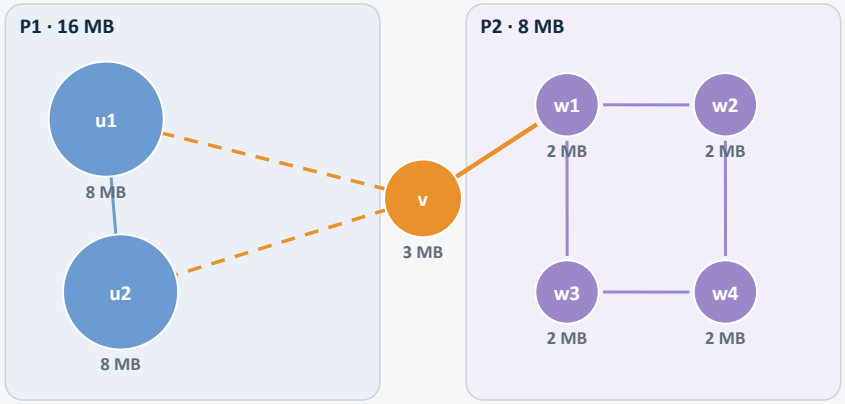
Takeaway only Pang et al. and RCP use semantics to place data; none uses explicit constraints.

* uses properties only through their storage size, not their meaning · Constraints "No": no explicit PG-Key, GFD or GGD-driven placement identified

Hammer: Volume Balance in an Example

Sakouhi et al., "Hammer Lightweight Graph Partitioner Based on Graph Data Volumes", Journal of Parallel and Distributed Computing (JPDC), 2021.

Placing v: more neighbours in P1, but P1 stores more



circle size = stored data · orange: v's edges; dashed = cut if v goes to P2

Reading the example

1 Each vertex and edge is measured by the data it stores, properties included.

2 v has more neighbours in P1, but P1 already stores twice as much.

3 Hammer places v in P2: one more cut edge, but 16 / 11 MB.

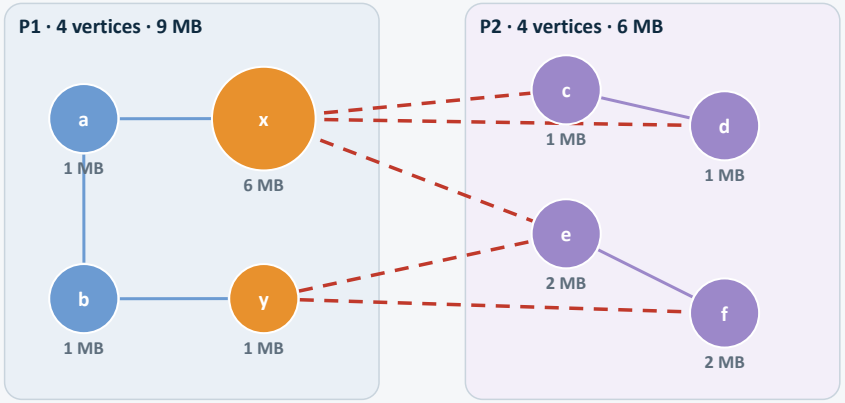
Place v in	v's neighbours	cut edges	volumes
P1 (by count)	2	1	19 / 8 MB
P2 (Hammer)	1	2	**16 / 11 MB**

Takeaway: Hammer prioritizes better storage balance, even when this means accepting a slightly higher edge cut in this example.

LSPGP: Two Balances in an Example

Cui et al., "Property Graph Partition Algorithm Based on Label Propagation and Sand Cat Swarm Optimization", Computer Engineering, 2025

Two candidate moves from P1 to P2



red dashed: cut edges · size bound 3–5 vertices, storage bound ≤ 10 MB

Reading the example

1 Label propagation gives an even split: 4 / 4 vertices.

2 Moving x removes the most cut edges, but P2 would store 12 MB.

3 Moving y removes one cut edge and keeps both balances: it is kept.

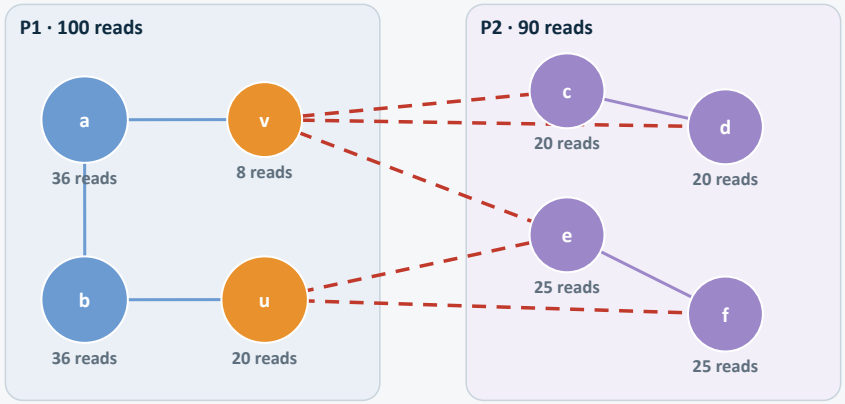
Move	cut change	size	storage	kept
x → P2	−2	3 / 5 ✓	3 / 12 MB ✗	no
y → P2	−1	3 / 5 ✓	8 / 7 MB ✓	yes

Takeaway: Properties enter through storage: a move must keep both size and stored data balanced.

Hermes: Read-Weighted Migration in an Example

Nicoara et al., "Hermes: Dynamic Partitioning for Distributed Social Network Graph Databases", EDBT 2015.

Stage 1, moves from P1 to P2: which vertices may move?



red dashed: cut edges · average load 95, allowed range 85.5–104.5

Reading the example

1 Gain = neighbours in P2 − neighbours in P1: v gains 2, u gains 1.

2 Moving v leaves 92 / 98 reads, inside the allowed range: v moves.

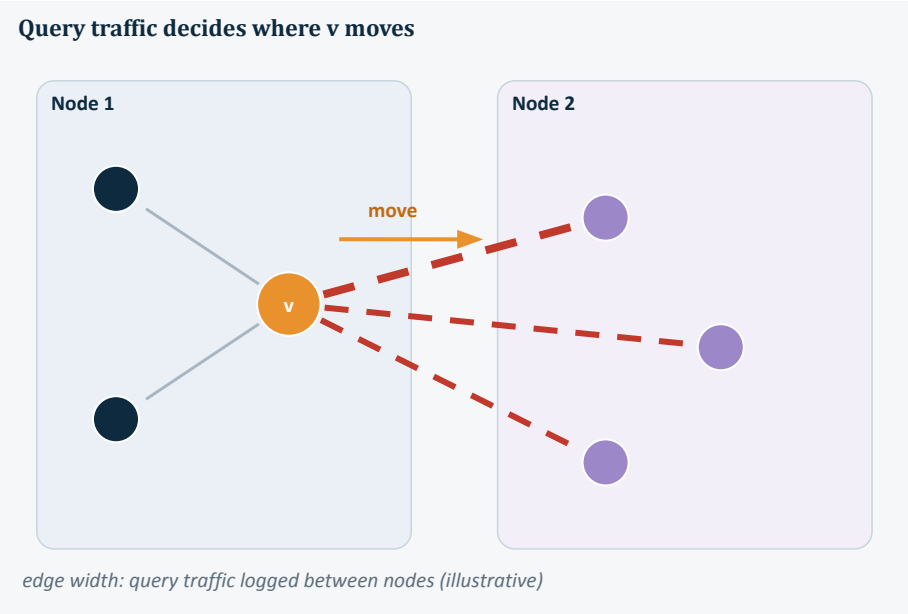
3 Moving u would leave 80 / 110 reads, outside the range: u stays.

Vertex	gain	reads	P1 / P2 after	moves
v	+2	8	92 / 98 ✓	yes
u	+1	20	80 / 110 ✗	no

Takeaway the workload sets the weights: a vertex follows its neighbours only if the read load stays balanced.

Adaptive Partitioning in a Property Graph Database

Svitáková, Valenta & Pokorný, "Enhanced Adaptive Partitioning in a Distributed Graph Database", J. Information and Telecommunication, 2020



How the partitioning happens

1 Log the network traffic that user queries generate (TinkerPop module).

2 Move vertices towards the node they communicate with most.

3 Balance the load, escape local optima, weight the edges by traffic.

TAKES INTO ACCOUNT

query traffic; load balance; edge weights

REPORTED

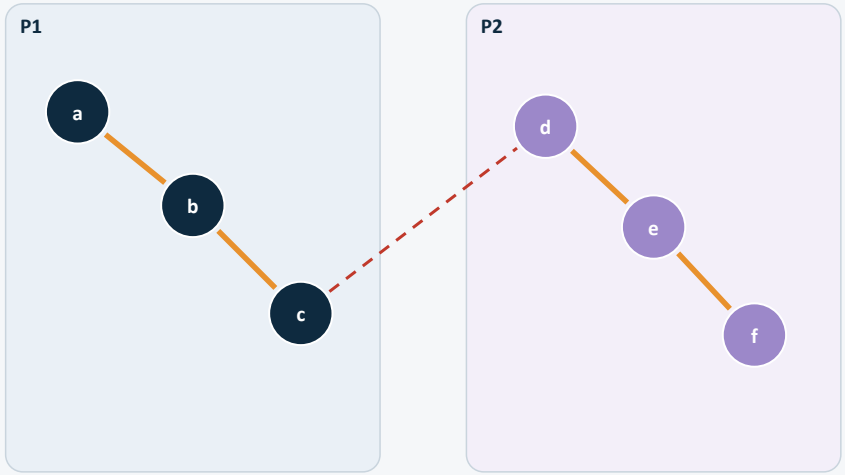
70–80% less communication between cluster nodes, comparable to Ja-be-Ja at lower cost.

Takeaway: The running workload, not the static structure, drives the partitioning.

Pang et al: Partitioning by Property Paths

Pang et al., "Partitioning Large-Scale Property Graph for Efficient Distributed Query Processing", IEEE HPCC/SmartCity/DSS 2019

A property path, decomposed and distributed



orange: sub-paths kept whole · red dashed: one break between sub-paths

How the partitioning happens

1 Find the semantic reachable property paths that queries follow.

2 Decompose long property paths into sub-paths.

3 Distribute the sub-paths across partitions, each kept whole.

TAKES INTO ACCOUNT

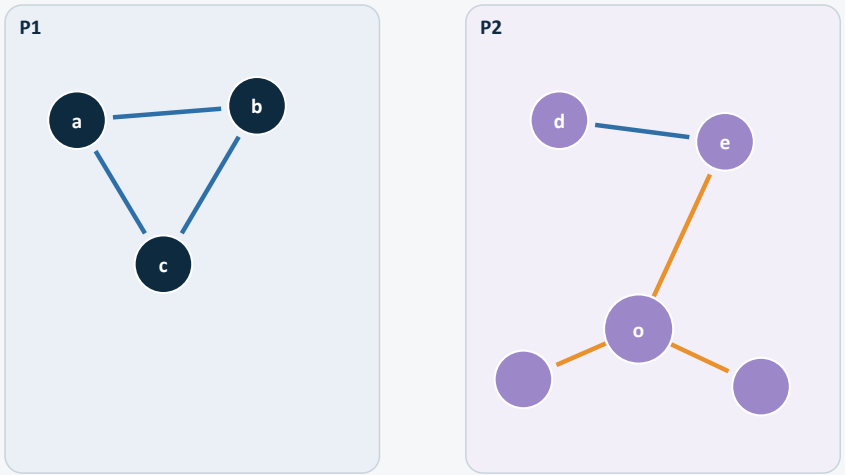
property paths; locality of path queries

Takeaway the unit of placement is a path that queries follow, not a single vertex.

RCP: Relationship-Type Connected Components

Shi et al., "Connectivity-Oriented Property Graph Partitioning for Distributed Graph Pattern Query Processing", SIGMOD 2024

Components per relationship type stay whole



blue: KNOWS, two components · orange: WORKS_AT, one component

How the partitioning happens

1 Split the edges by relationship type.

2 Compute the weakly connected components of each type.

3 Place every component whole in one partition.

TAKES INTO ACCOUNT

relationship types; connectivity; crossing query matches

REPORTED

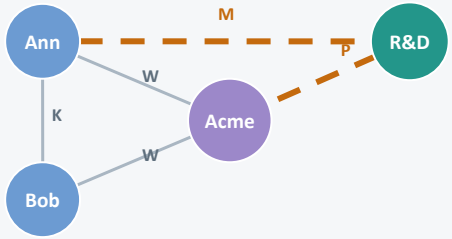
prototypes on Neo4j and JanusGraph; up to about 100× faster on billion-scale graphs.

Takeaway the relationship type decides what must stay together.

From Structural to Constraint-Aware Graph Partitioning

Each level adds information and changes what the partitioner keeps together

1 Structure



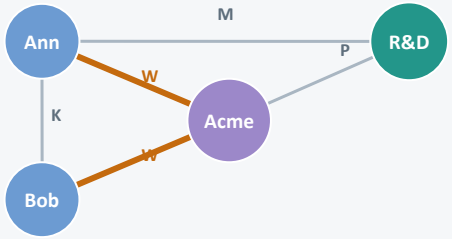
cut: 2 edges, chosen by count

SEES
vertices and edges

KEEPS TOGETHER
densely connected vertices

EXAMPLES
e.g., METIS, KaHIP, Fennel

2 Graph semantics



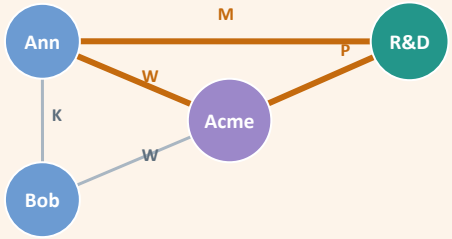
unit: all WORKS_AT matches

SEES
labels, relationship types, paths

KEEPS TOGETHER
all matches of a type or path

EXAMPLES
e.g., RCP, Pang et al.

3 Semantic constraints



unit: the edges the rule reads

SEES
rules the data must satisfy

KEEPS TOGETHER
every element a rule reads

EXAMPLES
not covered by current partitioners

K = KNOWS W = WORKS_AT M = MEMBER_OF P = PART_OF · rule: a person's department must be part of the company they work at

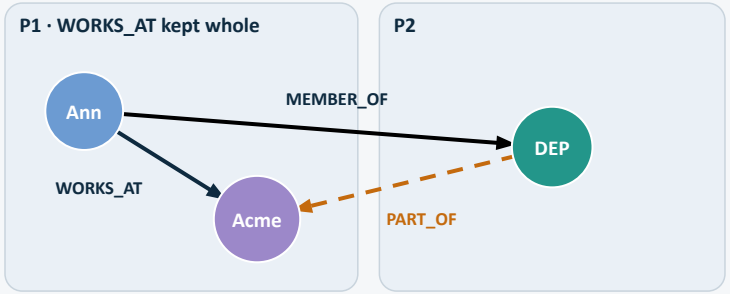
Takeaway structure keeps edges, semantics keeps meaning, constraints keep what must stay valid.

Beyond Graph Semantics: Understanding Formal Constraints

Conceptual comparison: why using labels and types does not guarantee constraint preservation

	Graph semantics	Semantic constraints
What it is	labels, relationship types, property paths	rules the data must satisfy: keys, cardinalities, dependencies
Example	:Person WORKS_AT a :Company	a person's department must be part of the company they work at
Scope	one label, one type or one path	several types, several hops, property values, or a whole label
Partitioning goal	the matches of a type or path	every element a rule reads

RULE: IF: `(p:Person)-[:MEMBER_OF]->(d:Department)`, `(p)-[:WORKS_AT]->(c:Company)` ⇒ THEN: `(d)-[:PART_OF]->(c)`



Types respected, rule split: two of the three edges the rule reads cross partitions.

1 Rules span several types

grouping by one relationship type can split a pattern that combines several

2 Rules read property values

labels and types say nothing about values such as codes, dates or keys

3 Some rules are global

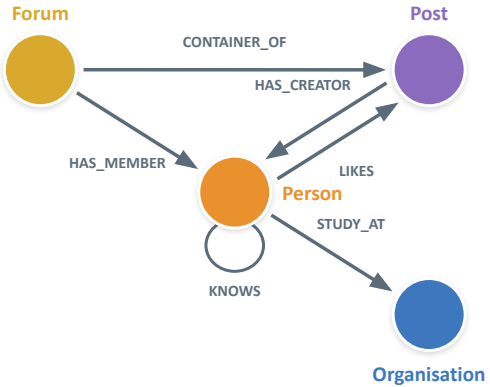
uniqueness over a label covers every vertex with that label, so every partition

Takeaway: Semantic-aware is not constraint-preserving: placement must follow the rules, not only the labels.

Semantic Constraints in Property Graphs: An Example

Shimomura, Fletcher & Yakovets, "GGDs: Graph Generating Dependencies", CIKM 2020 · data: LDBC Social Network Benchmark (Erling et al., SIGMOD 2015)

1 The LDBC SNB graph



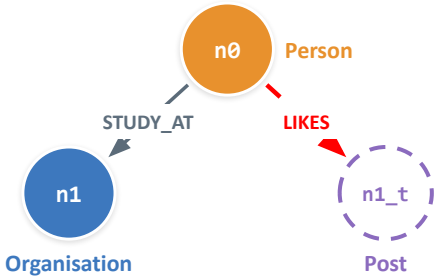
2 What does a GGD say?

$$Q_s[\bar{x}],\ \varphi_s \ \rightarrow\ Q_t[\bar{x}, \bar{y}],\ \varphi_t$$

IF the source pattern Q_s matches,
THEN the target pattern Q_t must also exist.
Variables ending in `_t` = new: "there exists some…"

3.a Infers a new node

NEW NODE + EDGE



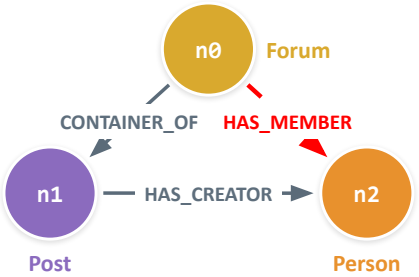
If **a person studies at an organisation**, they must **like some post** (possibly a new one).

```
(n0:Person)-[:STUDY_AT]->(n1:Organisation)

(n0)-[:LIKES]->(n1_t:Post)
```

3.b Infers a new edge

NEW EDGE ONLY



If **a person created a post in a forum**, they must **be a member of that forum**.

```
(n0:Forum)-[:CONTAINER_OF]->(n1:Post),
(n1)-[:HAS_CREATOR]->(n2:Person)

(n0)-[:HAS_MEMBER]->(n2)
```

IF: source pattern — THEN: target pattern (dashed circle = generated node)

Comparative Analysis of Partitioning Algorithms (1/3)

Categories of graph partitioning: classical, multilevel, and learning-based

	Algorithm	Cut model	Mode	Graph access	Workload	Labels / types	Optimises
1 Classical *spectral, local search, geometric*	**Spectral Bisection**	*edge-cut*	offline	whole graph	—	—	edge-cut (relaxed)
	Spectral Clustering	*edge-cut*	offline	whole graph	—	—	normalised cut (relaxed)
	Kernighan–Lin	*edge-cut*	refinement	graph + a partition	—	—	edge-cut, by pair swaps
	Fiduccia–Mattheyses	*edge-cut*	refinement	graph + a partition	—	—	cut size, by single moves
	RCB, RIB	*edge-cut*	offline	vertex coordinates	—	—	balance; locality by geometry
2 Multilevel *coarsen, partition, refine*	**METIS**	*edge-cut*	offline	whole graph	—	—	edge-cut or comm. volume
	SCOTCH	*edge-cut*	offline	whole graph	—	—	edge-cut; machine mapping
	KaHIP / KaFFPa	*edge-cut*	offline	whole graph	—	—	edge-cut (high quality)
	KaFFPaE	*edge-cut*	evolutionary	whole graph	—	—	edge-cut (best quality)
	KaMinPar	*edge-cut*	offline	whole graph	—	—	edge-cut, strict balance
	Huang et al.	*edge-cut + replicas*	offline	whole graph	—	—	local n-hop queries
3 Learning-based *neural model*	**GAP**	*edge-cut*	learned	whole graph	—	—	normalised cut, balance

■ property-graph algorithm ✓ uses directly (✓) partly * adapts at run time

Takeaway: Classical and multilevel methods primarily optimize structural objectives without explicitly considering property-graph semantics.

Comparative Analysis of Partitioning Algorithms (2/3)

Categories of graph partitioning: scalable, distributed and dynamic

	Algorithm	Cut model	Mode	Graph access	Workload	Labels / types	Optimises
4 Hash-based *place by a hash*	**Hash on vertex ID**	*edge-cut*	hashed	vertex ID	—	—	load balance only
	Random vertex-cut	*vertex-cut*	hashed	edge endpoints	—	—	edge balance
	Grid	*vertex-cut*	hashed	edge endpoints	—	—	bounded replication
	DBH	*vertex-cut*	hashed	endpoints + degree	—	—	replication factor
	PowerLyra	*hybrid-cut*	hashed	endpoints + degree	—	—	replication, balance
	SHAPE	*triple groups*	hashed	triples per vertex	—	—	local queries, balance
5 Streaming *one pass, decide once*	**LDG**	*edge-cut*	streaming	one pass	—	—	edge-cut, balance
	Fennel	*edge-cut*	streaming	one pass	—	—	edge-cut + size cost
	HDRF	*vertex-cut*	streaming	one pass + degrees	—	—	replication factor
6 Neighborhood-based Partitioning *swaps, label propagation*	**Ja-be-Ja**	*edge-cut*	iterative	neighbourhood	—	—	edge-cut (by swaps)
	Spinner	*edge-cut*	iterative	neighbourhood	—	—	edge-cut, balance
	DHPV	*edge-cut*	distributed	hubs + neighbours	—	—	edge-cut, balance
7 Dynamic *repartition on change*	**Adaptive Partitioning***	*edge-cut*	incremental	neighbourhood	—	—	edge-cut under change
	Leopard*	*edge-cut + replicas*	incremental	neighbourhood	—	—	effective edge-cut

■ property-graph algorithm ✓ uses directly (✓) partly * adapts at run time

Takeaway: Scalable partitioners use hashing, streaming, or local decisions to handle large graphs efficiently.

Comparative Analysis of Partitioning Algorithms (3/3)

Categories of graph partitioning: workload-, property- and semantic-aware

	Algorithm	Cut model	Mode	Graph access	Workload	Labels / types	Optimises
8 Workload-aware *follow the queries*	**TAPER***	*edge-cut*	incremental	neighbours + queries	✓	(✓)	inter-partition traversals
	Loom*	*edge-cut*	streaming	one pass + motifs	✓	(✓)	inter-partition traversals
	WARP	*edge-cut + replicas*	offline	graph + query log	✓	(✓)	local query patterns
	Partout	*query fragments*	offline	query log	✓	(✓)	local query fragments
	AdPart*	*hash, then moved*	incremental	hash + queries	✓	(✓)	communication
	Hermes*	*edge-cut*	incremental	neighbours + reads	✓	—	edge-cut, load balance
	Adaptive (Svitáková)*	*edge-cut*	incremental	neighbours + traffic	✓	—	inter-node communication
9 Property-aware *balance stored data*	**Hammer**	*edge-cut*	streaming	one pass + volumes	—	(✓)	volume balance, cut edges
	LSPGP	*edge-cut*	iterative	neighbours + storage	—	(✓)	edge-cut; storage balance
10 Semantic-aware *meaning decides placement*	**Xu et al.**	*class-based*	offline	class graph	—	✓	fewer crossing edges
	MPC	*property-cut*	offline	graph, by property	—	✓	fewest crossing properties
	DIAERESIS	*class-based*	offline	schema summary	—	✓	data accessed per query
	SemPart	*fragment-based*	offline	characteristic sets	—	✓	semantic fragments
	Pang et al.	*path-based*	offline	property paths	—	✓	locality of path queries
	RCP	*connectivity*	offline	relationship types	—	✓	crossing query matches

property-graph algorithm ✓ uses directly (✓) partly * adapts at run time

Takeaway: Only semantic-aware methods let labels, types or classes decide placement; none uses explicit constraints.

Partitioning in Property Graph Database Systems

How deployed graph databases distribute and store data today

System	How data is placed	User control	Uses meaning?
JanusGraph	random placement by default	explicit partitioning, optional	no
NebulaGraph	hash of the vertex ID	choice of partition number	no
ArangoDB	hash of the document key	SmartGraphs: shard by an attribute	partly
Neo4j composite	manual sharding	user defines the shards	by hand
Neo4j Infinigraph	topology together, properties sharded	managed by the system	partly
Amazon Neptune	managed graph storage	no explicit sharding	no
Memgraph	no native sharding	replication configuration	no

Takeaway: Existing data-placement strategies do not generally guarantee semantic-constraint preservation.

04

Evaluating Partitions

How do we measure whether a partition respects meaning?

Metrics, benchmarks and datasets.

IN THIS PART

01 Evaluation Metrics for Semantic-Aware Partitioning

02 Property Graph Benchmarks and Datasets

03 Semantic-Aware Partitioning: Key Takeaways

Evaluation Metrics for Semantic-Aware Partitioning

Structural metrics alone cannot capture the semantic quality of graph partitions

	Metric	What it measures
Structural Quality	**Edge-cut ratio**	Fraction of edges crossing partition boundaries
	Partition balance	How evenly vertices, edges or computational load are distributed
	Replication factor	Average number of partitions in which each vertex is stored
Semantic Preservation *what distinguishes semantic-aware methods*	**Constraint locality**	Fraction of constraint matches that are fully contained within a single partition
	Semantic group integrity	Fraction of semantic groups kept intact within partitions.
Query Performance	**Query execution time**	Time to execute property graph queries (e.g. Cypher, GQL)
	Cross-partition communication	Network messages or bytes transferred during query processing
Reasoning and Validation	**Reasoning execution time**	Time required for rule evaluation, e.g. GGD-style generation or repair
	Distributed reasoning overhead	Communication and synchronisation introduced by distributed reasoning
Scalability and Efficiency	**Partitioning runtime**	Time required to construct partitions
	Scalability	Behaviour as graph size, partition count or machine count increases

Takeaway: Select evaluation metrics according to the partitioning approach: label-, workload-, schema-, or constraint-aware

Property Graph Benchmarks & Datasets

Evaluating property-graph partitioning requires graph data, representative workloads, and, for constraint-aware methods, explicit semantic dependencies.

	Name	Graph model	Evaluation focus
Property Graph & Related Benchmarks *data, workload and procedure defined*	**LDBC SNB**	Property graph	Social network queries and analytics
	LDBC FinBench	Property graph	Financial transactions, temporal multi-hop
	LSQB	Typed/labeled graph	Subgraph pattern matching
	gMark	Schema-driven generation	Synthetic graphs and query workloads
	LDBC Graphalytics	General graph	BFS, PageRank, WCC, CDLP, LCC, SSSP
Real-world & Heterogeneous Datasets *representation and workload set by each paper*	**GDELT**	Heterogeneous graph*	Events, actors, locations
	CORDIS	Heterogeneous graph*	Research projects and organisations
	DBLP	Bibliographic graph*	Publications, authors, venues
	ogbn-mag (OGB)	Heterogeneous graph (MAG)	Papers, authors, institutions, fields of study
	MusicBrainz	Property graph	Music metadata with many labels

RDF, not native property graphs LUBM (ontology reasoning) · WatDiv (SPARQL workloads) · YAGO 4 (knowledge graph with SHACL constraints)

Semantic-Aware Partitioning: Key Takeaways

1 **Structure is not enough**

Classical partitioners optimise cuts and balance; labels, properties and dependencies are invisible to them.

2 **Property graphs carry rich semantics**

Labels, relationship types, properties, schema and dependencies define units of meaning.

3 **Existing approaches exploit semantics selectively**

Workload, property paths, relationship types or data volume guide placement; dependencies are not yet used.

5 **Evaluate meaning, not only cuts**

Semantic preservation and reasoning performance should complement edge-cut and balance metrics.

The open question : How can semantic constraints guide the partitioning of property graphs?

BLOCK 3.1

Property Graph Dependencies

Property graph keys, schemas, graph dependencies, and integrity constraints

Why Graph Dependencies Matter ?

Formal dependencies guide data grouping, constraint validation, and reasoning

What are graph dependencies?

Rules over a property graph: a pattern selects a group of nodes and edges, and a condition must hold on every match.

`Q` pattern match : `X` condition ⇒ `Y` must hold

PG-Keys
same entity?
PG-Keys, graph keys

GFDs
values agree?
Fan, Wu & Xu, 2016

GGDs
facts implied?
Shimomura et al., 2020

Why they matter

Data quality
They detect inconsistent values and missing facts.

Identity
Keys tell when two nodes denote the same entity.

Inference
GGDs add the facts the data implies: the closure.

Downstream tasks
Entity resolution, validation and integrity checks build on them.

Why semantic partitioning needs them

They define what belongs together
Each match names the nodes that must be read together.

They are known in advance
Declared rules, available before any workload is seen.

Ignoring them is costly
Split matches turn every check and inference into messages.

They give a measurable goal
Matches kept local, and violations caused by splitting.

Graph Dependencies at a Glance

What each dependency ranges over, what it constrains, and an example

Dependency	Scope	Constraints	New data	Example
gUC	Labelled nodes	Uniqueness of property values	No	No two Person nodes share an email
gFD	Labelled nodes	Property value dependencies	No	Persons with the same ip have the same city
PG-Key	Query-defined graph objects	Identity of graph objects (i.e., nodes, edges, and property values)	No	Each Person is identified by its email
GFD	Pattern matches	Equality of attribute values	No	Two persons in the same city agree on country
GGD	Pattern matches	Existence of nodes and edges	Yes	Two persons at the same Org know each other

Node-level (**gUC, gFD, PG-Key**), **Pattern-level** (**GFD, GGD**)

Only GGDs generate new structure

Graph Dependencies at a Glance

What each dependency ranges over, what it constrains, and an example

Dependency	Scope	Constraints	New data	Example
gUC	Labelled nodes	Uniqueness of property values	No	No two Person nodes share an email
gFD	Labelled nodes	Property value dependencies	No	Persons with the same ip have the same city
PG-Key	Query-defined graph objects	Identity of graph objects (i.e., nodes, edges, and property values)	No	Each Person is identified by its email
GFD	Pattern matches	Equality of attribute values	No	Two persons in the same city agree on country
GGD	Pattern matches	Existence of nodes and edges	Yes	Two persons at the same Org know each other

Node-level (**gUC, gFD, PG-Key**), **Pattern-level** (**GFD, GGD**)

Only GGDs generate new structure

gUC: Graph-tailored Uniqueness Constraint

gUC: Skavantzos et al., "Uniqueness Constraints on Property Graphs", CAiSE 2021.

$$L : P : X$$

where

L	a set of labels (e.g., `Person`)
P	the available properties (e.g., `id`, `email`, `firstName`)
$X \subseteq P$	the property subset required to be unique (e.g., `{email}`)

The (L, X) projection identifies at most one element in the graph.

gUC: Graph-tailored Uniqueness Constraint — Example

gUC: Skavantzos et al., "Uniqueness Constraints on Property Graphs", CAiSE 2021.

OBJECTIVE

- Guarantees unique node identification
- Eliminates duplicate graph mappings
- Maintains semantic integrity

$$(p_1{:}\text{Person}) \wedge (p_2{:}\text{Person}) \wedge p_1.\text{id} = p_2.\text{id} \Rightarrow p_1 = p_2$$

→ *Two distinct Person nodes cannot share the same ID*



gFD: Graph-tailored Functional Dependency

gFD: Skavantzos and Link, “Normalizing Property Graphs”, PVLDB 2023.

$$L : P : X \rightarrow Y$$

where

L	a set of labels
P	the properties under consideration
$X, Y \subseteq P$	determinant and dependent property subsets

Same X values imply same Y values for property-graph elements.

gFD: Graph-Tailored Functional Dependency — Example

gFD: Skavantzos and Link, "Normalizing Property Graphs", PVLDB 2023.

OBJECTIVE

- Enforces attribute consistency
- Captures graph dependencies
- Validates graph integrity

$$(u_1{:}\text{University}) \wedge (u_2{:}\text{University}) \wedge u_1.\text{name} = u_2.\text{name} \Rightarrow u_1.\text{city} = u_2.\text{city}$$

→ *Universities with the same name resolve to the same city* $\Rightarrow x = y$.



PG-Keys: Keys for Property Graphs

PG-Keys: Keys for Property Graphs paper (SIGMOD 2021)

Declarative key specification

```
FOR x WITHIN
IDENTIFIER y, z WITHIN
```

Example — "Each person is identified by their login":

```
FOR p WITHIN (p:Person) IDENTIFIER p.login;
```

PG-Keys are flexible because they can apply to nodes, edges, and property values, helping maintain data integrity.

01 **Flexible scope & key descriptors**

Customizable choices for the scope and the descriptor values to pinpoint constraint context.

02 **Universal entity coverage**

One uniform constraint model across nodes, edges and rich property collections.

03 **Identification & referencing**

Precise identification, cross-element referencing and semantic integrity on PG objects.

04 **Low-complexity validation**

Designed for direct, computationally efficient verification over large-scale property graphs.

PG-Keys: Keys for Property Graphs

PG-Keys: Keys for Property Graphs — R. Angles et al., SIGMOD 2021

Declarative key specification — Target Scope — Descriptor Values

Q1 Each **person** is identified by **their login**

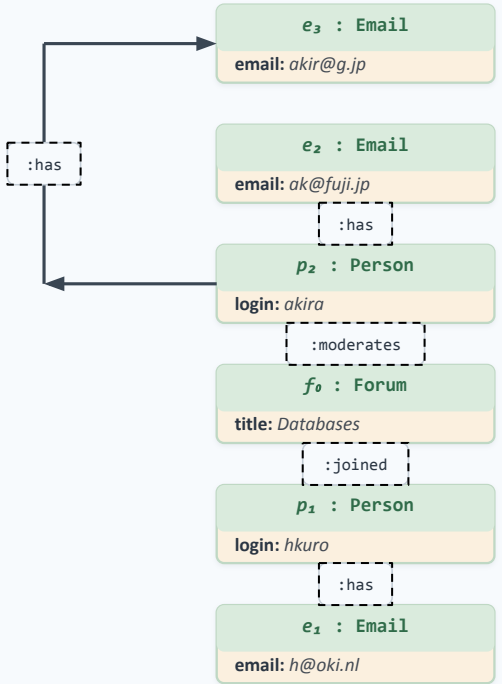
```
FOR p WITHIN (p:Person)  IDENTIFIER  p.login
```

Q2 Each **forum with a member** is identified by **its name & moderator**

```
FOR f WITHIN (f:Forum)<-[:joined]-(:Person)
IDENTIFIER f.title, p WITHIN (f)<-[:moderates]-(p:Person)
```

Declaring key scopes and property identification in a PG query language

PROPERTY GRAPH SAMPLE INSTANCE



PG-Keys: Checking a Constraint

PG-Keys: Keys for Property Graphs — R. Angles et al., SIGMOD 2021

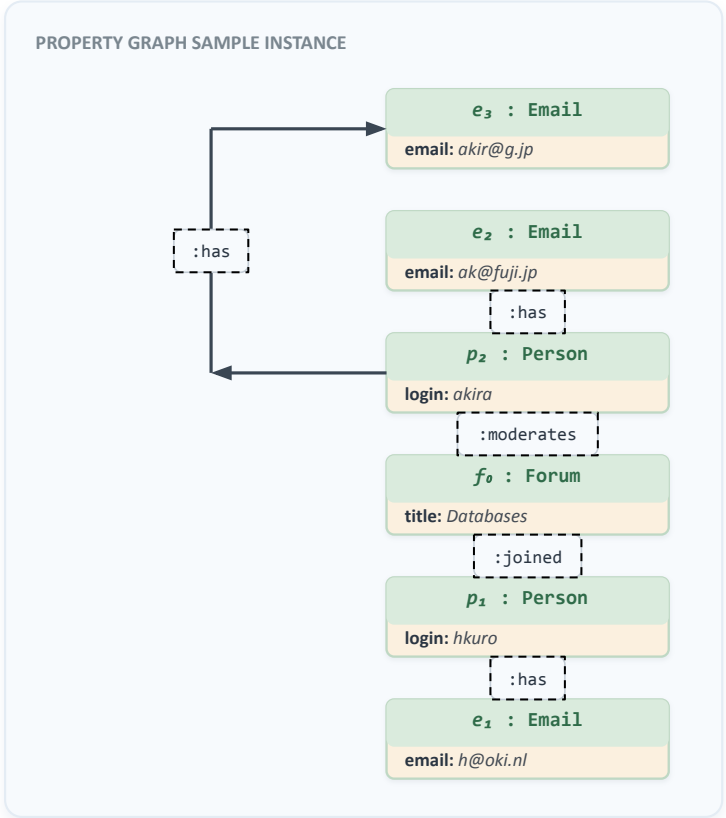
Constraint: *Each person has exactly one email (exclusive & mandatory)*

```
FOR p WITHIN (p:Person)
EXCLUSIVE MANDATORY e WITHIN (p)-[:has]->(e:Email);
```

Holds iff both violation queries return nothing:

```
MATCH (p1:Person)-[:has]->(:Email)<-[:has]-(p2:Person)
WHERE p1 <> p2 RETURN p1, p2;
```

```
MATCH (p:Person)
WHERE NOT EXISTS ((p)-[:has]->(:Email))
RETURN p;
```



PG-Keys: Identifier Semantics

PG-Keys: Keys for Property Graphs — R. Angles et al., SIGMOD 2021

EXCLUSIVE e.g., Two different persons cannot have the same email.

→ SQL UNIQUE

MANDATORY e.g., Every person must have at least one email.

→ SQL NOT NULL

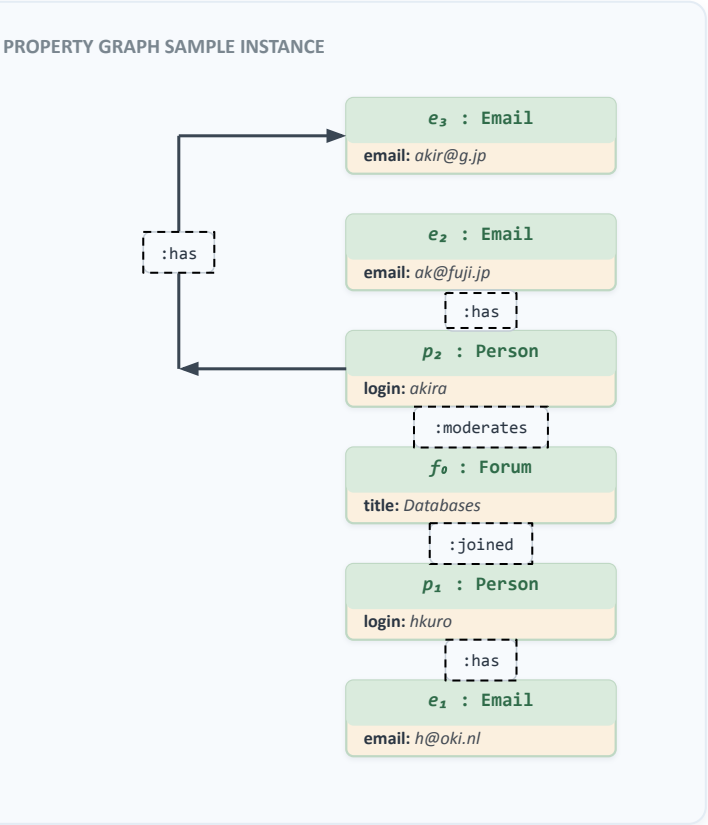
SINGLETON e.g., Each person can have at most one email.

→ ensured by 1NF

IDENTIFIER e.g., Each person has exactly one email, which no other person shares

→ PRIMARY KEY

```
FOR f WITHIN (f:Forum)<-[:joined]-(:Person)
IDENTIFIER f.name, p WITHIN (f)<-[:moderates]-(p:Person)
```



PG-Keys: Property Graph Keys — Example

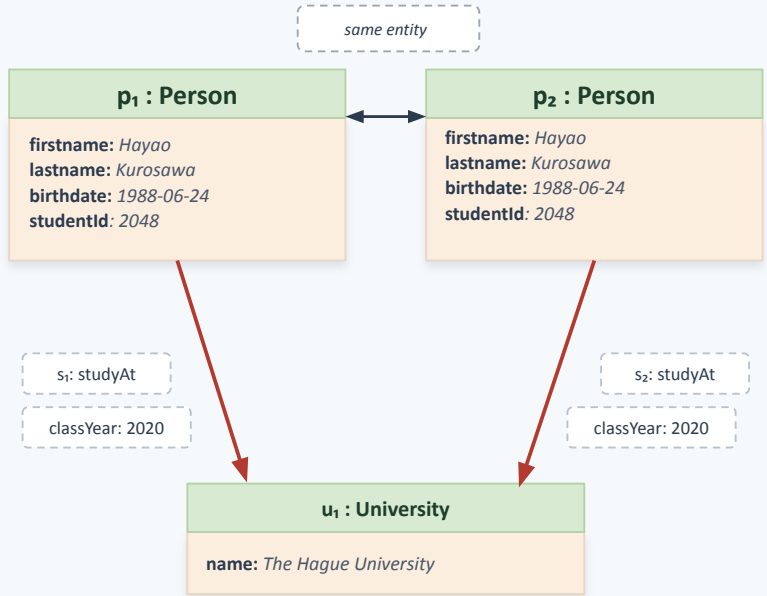
PG-Keys: Keys for Property Graphs — R. Angles et al., SIGMOD 2021

OBJECTIVE

- Uniquely identifies real-world entities
- Combines properties with graph topology
- Detects duplicates & enforces referential integrity

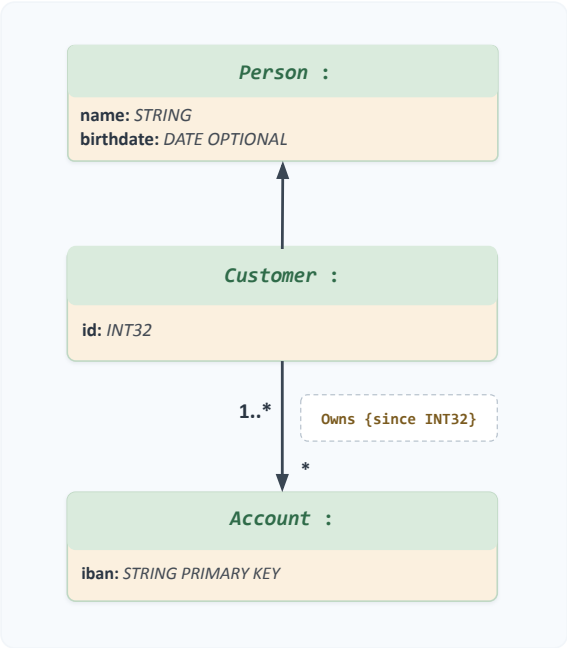
```
(p₁:Person {studentId:s})-[:studyAt]→(u:University)
∧
(p₂:Person {studentId:s})-[:studyAt]→(u:University)
⇒ p₁ = p₂
```

→ *Two Person nodes with the same student ID at the same university must represent the same person.*



PG-Schemas: Graph Database Schemas

PG-Schema — R. Angles et al., SIGMOD 2023




```
CREATE GRAPH TYPE customerGraph STRICT {
  (person: Person {name STRING, OPTIONAL birthdate DATE}),
  (company: Company {name STRING}),
  ABSTRACT (taxpayer: {taxPayerNumber STRING}),
  (customer: (person|company) & taxpayer & Customer {id INT32, OPEN}),
  (account: Account {iban STRING}),
  (:customer)-[:owns: Owns {since DATE}]->(:account),
  FOR (a:account) KEY a.iban,
  FOR (a:account) MANDATORY (:customer)-[:owns]->(a)
}
```

GFD: Graph Functional Dependency

GFD: Fan et al., "Functional Dependencies for Graphs", SIGMOD 2016.

$$\varphi = (Q[x] , X \rightarrow Y)$$

where

$Q[x]$	a graph pattern with variables x
X, Y	sets of attributes defined over variables in Q

If two matches of pattern Q agree on X, they must also agree on Y.

GFD: Graph Functional Dependency — Example

GFD: Fan et al., "Functional Dependencies for Graphs", SIGMOD 2016.

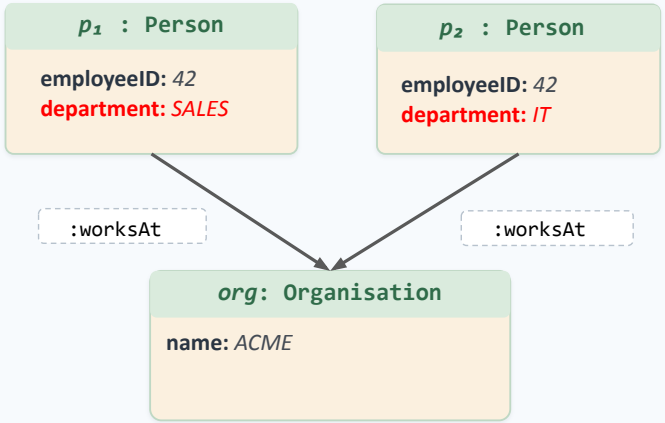
OBJECTIVE

- Enforces unique graph structures
- Identifies duplicate entities
- Improves data quality

$$(p_1{:}\text{Person})\text{-}[:\text{WORKS_AT}]\rightarrow(c{:}\text{Company}) \wedge (p_2{:}\text{Person})\text{-}[:\text{WORKS_AT}]\rightarrow(c{:}\text{Company}) \wedge$$

$$p_1.\text{employeeId} = p_2.\text{employeeID} \Rightarrow p_1.\text{department} = p_2.\text{department}$$

→ *Employees working at the same company with the same employee ID must belong to the same department.*



GGD: Graph Generating Dependency

GGD: Shimomura et al., “GGDs: Graph Generating Dependencies”, CIKM 2020

$$Q□[x] , φ□ \rightarrow Q□[x,y] , φ□$$

where

$Q□[x]$	source graph pattern (variables x)
$Q□[x,y]$	target pattern (extends source with new variables y)
$φ□ , φ□$	source / target constraints (conditions over x, x ∪ y)

Matching the source forces the target to exist or to be created.

GGD: Graph Generating Dependency — Example

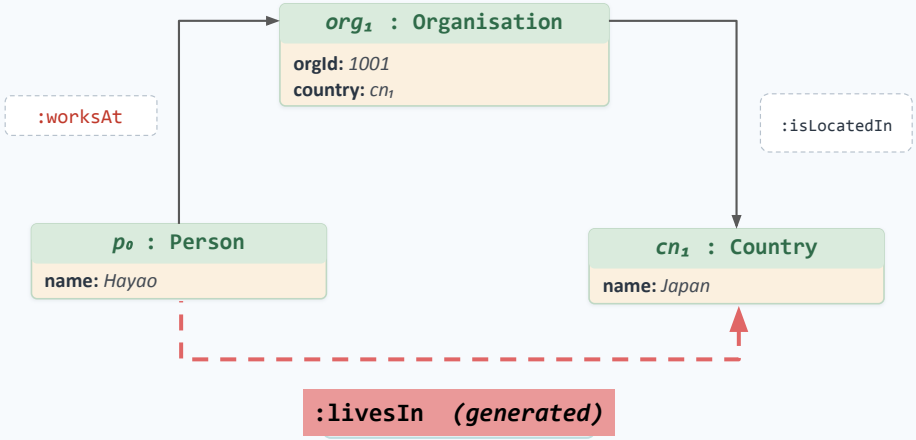
GGD: Shimomura et al., "GGDs: Graph Generating Dependencies", CIKM 2020

OBJECTIVE

- Infers new graph facts
- Captures semantic dependencies
- Enables graph reasoning

$(p_0{:}\text{Person})\text{-}[:\text{worksAt}]\rightarrow(org_1{:}\text{Organisation})\text{-}[:\text{isLocatedIn}]\rightarrow(cn_1{:}\text{Country}) \Rightarrow (p_0{:}\text{Person})\text{-}[:\text{livesIn}]\rightarrow(cn_1{:}\text{Country})$

→ *Working in a country's organisation generates a lives-in edge.*



PART 4

Hands-on Demonstration & Open Challenges

Demonstration, Hands-On Session, and Open Challenges

capPG: Constraint-Aware Partitioning for Property Graphs

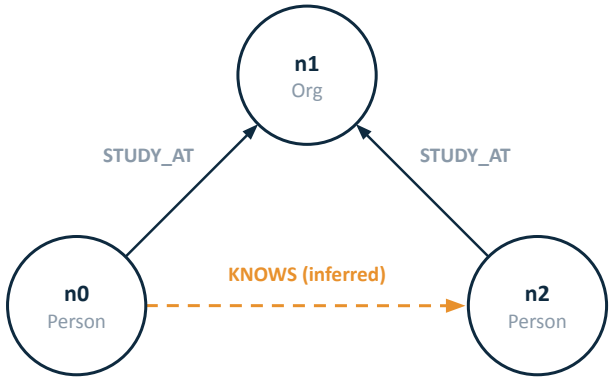
E. Ymeralli, S. Dumbrava, D. Plexousakis, H. Kondylakis · FORTH-ICS & CSD-UOC · ENSIIE & SAMOVAR, Télécom SudParis

Motivation	Traditional graph partitioners may separate semantically related nodes, increasing communication costs during reasoning.
Approach	capPG uses graph dependencies to identify related nodes, form semantic groups, and keep them together in the same partition.
Method	Semantic groups are coarsened into supernodes, partitioned using KaHIP, and refined during uncoarsening.

In the demo

We evaluate three datasets throughout the capPG workflow: its constraints, the semantic groups they induce, the resulting partitions, and reasoning over those partitions.

A constraint kept local by capPG

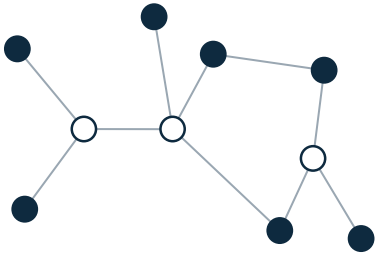


GGD: *Two distinct persons who study at the same university must know each other.*

If n0 and n2 sit in different partitions, inferring KNOWS needs both machines.

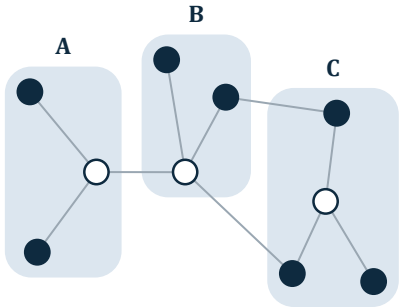
From Constraints to Semantic Partitioning: The capPG Workflow

1 Data ingestion



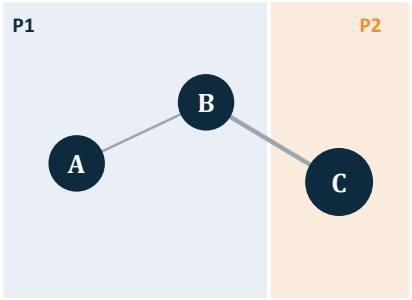
Nodes, edges and properties loaded into Neo4j; GGDs parsed.

2 Semantic groups



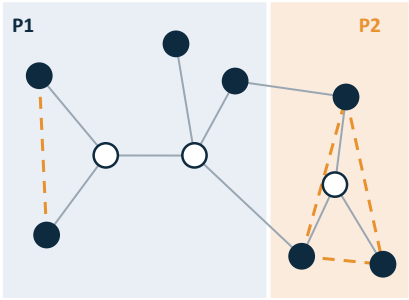
Constraint patterns matched; overlapping matches merged into groups.

3 Multi-level partitioning



Groups become supernodes, KaHIP splits the super-graph, uncoarsening refines.

4 Constraint-driven reasoning



Closure computed inside each partition; violations reported.

In the demo

1 Load LDBC, CORDIS or GDELT and explore its constraints

2 Apply selected GGDs and observe group formation and merging

3 Choose k, run the pipeline and inspect the partitions

4 Reason on the partitions and compare with METIS and KaHIP

Demo setup: Datasets, Baselines and Metrics

Datasets

Dataset	Domain	Source
LDBC SNB	Social network benchmark	Angles et al., 2020
CORDIS	EU research projects and organisations	European Commission
GDELT	Global news events, actors and locations	GDELT Project

Algorithm

- KaHIP (Sanders & Schulz)
- Multilevel graph partitioning
- Run on the semantic supergraph

Baselines

- METIS (Karypis & Kumar)
- KaHIP (Sanders & Schulz)
- Run on the original dataset

Metrics

- Reasoning time, for k = 2,4,6,8,10
- Constraint violations across partitions
- Constraint preservation, edge-cut, balance

Constraints graph dependencies (i.e., GGDs) **Environment** 4-node Spark cluster (Spark 3.0.0), Scala 2.12, and Neo4j 4.4

Demo isl.ics.forth.gr/capPG

PART P4

Open Challenges

1. **Joint Optimization**
   Unify structural, semantic & dependency objectives in one framework
2. **Dynamic Graphs**
   Keep partitions coherent as the graph changes
3. **Scalable Partitioners**
   Balancing massive graphs at ultra scale
4. **Distributed Reasoning and Validation**
   Scale dependency checking across partitions
5. **Standard Benchmarks**
   Metrics & datasets for constraint-aware partitioning
6. **Replication vs. Locality**
   Buy locality with the smallest replication budget

PART P4

Open Challenges

1. **Joint Optimization**
   Unify structural, semantic & dependency objectives in one framework

2. **Dynamic Graphs**
   Keep partitions coherent as the graph changes

3. **Scalable Partitioners**
   Balancing massive graphs at ultra scale

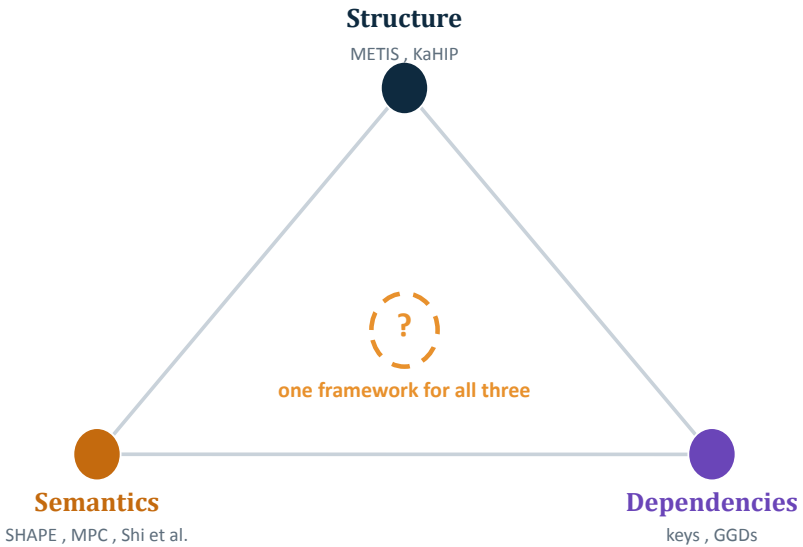
4. **Distributed Reasoning and Validation**
   Scale dependency checking across partitions

5. **Standard Benchmarks**
   Metrics & datasets for constraint-aware partitioning

6. **Replication vs. Locality**
   Buy locality with the smallest replication budget

Joint Optimization

Can one partitioner optimise structure, semantics and dependencies at the same time?



Why it is hard

- The objectives pull in different directions: the cheapest cut may split a semantic unit or a constraint scope.
- They are measured in different units (cut edges, split units, split scopes), so there is no natural way to add them up.
- The right balance depends on the workload, and the workload changes.
- Each extra objective makes an already NP-hard search larger and slower.

Directions

- Multi-objective formulations: weighted sums or Pareto fronts.
- Coarsen by semantic units first, then refine for structure (multilevel).
- Report all objectives side by side, not edge-cut alone.

Research Idea: Let the schema define the units and a structural partitioner balance them: semantics sets the units, structure sets the cut.

PART P4

Open Challenges

1. **Joint Optimization**
   Unify structural, semantic & dependency objectives in one framework

2. **Dynamic Graphs**
   Keep partitions coherent as the graph changes

3. **Scalable Partitioners**
   Balancing massive graphs at ultra scale

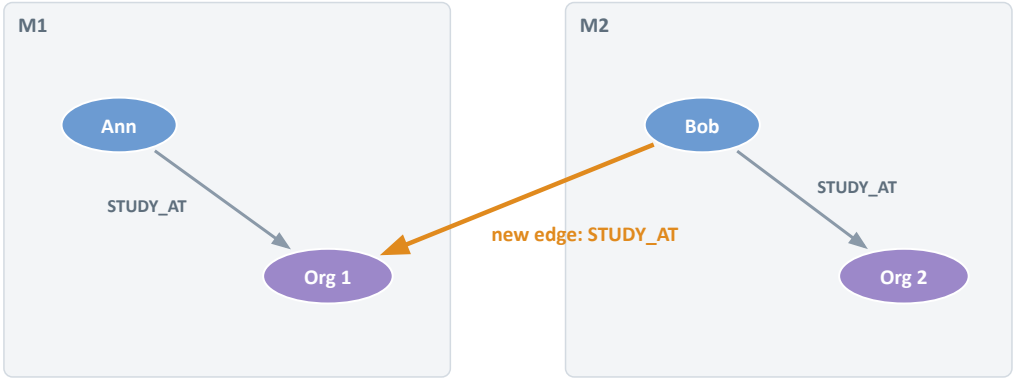
4. **Distributed Reasoning and Validation**
   Scale dependency checking across partitions

5. **Standard Benchmarks**
   Metrics & datasets for constraint-aware partitioning

6. **Replication vs. Locality**
   Buy locality with the smallest replication budget

Dynamic Graphs

How do we keep partitions semantically coherent as the graph changes?



An update adds Bob STUDY_AT Org 1, a new edge between M2 and M1.

One edge breaks the locality the partition was built for.

Why it is hard

- A single new edge can link data that sits on different machines.
- Updates arrive continuously; re-partitioning from scratch after each one is too costly.
- Moving data to restore locality has its own cost and can upset balance.
- Streaming partitioners (LDG, Fennel, HDRF) place new elements by structure alone.

Directions

- Update semantic units incrementally as nodes and edges are added and removed.
- Adaptive re-partitioning, as in Hermes and Leopard, guided by semantics.
- Decide when to move data and when to replicate it.

Research idea: Move only the semantic unit that an update splits, not the whole graph.

PART P4

Open Challenges

1. **Joint Optimization**
   Unify structural, semantic & dependency objectives in one framework
2. **Dynamic Graphs**
   Keep partitions coherent as the graph changes
3. **Scalable Partitioners**
   Balancing massive graphs at ultra scale
4. **Distributed Reasoning and Validation**
   Scale dependency checking across partitions
5. **Standard Benchmarks**
   Metrics & datasets for constraint-aware partitioning
6. **Replication vs. Locality**
   Buy locality with the smallest replication budget

Scalable Partitioners

Can semantic-aware partitioning run at the scale of today's graphs?



Why it is hard

- Semantic methods read more input: labels, properties, schemas and query logs.
- The best-quality methods need the whole graph in memory; one-pass streaming methods lose quality.
- Graphs keep growing and changing, so partitioning is never done only once.
- Few semantic-aware methods have been evaluated on very large graphs.

Directions

- Streaming and buffered partitioners that read labels and types.
- Multilevel partitioning that coarsens by semantic units.
- Distribute the partitioner itself.

Research Idea: Coarsen by labels or schema units before multilevel partitioning, so semantics also makes the graph smaller and faster to split.

PART P4

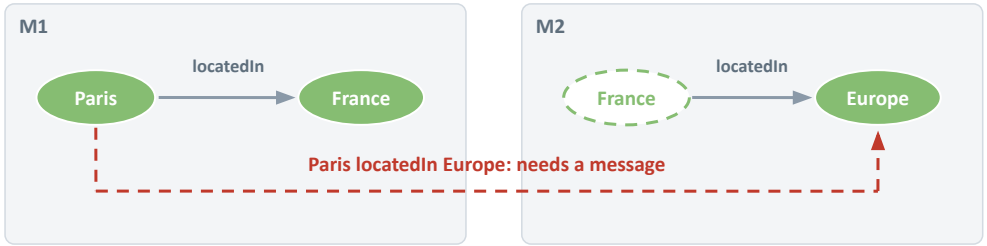
Open Challenges

1. **Joint Optimization**
   Unify structural, semantic & dependency objectives in one framework
2. **Dynamic Graphs**
   Keep partitions coherent as the graph changes
3. **Scalable Partitioners**
   Balancing massive graphs at ultra scale
4. **Distributed Reasoning and Validation**
   Scale dependency checking across partitions
5. **Standard Benchmarks**
   Metrics & datasets for constraint-aware partitioning
6. **Replication vs. Locality**
   Buy locality with the smallest replication budget

Distributed Reasoning and Validation

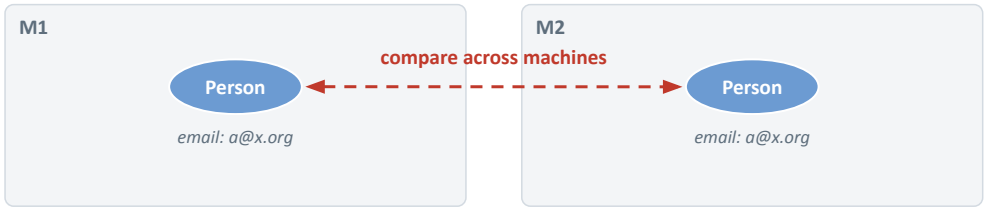
How do we derive new facts and check constraints when the data they need is split across machines?

Reasoning: derive new facts



Validation: check constraints

Key: two :Person with the same email are the same person



Why it is hard

- Recursive rules, such as transitivity, need several rounds of messages until no new fact appears.
- A constraint must see every vertex it matches, wherever that vertex is stored.
- Reasoning and validation need different things kept together, but run on the same partition.
- Updates can create new facts and new violations anywhere in the graph.

Directions

- Distributed materialisation with message exchange between servers (Ajileye & Motik 2022).
- Parallel validation of graph dependencies (Fan, Wu & Xu, SIGMOD 2016).
- Partition so that rule premises and constraint scopes stay local.

Research Idea: Design one placement that keeps both rule premises and constraint scopes local, and measure messages for both tasks together.

PART P4

Open Challenges

1. **Joint Optimization**
   Unify structural, semantic & dependency objectives in one framework

2. **Dynamic Graphs**
   Keep partitions coherent as the graph changes

3. **Scalable Partitioners**
   Balancing massive graphs at ultra scale

4. **Distributed Reasoning and Validation**
   Scale dependency checking across partitions

5. **Standard Benchmarks**
   Metrics & datasets for constraint-aware partitioning

6. **Replication vs. Locality**
   Buy locality with the smallest replication budget

Standard Benchmarks

What would a benchmark for semantic-aware partitioning need to provide?

A benchmark needs

Labelled data	**available**	LDBC SNB · LSQB · WatDiv
Query workload	**available**	LDBC SNB · LSQB · WatDiv
Constraints and rules	**missing**	not provided for partitioning
Partition-quality metrics	**missing**	no agreed measure of semantic locality

Why it is hard

- Each paper uses its own graphs, workloads and metrics, so results cannot be compared.
- Semantic locality has no agreed measure.
- Existing benchmarks come with queries, but not with rules or constraints that define units.
- Partition quality depends on the workload, so a benchmark must fix one.

Directions

- Extend LDBC SNB or LSQB with constraints and rules.
- Agree on metrics: split units, split constraint scopes, inter-partition traversals, query latency.
- Publish partitions and code, not only results.

Research idea: Extend LDBC SNB with a small set of constraints and report split units next to edge-cut and query latency.

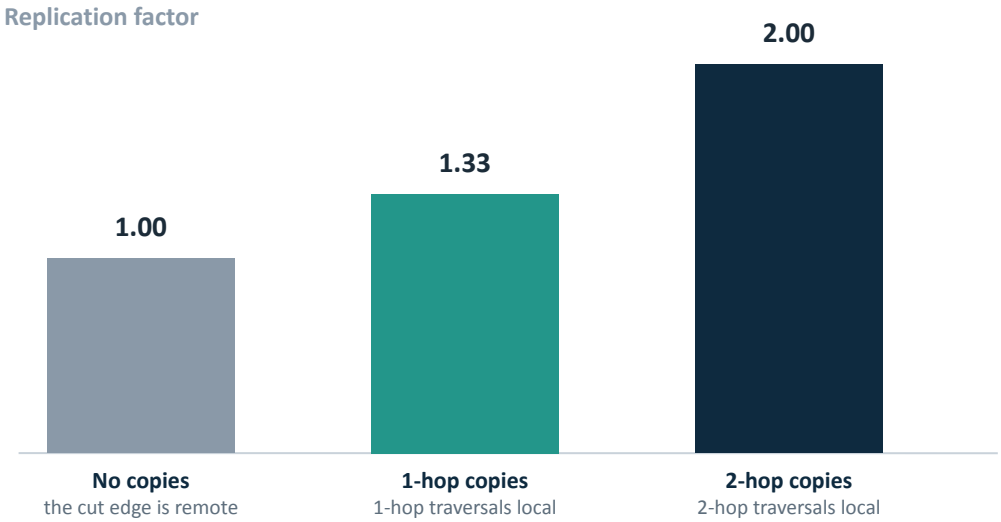
PART P4

Open Challenges

1. **Joint Optimization**
   Unify structural, semantic & dependency objectives in one framework
2. **Dynamic Graphs**
   Keep partitions coherent as the graph changes
3. **Scalable Partitioners**
   Balancing massive graphs at ultra scale
4. **Distributed Reasoning and Validation**
   Scale dependency checking across partitions
5. **Standard Benchmarks**
   Metrics & datasets for constraint-aware partitioning
6. **Replication vs. Locality**
   Buy locality with the smallest replication budget

Replication vs. Locality

How much replication buys how much locality, and where should the copies go?



6 vertices, 2 machines, 1 cut edge

Why it is hard

- Copies cost memory and network, and must be kept consistent on every update.
- Uniform n-hop replication grows quickly with n, especially around high-degree vertices.
- Which copies matter depends on the workload, and the workload changes.
- Copies add load to every machine, so replication interacts with balance.

Directions

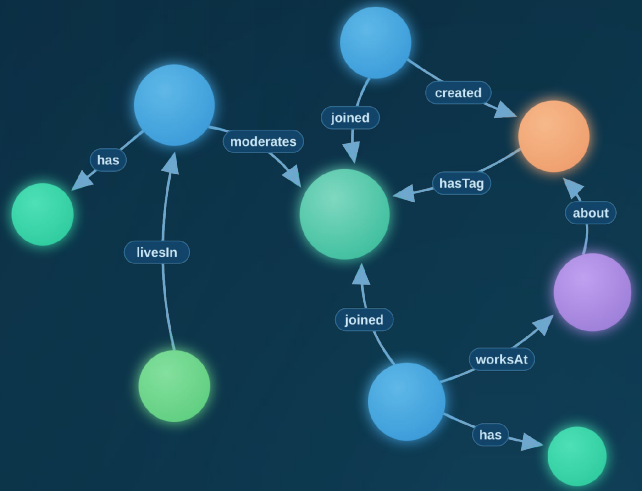
- Replicate what the workload and the schema need, not uniform hops.
- Use the n-hop guarantee as the baseline (Huang, Abadi & Ren 2011).
- Treat replication as a budget chosen together with the partition.

Research idea: Spend the replication budget on the vertices that frequent queries and rules actually cross, not on uniform hops.

ISWC 2026 · HALF-DAY TUTORIAL

Thank You!

Questions & Discussions



E. Ymeralli[1] · G. Troullinou[1] · S. Dumbrava[2] · H. Kondylakis[1]

[1] Institute of Computer Science, ICS-FORTH, UOC, Greece

[2] ENSIIE & SAMOVAR – Télécom SudParis, France

ISWC 2026 · Bari, Italy · 25–29 October 2026